

Agentic Deep Research System

面试技术拷打笔记（TeX 正式版）

面向：技术面试 / 架构深挖 / 治理闭环追问 / 高压交叉盘问

版本：v2.1

日期：2026-07-27

目录

1	怎么使用这份笔记	1
2	第一轮拷打：先把项目说清楚	1
2.1	先用一句话定义这个项目	2
2.2	面试开场应该怎么讲	2
2.3	系统的分层怎么看	2
2.4	为什么这个系统不是简单的 RAG 问答	3
3	第二轮拷打：为什么这么选，而不是别的	3
3.1	Agent 编排框架：为什么选择 LangGraph	4
3.2	LangGraph 和 DAG 的关系	5
3.3	为什么还要 Harness，而不是只靠 LangGraph	6
3.4	为什么要引入 Skill 体系，而不是只拼 Prompt	7
3.5	如果 Skill 规模很大，怎么避免启动慢、匹配慢和运行中断线	8
3.6	LLM 选型：为什么强调结构化输出和工具调用	9
3.7	浏览器自动化：为什么选择 Playwright	10
3.8	为什么要实现三层浏览器提取策略	10
3.9	RAG 与向量库：为什么选择 pgvector	10
3.10	为什么内部检索不是“查一下就回答”	11
3.11	生产追问：QA Agent 和 Deep Search Agent 的 SLO 为什么不同	11
4	第三轮拷打：系统到底怎么跑，状态怎么流	12
4.1	从 API 到报告的完整生命周期	13
4.2	生产追问：这个 Agent runtime 到底是 autonomous loop 还是显式 DAG	13
4.3	公开状态与内部运行状态为什么要分开	14
4.4	ResearchState 为什么是这个系统的核心	14
4.5	Planner 如何生成 DAG，为什么还需要 fallback	15
4.6	DAG 的正确性如何保证	17
4.7	为什么要先 internal RAG，再决定要不要联网	17
4.8	工具节点为什么必须返回显式结果契约	18
4.9	怎样避免单点故障把整条链路拖死	19

4.10	系统会不会从历史失败里吸取教训	20
4.11	Search / Browser / RAG 三类节点如何分工	21
5	第四轮拷打：关键机制是不是你真的做过	21
5.1	Analyst 为什么单独存在	21
5.2	Reflection 为什么不能省掉	22
5.3	Reflection 和传统 reviewer 的区别	23
5.4	RRF 融合检索为什么合理	23
5.5	Checkpoint 的真实价值是什么	23
5.6	为什么 SSE 对这个系统重要	24
6	第五轮拷打：性能、成本和资源怎么扛	24
6.1	为什么并行执行能显著降低时延	25
6.2	Token 优化策略	25
6.3	数据库优化怎么看	26
7	第六轮拷打：你怎么证明它没胡来	27
7.1	我会看哪些指标	27
7.2	Embedding 怎么评估，光看 MTEB 榜单行不行	31
7.3	生产追问：Agent Eval 怎么拆到可定位故障	35
7.4	生产追问：无增益 tool call 怎么量化和治理	36
7.5	生产追问：Chunking 如何避免破坏证据边界	36
7.6	生产追问：多格式文档如何做 canonical representation	37
7.7	生产追问：Embedding、稠密召回和 Rerank 的取舍边界	37
7.8	生产追问：RAG 事故如何定位到 retrieval、ranking 还是 generation	39
7.9	trace 为什么要分成几条线	39
8	第七轮拷打：哪些能吹，哪些不能吹	40
8.1	当前真实完成度应该怎么诚实表达	40
8.2	失败语义为什么是治理第一优先级	41
8.3	会话级预算治理现在到了什么程度	42
8.4	法证级审计现在到了什么程度	43
9	第八轮拷打：邪恶 Agent 怎么管住	44
9.1	治理闭环现状判断	44

9.2	审批状态源 (Approval Source of Truth)	45
9.3	为什么外部审批只能是信号，不能是唯一裁决	46
9.4	Harness 外挂设计	46
9.5	MCP 安全接入原则	47
9.6	生产追问：MCP 协议层、传输层和治理层怎么拆	48
9.7	为什么 Agent 调工具时更容易出事	54
9.8	Harness 工具层怎么挡住越权和带毒参数	55
9.9	五类幻觉怎么治理	57
10	第九轮拷打：高压追问怎么接	57
10.1	为什么选择 LangGraph，而不是直接做多 Agent 对话	58
10.2	为什么动作审批的权威源必须是 DB	58
10.3	为什么外部审批不能直接作为执行依据	58
10.4	为什么要在 LangGraph 外再加 Harness	59
10.5	为什么 MCP 必须先过 policy proxy	59
10.6	预算治理为什么不能只做软提示	59
11	附录：Agent 岗底层推理拷打	60
11.1	底层题：不用框架写 multi-head attention，如何证明 shape 和 mask 没错	60
12	第十轮拷打：优化记录总表与实施证据	62
13	第十一轮拷打：Harness 每一处细节怎么拆	64
13.1	为什么 Harness 不是“再加一个日志文件”	64
13.2	Progress file 为什么是 append-only	65
13.3	Tasks JSON 为什么是 structured state	65
13.4	Active marker 的意义	66
13.5	Lease 为什么是长任务 Agent 的核心字段	66
13.6	为什么恢复边界不能只靠 LangGraph checkpointer	67
13.7	Rollback 为什么不能随便做	67
13.8	Dependency 与 blocked propagation	68
13.9	Stats 为什么是恢复协议的一部分	68
13.10	Harness 面试追问清单	68
14	第十二轮拷打：MCP 协议与治理怎么讲深	69

14.1 MCP 的三层结构	69
14.2 为什么 JSON-RPC 适合 MCP	70
14.3 stdio 与 Streamable HTTP 怎么比较	70
14.4 Tool inputSchema 为什么必须落到执行前	70
14.5 为什么 tool annotations 不能当安全依据	71
14.6 MCP authorization 怎么回答	71
14.7 MCP 面试追问清单	71
15 第十三轮拷打: Agent 开发岗位底层能力	72
15.1 平台基础设施线	72
15.2 应用/RAG 研究线	73
15.3 安全治理线	74
15.4 Agent 岗高压题库: 平台基础设施	74
15.5 Agent 岗高压题库: 应用与 RAG	76
15.6 Agent 岗高压题库: 安全治理	77
16 第十四轮拷打: Skill 体系深挖	78
16.1 为什么 Markdown + YAML frontmatter	78
16.2 Progressive disclosure 在 Skill 里的含义	78
16.3 Scope 为什么不能被 manual enable 绕过	79
16.4 Snapshot 为什么是 Skill 运行时的关键	79
16.5 Skill 和 MCP 的关系	79
16.6 Skill 面试追问清单	79
17 第十五轮拷打: 优化路线图怎么讲不虚	80
18 第十五轮补充: 行业调研来源与选型记录	81
18.1 外部资料对当前设计的校准	81
18.2 Harness 选型记录	83
18.3 MCP 选型记录	84
18.4 Agent 架构选型记录	85
18.5 Skill 选型记录	85
18.6 本轮实现 TODO 与验收证据	86
19 第十七轮拷打: 继续优化还能补哪些细节	87

19.1 本轮发现的可优化点	87
19.2 选型对比与决策	88
19.3 本轮落地证据	89
19.4 下一阶段可扩展功能清单	90
20 第十八轮拷打：高级工程师故障注入题	91
20.1 事故 1：审批回调重复到达，写工具会不会执行两次	91
20.2 事故 2：MCP server 返回的 structuredContent 结构错了怎么办	91
20.3 事故 3：两个 worker 同时写 Harness 状态，checkpoint 丢了怎么办	92
20.4 事故 4：Skill 手动启用绕租户 scope 怎么办	92
20.5 事故 5：RAG 答案错了，如何定位不是一句“模型幻觉”	93
20.6 事故 6：预算硬熔断发生在证据不足时，如何避免粗暴失败	93
20.7 事故 7：Prompt injection 藏在网页正文里，为什么 schema 仍不够	93
21 第十六轮拷打：Agent 开发岗单页题卡	94
21.1 题卡 1：Agent 和普通 LLM 应用的分水岭是什么	95
21.2 题卡 2：为什么 Agent 系统的核心是状态机	96
21.3 题卡 3：DAG 和 StateGraph 有什么区别	97
21.4 题卡 4：为什么 Planner 不能直接写最终答案	98
21.5 题卡 5：为什么需要 fallback DAG	99
21.6 题卡 6：为什么要区分 public status 和 runtime status	100
21.7 题卡 7：retryable failed 和 terminal failed 怎么区分	101
21.8 题卡 8：Agent 为什么需要 failure memory	102
21.9 题卡 9：为什么预算治理必须是硬控制	103
21.10题卡 10：为什么 tool call 是高风险执行面	104
21.11题卡 11：为什么不能相信模型自己做权限判断	105
21.12题卡 12：为什么工具参数必须 schema 校验	106
21.13题卡 13：为什么工具返回也要治理	107
21.14题卡 14：为什么审批要以 DB 为 source of truth	108
21.15题卡 15：审批恢复最容易出什么 bug	109
21.16题卡 16：为什么 SSE 不是任务事实源	110
21.17题卡 17：为什么 trace 和 audit 要分开	111
21.18题卡 18：为什么 Agent eval 不能只看答案分数	112

21.19题卡 19: 为什么 RAG 不能替代 Browser	113
21.20题卡 20: 为什么 citation 是研究型 Agent 的生命线	114
21.21题卡 21: 为什么 Browser 内容要当作不可信数据	115
21.22题卡 22: 为什么 internal-first 是治理策略	116
21.23题卡 23: 为什么 replan 要带 failure hints	117
21.24题卡 24: 为什么多 Agent 对话不适合强流程研究	118
21.25题卡 25: 为什么需要 tool allowlist	119
21.26题卡 26: 为什么 Skill 不是 prompt 附件	120
21.27题卡 27: 为什么 Skill 要 top-k	121
21.28题卡 28: 为什么 Skill snapshot 能防止运行漂移	122
21.29题卡 29: 为什么 MCP 和 Skill 不能混为一谈	123
21.30题卡 30: 为什么 MCP 必须有 server registry	124
21.31题卡 31: 为什么 MCP tool schema 不能完全信 server	125
21.32题卡 32: 为什么 result hash 有价值	126
21.33题卡 33: 为什么要区分 schema invalid 和 tool not allowed	127
21.34题卡 34: 为什么 write tool 默认审批	128
21.35题卡 35: 为什么 summary length 也是安全策略	129
21.36题卡 36: 为什么 long-running Agent 要 session config	130
21.37题卡 37: 为什么 active marker 要在 idle 时清理	131
21.38题卡 38: 为什么 JSON 损坏不能静默忽略	132
21.39题卡 39: 为什么恢复时不能只看最后一行日志	133
21.40题卡 40: 为什么多 worker 必须独立工作树	134
21.41题卡 41: 为什么 Agent 需要 idempotency	135
21.42题卡 42: 为什么 model fallback 要保护 schema	136
21.43题卡 43: 为什么不能把 eval 只做成离线脚本	137
21.44题卡 44: 为什么 evidence 和 answer 要分开存	138
21.45题卡 45: 为什么浏览器自动化需要多层降级	139
21.46题卡 46: 为什么 Search 只是入口不是证据终点	140
21.47题卡 47: 为什么 Reflection 不应无限循环	141
21.48题卡 48: 为什么报告要写不确定性	142
21.49题卡 49: 为什么 Agent 需要 human-in-the-loop	143
21.50题卡 50: 为什么“暂停”不等于“失败”	144
21.51题卡 51: 为什么工具错误要净化	145

21.52题卡 52: 为什么 DB 是预算事实源	146
21.53题卡 53: 为什么 Postgres + pgvector 适合这个项目	147
21.54题卡 54: 为什么结构化输出很重要	148
21.55题卡 55: 为什么 prompt injection 不是只靠 prompt 解决	149
21.56题卡 56: 为什么 Agent 需要最小权限	150
21.57题卡 57: 为什么“会调用工具”不等于“会安全调用工具”	151
21.58题卡 58: 为什么要记录 server fingerprint	152
21.59题卡 59: 为什么 untrusted server 要直接拒绝	153
21.60题卡 60: 为什么 schema invalid 不应该自动重试同样参数	154
21.61题卡 61: 为什么报告生成要和取证解耦	155
21.62题卡 62: 为什么 Agent 需要可解释路由	156
21.63题卡 63: 为什么数据源可信度要分层	157
21.64题卡 64: 为什么检索命中不代表可以回答	158
21.65题卡 65: 为什么 Browser 节点要受预算控制	159
21.66题卡 66: 为什么 search query 要由 Planner 约束	160
21.67题卡 67: 为什么要记录 current batch	161
21.68题卡 68: 为什么 completed nodes 不能只靠模型声明	162
21.69题卡 69: 为什么 Agent 需要停止条件	163
21.70题卡 70: 为什么“部分报告”是优雅降级	164
21.71题卡 71: 为什么 Agent 需要配置化风险等级	165
21.72题卡 72: 为什么 tool timeout 是治理而不是性能参数	166
21.73题卡 73: 为什么 Agent 需要环境初始化	167
21.74题卡 74: 为什么测试输出要为 Agent 设计	168
21.75题卡 75: 为什么不能只用日志做恢复	169
21.76题卡 76: 为什么 DB 和 Harness 会有双写问题	170
21.77题卡 77: 为什么工具审计不能只存在 session JSON	171
21.78题卡 78: 为什么 Agent 需要 observability	172
21.79题卡 79: 为什么 prompt 也要版本化	173
21.80题卡 80: 为什么 Agent 需要 schema repair	174
21.81题卡 81: 为什么“工具越多越强”是误区	175
21.82题卡 82: 为什么上下文窗口不是可靠记忆	176
21.83题卡 83: 为什么 Agent 要记录 artifacts	177
21.84题卡 84: 为什么用户确认和审批不同	178

21.85题卡 85: 为什么要把 source of truth 讲清楚	179
21.86题卡 86: 为什么 Agent 安全要做纵深防御	180
21.87题卡 87: 为什么要做红队样本库	181
21.88题卡 88: 为什么 Agent 系统要有降级路径	182
21.89题卡 89: 为什么“完成”要有验收证据	183
21.90题卡 90: 为什么要把未完成写进笔记	184
21.91题卡 91: 为什么 Agent 需要任务优先级	185
21.92题卡 92: 为什么 dependency cycle 要检测	186
21.93题卡 93: 为什么 blocked 状态不一定要持久化	187
21.94题卡 94: 为什么 approval request 要有 request payload	188
21.95题卡 95: 为什么工具层要返回统一结构	189
21.96题卡 96: 为什么 Agent 要有 query scope	190
21.97题卡 97: 为什么 Agent 需要成本归因	191
21.98题卡 98: 为什么重试要有 backoff	192
21.99题卡 99: 为什么 tool result 不能直接作为最终答案	193
21.10题卡 100: Agent 平台的控制面和数据面怎么拆	194
21.10题卡 101: 为什么要区分工具发现和工具授权	195
21.10题卡 102: 为什么 Agent 要有审计友好的错误码	196
21.10题卡 103: 为什么上下文压缩会影响恢复	197
21.10题卡 104: 为什么 Agent 要可暂停	198
21.10题卡 105: 为什么 Agent 需要边界表达	199
22 生产级韧性与安全重构: 设计说明与面试答法	199
22.1 可退化架构: 从全有或全无变成健康、降级、恢复	200
22.2 安全边界: 数据归属、令牌生命周期与默认拒绝	201
22.3 配置与密钥: 按主体隔离, 并讲清威胁模型	201
22.4 兼容层与 facade: 重构不应一次性破坏调用面	202
22.5 统一 LLM adapter: 把 provider 差异隔离在边界	202
22.6 提示注入: 把不可信数据放进数据流, 而不是一句 system prompt	203
22.7 健康检查: liveness 与 readiness 必须分开	203
22.8 上传与文档生命周期: 不是一个 save(file)	203
22.9 备份导出/导入: merge-only 仍然要有事务边界	204
22.10本地持久化: JSON 也要遵守数据库式原子写	204

22.11测试文化：每个真实事故都留下可回归的名字	204
22.12不在本轮范围的后台自动化与未来多主体	205
22.13最终取舍总表	205
23 附录：当前仓库现实、边界与演进路线	206
23.1 当前已核实的现实结论	206
23.2 当前仍存在的关键边界	206
23.3 建议的三阶段演进	207
23.4 最终的面试总结句	207

1 如何使用这份笔记

这份文档不是项目报告

这份文档的用途不是把项目按模块介绍一遍，而是训练一种更适合技术面试的表达方式：面试官会先确认你是不是自己做过，再追问你为什么这么选，接着看你能不能把边界、缺陷、治理和演进路线讲清楚。

建议的回答顺序

回答时不要一上来就堆框架名。最稳的顺序是：先定义系统解决什么问题，再讲主链路怎么跑，再讲为什么这么选，再讲当前已经做到哪里，最后讲哪些地方还没做完、准备怎么补。这样既像工程师，也不像背稿。

使用方法

这份笔记更适合按“结论、深入分析、边界”三层来准备。先用一两句话给出立场，再补业务抽象、替代方案、代价和故障模式，最后明确哪些能力已经落地、哪些仍有边界。面试官真正想听的通常不是定义本身，而是你是否能在压力下把判断讲完整。

这份笔记的使用原则

第一，不要把设计稿说成已上线功能。第二，不要把历史 bug 说成当前仍存在的问题。第三，不要把治理愿景说成生产级闭环。第四，遇到面试官追问时，优先讲事实、边界和取舍，不要用空话兜底。

2 第一轮拷打：先把项目说清楚

这一轮面试官在试什么

这一轮不是考你会不会背名词，而是在判断你能不能用两到三分钟把系统定义准确。面试官真正想听的是：这到底是不是一个严肃的研究型系统，还是只是把搜索、RAG 和报告拼到一起的 Demo。

这一轮最常见的追问

- 这个项目一句话到底是什么，不要给我讲一堆模块名。
- 它和普通 RAG、普通搜索问答、普通 workflow 引擎到底差在哪。
- 你这个系统的输入、处理中间态、输出结果分别是什么。
- 如果我只给你两分钟，你最希望我记住这个项目的哪三个关键词。

2.1 先用一句话定义这个项目

一句话总述

这是一个以 LangGraph 为业务编排内核、以 Search / Browser / RAG 为取证能力、以 Analyst / Reflection / Report 为分析闭环、并逐步向治理型 Agent 演进的研究系统。

2.2 面试开场应该怎么讲

推荐开场

如果面试官先问“这个系统是干什么的”，最稳的讲法不是直接说“这是一个 AI Agent”，而是先把它定义成一个 **evidence-driven deep research workflow**。它的核心目标不是闲聊，不是通用办公自动化，而是围绕“搜索、取证、对比、分析、引用、报告”构造一条可重复执行的研究链路。

我通常会用下面这段话开场：

标准开场答案

这个系统的目标是把用户问题转换成一张可执行研究图。Planner 先把问题拆成 DAG，Search / Browser / RAG 节点分别去互联网、网页和内部知识库取证，Analyst 负责综合证据形成结构化分析，Reflection 负责做事实、一致性和引用覆盖率校验，最后 Report 输出带引用的研究报告。系统的设计重点不是单次回答得多花哨，而是证据链、失败语义、预算控制和后续治理扩展能力。

2.3 系统的分层怎么看

- 编排层：LangGraph StateGraph，负责节点、条件边、循环和并行。
- 能力层：Planner / Search / Browser / RAG / Analyst / Reflection / Report。
- 状态层：ResearchState、DAG、checkpoint、trace、tool audit、budget state。
- 数据层：PostgreSQL + pgvector，存文档、session、citation、tool audit、budget state。
- 交互层：FastAPI + SSE，负责创建任务、查询状态、流式推送进度。
- 治理层：guardrail、runtime status、budget breaker、approval source of truth、Harness supervisor、MCP proxy。

2.4 为什么这个系统不是简单的 RAG 问答

核心差异

RAG 问答通常是“检索一轮，再回答一轮”。这个系统做的是“先规划，再多源取证，再综合分析，再反思校验，再决定是否重规划”。它把单次回答升级成了一条带状态、带条件分支、带恢复与治理约束的研究流水线。

深入分析

这道题不能只停在“步骤更多”。面试官真正想确认的是，你做的到底是一个回答器，还是一个研究执行系统。普通 RAG 的默认假设是：问题边界比较清楚，检索一次拿到上下文后，模型就可以直接组织答案。所以它的核心矛盾是“召回不准、答案顺不顺”。而研究型 Agent 的假设完全不同，它默认问题可能被拆错、证据可能互相冲突、某个来源可能暂时不可用、预算可能不允许无限扩张、以及第一次答案可能需要被系统自己否掉重做。

因此两者的差异不只是模块数量，而是执行模型。简单 RAG 的主对象是“上下文窗口”，研究型系统的主对象是“任务状态机”。前者关注把哪些文档塞进 prompt，后者关注当前任务处于哪一阶段、还有哪些节点没跑、失败是可重试还是终止、是否允许继续联网、是否需要人工审批、是否还有预算继续扩张。只要把主对象换掉，系统设计就会从“检索增强回答”自然演进成“可治理的研究流程”。

这一点在面试里必须讲透，因为很多人会把 Search、Browser、RAG、Report 都堆在一起，自称做了 Agent。真正的分水岭不是工具数量，而是是否有显式状态、显式路由、显式失败语义和显式治理约束。你要把这层抽象抬出来，面试官才会认为你不是在做功能拼装。

3 第二轮拷打：为什么这么选，而不是别的

这一轮面试官在试什么

当你把系统讲清楚之后，面试官通常会立刻追问选型理由。这里最怕两种答法：一种是“社区都这么用”，另一种是“我觉得这个更先进”。正确答法必须同时包含备选方案、比较维度和最终取舍。

这一轮最容易翻车的地方

不要把“能用”讲成“唯一正确”。也不要回避被否选方案，因为真正做过工程的人一定经历过权衡，而不是从一开始就知道唯一答案。

这一轮最常见的追问

- 为什么是 LangGraph，不是 LangChain、AutoGen、CrewAI，或者你自己写调度器。
- 为什么浏览器自动化选 Playwright，不是 Puppeteer 或 Selenium。
- 为什么检索层放 pgvector，不直接用专门向量库。
- 为什么还要 Harness，说明 LangGraph 还不够，那你一开始为什么还选它。

3.1 Agent 编排框架：为什么选择 LangGraph

结论

研究任务本质上不是线性 Chain，而是带并行、条件分支和重规划循环的状态机，所以选择 LangGraph，而不是只用 LangChain 的串联式抽象。

维度	LangGraph	LangChain	AutoGen	CrewAI
核心抽象	状态机 / 图编排	链和工具集成层	对话式多 Agent	角色协作脚本
并行与 fan-out	原生 Send API	需手动拼接	可做但偏消息流	较弱
条件边	强	中	中	弱
循环与重规划	强	弱	中	弱
检查点恢复	支持	有限	有限	弱
适合研究 DAG	很适合	一般	偏实验	偏演示

选择理由

这里至少有两个可行方案。方案 A 是继续用 LangChain，把 Planner、Search、Report 串起来，再自己维护一套 if-else。优点是简单，缺点是并行、循环、恢复都会越来越别扭。方案 B 是直接把研究流程建模成一张图，让节点、边、状态、路由都显式化。最终选择 LangGraph，就是选择方案 B。

面试回答模板

LangGraph 解决的是“复杂多步骤任务如何以显式状态机方式执行”。研究型任务天然需要 DAG、并行、条件边和循环重规划，这四点 是 LangGraph 的强项。LangChain 很适合做工具和模型的胶水层，但不适合单独承担复杂业务 workflow 编排。

深入分析

这一题真正的考点不是“你知不知道 LangGraph”，而是“你有没有把业务问题正确映射到执行模型上”。如果业务天然线性的，比如固定三步：检索、总结、输出，那么 LangChain 的链式抽象完全够用，甚至更轻。但研究型任务不一样，它会同时出现三类复杂性。第一类是并行性，你希望 search、browser、rag 在某些阶段同时跑，而不是人为串起来浪费 I/O 时间。第二类是条件性，你要根据 RAG 命中情况决定是否继续联网，根据 Reflection 结果决定是否 replan，根据审批结果决定是否恢复。第三类是循环性，你不能假设第一次规划一定正确，所以必须给重规划留合法路径。

如果用普通 chain 硬做这些事，短期能跑，长期会出现两个工程问题。一个问题是状态散落。你会把当前步骤、已完成节点、预算、失败次数、审批状态藏在 if-else 和局部变量里，导致一旦出 bug 很难解释“系统为什么走到了这里”。另一个问题是恢复困难。因为链式抽象默认关注的是一步接一步的推理，而不是一个可中断、可恢复、可审计的长期工作流。

所以这里选择 LangGraph，本质上不是“选一个更潮的框架”，而是承认这个系统从第一天起就是状态机问题，而不是提示词编排问题。面试时如果你能把这层抽象对齐讲出来，可信度会比单纯报框架名高很多。

3.2 LangGraph 和 DAG 的关系

追问答案

DAG 是业务层面的研究计划图，StateGraph 是执行层面的工作流图。Planner 生成的 DAG 解决“研究步骤怎么拆”，LangGraph StateGraph 解决“这些步骤怎样被调度、聚合、校验、循环和恢复”。DAG 是数据，StateGraph 是执行器。

深入分析

这里最容易犯的错，是把 DAG 和 LangGraph 讲成同一个东西。实际上它们回答的是两个不同问题。DAG 是“这次研究任务需要做哪些事、谁依赖谁”，本质上是业务计划；LangGraph StateGraph 是“这些计划节点在运行时怎么被驱动、怎么聚合结果、怎么决定下一步”，本质上是执行引擎。一个是业务数据结构，一个是控制流框架。

为什么要分开？因为业务计划和执行控制的变化频率不同。Planner 今天可能多生成一种节点类型，或者把 Web 节点拆得更细，这属于 DAG 层的变化；但超时重试、审批暂停、反思重规划、预算熔断这类机制，不应该每次都由 Planner 直接负担，否则模型输出一旦不稳定，执行层就会跟着失控。把 DAG 当数据、把 StateGraph 当执行器，实际上是在保护控制面不被 LLM 计划输出直接污染。

还有一个关键收益是可解释性。如果 DAG 是数据，你就能清晰地把“计划是否合理”和“执行是否正确”拆开分析。前者看节点设计、依赖关系、fallback 是否健全；后者看路由、失败语义、恢复、预算和审计是否可靠。面试里把这两个层

次拆开，会显得你对系统建模有清晰边界感。

3.3 为什么还要 Harness，而不是只靠 LangGraph

方案	做法	优点	缺点
方案 A	生命周期、重试、审批、恢复都塞进 LangGraph	单层编排、概念简单	图会越来越重，治理逻辑和业务逻辑耦合
方案 B	LangGraph 管业务图，Harness 管 supervisor	治理边界清晰，适合长任务	多一层状态同步与恢复控制

最终选择

当前项目的长期正确方向是方案 B：LangGraph 负责业务编排，Harness 负责任务级控制面。因为你真正要治理的是“失败后怎么恢复、预算超了怎么刹车、审批后怎么恢复、worker 怎么抢租约”，这些都不是业务 DAG 最擅长表达的东西。

深入分析

这一题最容易答浅。很多人会说“LangGraph 不够，所以再加 Harness”，这句话不够严谨。更准确的说法是：LangGraph 擅长描述业务执行图，但不擅长承担完整的任务级控制面。业务执行图关心的是当前任务应该怎么做，supervisor 关心的是当前任务该不该继续做、由谁做、从哪里恢复做、预算是否允许做、审批是否允许做。两者不是强弱关系，而是职责分层关系。

如果把 supervisor 语义硬塞进图里，短期看起来只有一层系统，长期会有三个副作用。第一，图的节点语义会被污染。原本 analyst、reflection、report 这些节点是在表达业务，而不是在表达租约、暂停、恢复、超时、人工确认。第二，图会变得很难演进。因为每加一个治理策略，你都要再给图加分支、加状态、加条件，最后图就不再是业务图，而是一个混杂业务与控制的巨型控制器。第三，恢复语义会失真。任务级恢复往往需要依赖外部状态源，例如 harness 文件、worker 租约、checkpoint 版本，而这些都不是图内局部状态能优雅表达的。

所以外置 Harness 的关键价值在于分域定权：图内负责“怎么执行”，图外负责“是否继续执行、如何恢复执行”。这一点如果讲透，面试官会更容易相信你不是在“为了多一个名词而多一个组件”，而是在主动控制系统复杂度的来源。

3.4 为什么要引入 Skill 体系，而不是只拼 Prompt

方案	做法	优点	缺点
方案 A	直接把一段 Markdown 或 system prompt 拼到 Planner 前面	实现快、改动小	只影响文案，不是体系对象，无法约束工具和 session
方案 B	把 skill 解析成可注册对象，进入 session、planner、agent、tool allowlist	能真正纳入体系，可解释、可审计、可热更新	需要额外的 parser、registry、CRUD 和状态同步

最终选择

如果目标只是“给模型多一点领域提示”，方案 A 就够了；但如果目标是“把领域能力做成系统级扩展点”，就必须选方案 B。当前项目已经按方案 B 落地：skill 以 Markdown + YAML frontmatter 形式存储，启动时加载进 runtime registry，session 会自动匹配并允许手动启用/禁用，最终同时影响 Planner、后续 Agent 提示和 tool allowlist。

深入分析

这道题真正考的不是“你会不会做配置文件”，而是“你有没有把扩展能力做成一等系统对象”。如果只是把 skill 文件读出来，然后原样拼到 system prompt 里，它本质上还是一段文本，不是一个可治理对象。这样做最大的三个问题是：第一，session 不知道自己到底启用了哪些领域能力；第二，工具层不知道哪些 skill 允许哪些工具；第三，前端和审计层无法解释某次执行为什么走了这条策略。

至少有两个可行方案。方案 A 是 prompt-only，把 skill 当成额外文案。这种方式的优点是速度快、侵入性低，但它不会改变系统结构，最终也无法形成真正的 CRUD、热更新和 session 级开关。方案 B 是 registry 模式：skill 有稳定元数据、触发规则、允许工具、agent hints 和正文内容，运行时先做匹配，再把结果下沉到 session state。这样 Planner 可以按 skill 改写研究策略，Analyst / Reflection / Report 可以按 skill 加领域提示，工具层也能按 skill 收缩能力边界。这里必须选方案 B，因为“真正纳入体系”的定义，不是能被读出来，而是能影响调度、提示和权限。

为什么 skill 文件最终选择 Markdown + YAML frontmatter？因为这里同时有两类诉求：一类是机器可校验的稳定字段，例如 name、slug、trigger_patterns、allowed_tools；另一类是人类可编辑的正文说明，例如 overview、prompt、constraints。纯 JSON / YAML 对非工程用户不够友好，纯 Markdown 分节又太脆弱，frontmatter 则刚好把“结构化元数据”和“可读正文”分开。面试里如果你把这个点讲明白，面试官会知道你不是只在堆配置，而是在做一个面向运行时和面向人类编辑都可接受的能力层抽象。

3.5 如果 Skill 规模很大，怎么避免启动慢、匹配慢和运行中断线

结论

海量 skill 的最优解，不是把所有 skill 全量读进内存，而是做成一套 meta 常驻、content 延迟、匹配分层、session 固化、版本化回收的链路。这样既能处理冷启动和检索精度，也能保证 skill 更新或卸载后，运行中的 session 不断线。

方案	做法	优点	缺点
方案 A	启动时全量加载所有 skill 正文并全量扫描匹配	实现简单	冷启动慢、内存重、热更新抖、难扩展
方案 B	meta/content 分层、候选渐进式匹配、正文按需加载、session snapshot 固化	冷启动和精度可平衡，运行时稳定	实现复杂度更高
方案 C	外部检索服务化，registry 只做路由	超大规模扩展性最好	过早分布式，运维成本高

最终选择

当前最合理的是方案 B。因为它不要求一开始就把 skill 服务化，但已经能解决三个真实问题：第一，启动时不再依赖全量正文加载；第二，session 匹配时不再对所有 skill 做重规则扫描；第三，skill 更新和卸载后，新老 session 可以稳定分流，而不会把正在运行的任务打断。

深入分析

这个问题表面是在问“怎么做大规模加载”，但真正有三个耦合点。第一个是检索精度。如果你为了快，粗筛得太狠，就会漏掉真正该命中的 skill；如果你为了准，对所有 skill 做正则、规则和正文语义扫描，又会把延迟打爆。第二个是冷启动。如果启动阶段必须把所有 skill 的全文、正则和提示都预热进内存，实例扩容和服务重启会非常慢。第三个是状态连续性。如果运行中的 session 还依赖全局 registry 的当前态，那么 skill 一旦被更新、禁用或删除，就可能让同一个 session 在执行中前后行为漂移。

所以最优解必须同时回答这三件事。第一，在存储上做 meta/content 分离。meta 层只保留匹配和控制所需的轻字段，例如 slug、tenant、enabled、priority、keywords、allowed tools、agent hint 摘要；content 层保存完整 Markdown、解析后的 prompt 片段和较大的说明文本。启动时只加载 meta，不加载正文。第二，在匹配上做渐进式披露。先按 tenant、project、scope、enabled 做硬过滤，再用关键词、tag、domain 或全文索引做粗筛，把候选集从上万压到几十；然后只对候选集做正则、复杂 trigger rule 和更精细的策略合并；最后只为 top-N 命中的 skill 加载正文。这里“渐进式披露”不是前端折叠，而是计算成本和信息量逐级展开。第三，在运行态上做 session 固化。session 创建时生成 resolved skill snapshot，把 skill_id、

version、matched reason、effective tools、prompt fragments 都固化到会话里。之后 Planner、Analyst、Report 和 tool allowlist 全都只读这个 snapshot，而不是反复回查全局 registry。

这套设计为什么能解决 skill 卸载后不断线？关键在于版本固定和延迟回收。skill 更新不是原地覆盖，而是新版本；skill 卸载不是立刻把正文硬删，而是让它不再参与新 session 匹配。正在运行的 session 因为已经绑定了特定 version 的 snapshot，所以会继续沿用原版本执行。只有当没有 active session 再引用这个版本时，系统才真正回收正文内容。这就把“新流量不可见”和“旧流量不断线”分开了。

再往深一点说，海量 skill 的最大陷阱其实不是加载，而是 prompt 膨胀。即使你匹配很快，如果最后把二十几个 skill 的长正文全拼进 Planner 和 Report 的 prompt，LLM 侧还是会失控。所以真正成熟的系统还会再加一层约束：只让 top-N skill 进入执行，或把同类 skill 合并成摘要，并给 skill 注入本身设置 token budget 上限。面试里如果能把“匹配快”和“提示可控”一起讲出来，可信度会高很多。

3.6 LLM 选型：为什么强调结构化输出和工具调用

选型标准

这个系统里的模型不是单纯负责写文案，它要做 DAG 生成、JSON 结构化输出、事实校验、引用覆盖率分析和报告生成，所以第一优先级不是“最会聊天”，而是“最稳定地产生结构化结果”。

维度	倾向项	原因	对本项目的影响
工具调用稳定性	高优先级	Planner / Reflection 都依赖结构化结果	降低解析失败和 fall-back 频率
中文能力	高优先级	用户查询、网页、报告以中文为主	减少跨语言语义损耗
成本	高优先级	深研究任务 token 消耗天然高	决定是否能高频运行
通用推理上限	中优先级	重要，但不是唯一约束	可由反思、检索和多轮流程补强

面试时的稳妥表述

我对模型选型的原则不是盲目追逐最强模型，而是优先看三个维度：结构化输出是否稳定、工具调用是否稳定、中文与成本是否平衡。因为在研究型 Agent 里，模型失败往往不是“文风不够好”，而是 JSON 不可解析、校验结果不稳定、成本高到没法长期运行。

3.7 浏览器自动化：为什么选择 Playwright

维度	Playwright	Puppeteer	Selenium	结论
异步模型	原生 async	原生 async	传统同步为主	Playwright 更适合 Python async 服务
自动等待	强	中	弱	减少页面交互竞态
多浏览器支持	Chromium/FF/WebKit	仅 Chromium	强	Playwright 更平衡
工程稳定性	高	中	中	对研究型抓取更合适

核心回答

我选 Playwright 的原因不是“它比较新”，而是它更贴合 Browser Use 场景。研究型浏览器节点经常面对动态加载、延迟渲染、滚动加载和反检测问题，Playwright 的自动等待、页面生命周期控制和无头执行能力更容易形成稳定工程实现。

3.8 为什么要实现三层浏览器提取策略

设计理由

不是所有页面都值得深度提取。浏览器节点如果默认把整页内容吃满，会直接推高 token 消耗、拉长时延、放大噪音。所以设计成 snippet / skim / deep 三层提取，本质上是把内容质量和成本一起治理。

- snippet: 优先抓 meta 和少量核心文本，适合快速判断页面是否有价值。
- skim: 抓主要段落，适合普通文章和资讯。
- deep: 抓更多标题、段落、列表、结构化片段，适合权威报告和高价值页面。

3.9 RAG 与向量库：为什么选择 pgvector

维度	pgvector	Qdrant	Chroma	Pinecone
部署复杂度	低	中	低	SaaS
事务一致性	强	弱	弱	弱
和业务表 JOIN	方便	较弱	较弱	不方便
全文搜索协同	强	需外部系统	弱	需外部系统
运维成本	低	中	低	高

最终选择

我选 pgvector，不是因为它在纯向量召回上一定最强，而是因为它对这个项目的整体工程复杂度最低。文档、元数据、citation、research session、全文索引和向量索引都在同一套 PostgreSQL 里，事务一致性和数据联查能力非常强。

深入分析

这里的关键不是“向量数据库哪个好”，而是“这个项目最需要优化的到底是哪一层”。如果你的系统是一个高吞吐、超大规模、纯向量召回优先的检索系统，那专用向量库可能更合理。但当前这个项目不是以 ANN 性能极限为第一目标，它更像一个研究工作流系统。研究工作流系统最常见的痛点不是“向量召回每秒少了几百次”，而是“文档、会话、引用、审计、预算、审批这些状态分散在多套存储里，最后查一次完整证据链要跨很多系统”。

pgvector 的优势正体现在这里。第一，它允许文档正文、结构化元数据、全文索引、向量索引、session 记录共库，从而把一致性问题压到最低。第二，很多研究场景并不是只靠向量召回，往往还要混合关键词搜索、过滤 group/source、回查 citation 元数据，PostgreSQL 在这些组合查询上非常顺手。第三，运维心智负担更低。对于一个还在快速演进治理闭环的系统来说，减少基础设施种类往往比追求单点最优性能更重要。

这题如果只回答“因为部署方便”会太浅。更好的讲法是：我不是在做一个独立的向量检索产品，我是在做一个带审计、带引用、带任务状态的研究系统，所以我优先优化的是系统整体复杂度，而不是某一个检索子模块的局部极值。

3.10 为什么内部检索不是“查一下就回答”

核心区分

内部 RAG 在这个系统里不是回答器，而是证据源。它负责给出可引用片段、提供内部知识背景、帮助决定是否还要联网搜索，但最终回答仍然要经过 Analyst、Reflection 和 Report 链路。

3.11 生产追问：QA Agent 和 Deep Search Agent 的 SLO 为什么不同

工程结论

QA 业务型 Agent 的 SLO 是“确定边界内稳定、低时延、可控地答对”；Deep Search Agent 的 SLO 是“开放问题下覆盖充分、证据可追溯、冲突可解释”。这不是产品名差异，而是状态机、评估集、预算和失败语义都不同。

放回 DeepIntel 怎么讲

这题如果脱离项目讲，很容易变成抽象概念题；放回 DeepIntel 会清楚很多。DeepIntel 更接近 Deep Search Agent，而不是单轮 QA Bot。它不是拿一个 query 直接查知识库然后输出，而是先由 Planner 拆成 DAG，再让 Search / Browser / RAG 分别去不同证据源取材，最后由 Analyst 做证据综合、Reflection 做质量校验，必要时还会 replan。也就是说，它的核心不是“快答”，而是“可验证地研究”。

至少有两种做法。方案 A 是做 QA 业务型 Agent：固定意图分类、固定工具路由、固定回答模板，优点是时延稳、维护成本低、SLA 好做，适合客服问答、内部知识问答、流程查询；缺点是开放问题时容易覆盖不足。方案 B 是做 Deep Search Agent：允许多跳检索、网页深读、多源比对和反思重规划，优点是复杂问题上更有上限，缺点是成本、时延和治理复杂度都更高。DeepIntel 选的是方案 B，因为它要解决的是“研究型任务”，而不是“FAQ 问答”。

面试时最稳的落点是：不要把两类 Agent 讲成谁替代谁，而要讲成优化目标不同。QA 业务型 Agent 更像高频、确定边界、强 SLA 的生产答题器；Deep Search Agent 更像低频、开放问题、证据优先的研究员。DeepIntel 当前主骨架明显站在后者这一侧。

4 第三轮拷打：系统到底怎么跑，状态怎么流

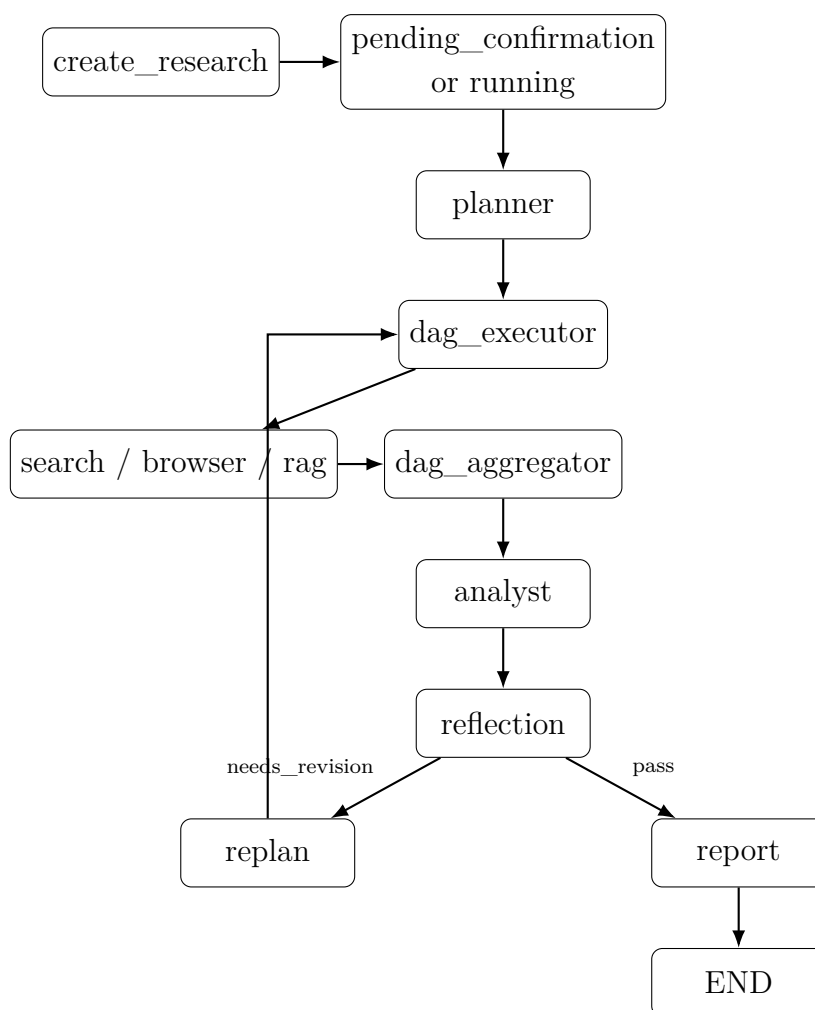
这一轮面试官在试什么

这一轮会从“你为什么这样设计”切到“这个系统运行时到底发生了什么”。如果你只能讲模块名，讲不清状态怎么变化、节点怎么路由、失败怎么传播，面试官会判断你更像读过代码，而不是做过系统。

这一轮最常见的追问

- 一个任务从 API 创建到报告产出，中间经过哪些确定的节点。
- Planner 产出的 DAG 和 LangGraph 本身是什么关系。
- public status 和 runtime status 为什么要分开，不分会出什么问题。
- 工具节点失败以后，状态怎么传给聚合器，聚合器怎么决定下一步。

4.1 从 API 到报告的完整生命周期



现状说明

当前生命周期是: `create_research` 建 session, 状态先进入 `pending_confirmation` 或 `running`, 随后后台运行 `LangGraph: planner -> dag_executor -> search/browser/rag -> dag_aggregator -> analyst -> reflection -> replan/report -> END`。这条主链路已经打通, 是系统最核心的业务骨架。

4.2 生产追问: 这个 Agent runtime 到底是 autonomous loop 还是显式 DAG

工程结论

DeepIntel 不是多 Agent 自由聊天, 也不是单体 `autonomous loop`, 而是 Planner 产显式 DAG, `LangGraph` 按依赖驱动 `Search / Browser / RAG / MCP` 节点, `Harness` 和 `policy guardrail` 在图外提供恢复、预算、审批和审计控制面。

按运行时语义拆开讲

第一步是规划。用户问题进来后，不是立刻搜索，而是先由 Planner 把任务拆成一个显式 DAG，节点类型主要包括 search、browser、rag，另外还预留了 mcp。这里的重点不是“有个 JSON 计划”，而是依赖关系、并行性和节点类型都被结构化，后续执行器才能知道哪些节点能一起跑、哪些必须等前置结果。

第二步是取证。LangGraph 里的 dag_executor 会按可执行批次把节点分发给对应 Agent。Search 负责广撒网找入口，Browser 负责深读页面，RAG 负责查内部资料，而且项目里还显式做了 internal-first 约束：Web 节点默认要在内部 RAG 之后跑。也就是说，这不是“搜索一下再回答”，而是受图约束的并行取证。

第三步是综合。工具节点返回的不是最终答案，而是 evidence、tool history 和 node outcome。等 DAG 跑完后，Analyst 才把多源证据转成结构化分析；它会区分事实和推断、识别冲突、输出带 citation 的分析文本。第四步是校验与收口。Reflection 会把答案质量转成 needs_revision、overall_confidence 这类信号；如果校验不过且修订次数没到上限，就 replan 再走一轮，否则进入 Report。

方案对比也可以顺手讲一下。方案 A 是让多个 Agent 直接对话协作，优点是灵活，缺点是状态、依赖和恢复都很难治理。方案 B 是像 DeepIntel 这样，用 LangGraph + DAG 做显式编排，优点是执行顺序、预算、审批、审计和恢复都有稳定插点，缺点是前期建模更重。这个项目选方案 B，本质上是为了工程可控性。

4.3 公开状态与内部运行状态为什么要分开

核心设计

对外 API 的状态要稳定，对内运行状态要足够细。公开层只需要 pending / running / completed / failed 这种粗粒度语义，但治理层必须区分 awaiting_approval / retryable_failed / terminal_failed / completed，否则失败恢复、审批暂停和预算熔断都没法精确表达。

4.4 ResearchState 为什么是这个系统的核心

Listing 1: ResearchState 中最关键的状态维度

```
task_id / user_query / status / session
dag / current_executing_nodes / completed_nodes
tool_histories / node_outcomes / collected_evidence
verification / revision_needed / revision_count
runtime_status / budget_state / pending_approvals
agent_trace / guardrail_trace / errors
```

设计原则

ResearchState 不是简单的“共享上下文容器”，而是整个工作流的单一运行时状态机载体。凡是会影响路由、恢复、预算、审批和审计的字段，都应该显式出现在 state 里，而不是藏在零散的局部变量中。

深入分析

ResearchState 这一题本质上是在回答“系统的 source of execution truth 在哪里”。如果一个工作流系统没有统一的运行时状态载体，通常会出现三类问题。第一类问题是节点之间靠隐式约定通信，比如某个节点默认下游一定会去读某个临时变量或者某个 trace 片段，这会让路由和恢复逻辑变得脆弱。第二类问题是状态不完整，表面上流程能跑，但到了出错、回放、审计的时候才发现缺关键字段。第三类问题是治理字段没有一等公民地位，预算、审批、失败分类、runtime status 这类字段被塞进 review_status 或普通文本里，后面很难做严肃的控制和恢复。把这些状态提升到 ResearchState 层有一个很重要的工程收益：你可以把“路由条件”“恢复条件”“审计快照”“预算熔断”都建模为状态上的显式变换，而不是 if-else 里的旁路副作用。这样做的代价当然也有，就是 state 会逐步变大、会需要整理 merge 语义、会需要对哪些字段累加、哪些字段覆盖有严格约束。但对于一个长期任务系统来说，这个代价是值得付的，因为它换来的是可解释性和可恢复性。

4.5 Planner 如何生成 DAG，为什么还需要 fallback

面试回答

Planner 的职责不是直接给最终答案，而是把用户问题拆成一张可执行研究图。它会让 LLM 生成 DAGDefinition，里面包含节点类型、查询内容、依赖关系和并行性。但因为 LLM 可能输出坏 JSON、空结果或不合理节点，所以必须准备 fallback DAG，保证流程至少能退化成“搜索 + 浏览 + 内部检索 + 分析”这条保底路径。

深入分析

Planner 这一题考的不是“会不会让模型吐 JSON”，而是“你有没有把 LLM 放在一个可失败的位置上”。如果把 Planner 产物当绝对真理，系统会在两个地方变脆。第一，结构脆弱。模型可能输出坏 JSON、漏依赖、生成不存在的节点类型，或者把本该并行的步骤串起来。第二，策略脆弱。即便 JSON 结构合法，它也可能给出一个业务上很差的计划，比如过度依赖外网、把低价值网页抓取得过深、或者在证据不足时就安排报告节点。

所以 fallback 的意义不是给 Planner 擦屁股，而是承认 Planner 是智能组件，不是可信编译器。工程上至少有两个方案。方案 A 是 Planner 失败就让整个任务失败，优点是语义简单，缺点是把整个系统可用性绑死在一次模型输出上。方案 B

是 Planner 尽量生成高质量 DAG，但执行层永远保留一条保底研究路径。这里显然要选方案 B，因为研究系统的首要目标是“任务可继续”，而不是“计划一定漂亮”。

更深一层看，fallback 还承担了平台语义稳定器的角色。它把最低可执行能力固定成一套可预测的研究骨架，使得即便未来换模型、换 prompt、换节点模板，系统也不会因为 Planner 一次抖动就彻底失能。面试时如果你能把 fallback 讲成“系统韧性设计”，而不是“异常处理补丁”，说服力会强很多。

如果面试官追问“未知问题下灵活性会不会为零”

不会。这里的灵活性不是靠预先写死所有备选方案，而是靠三层机制叠加出来的。第一层是 Planner 动态生成 DAG，而不是固定流程。第二层是 Planner 失败时有 fallback DAG，保证任务不会因为一次模型输出坏掉就整体失能。第三层是 Reflection 和 replan 形成闭环，如果第一次计划覆盖不够或者执行中暴露出失败模式，系统会带着失败记忆重新规划，而不是机械重跑旧图。

原因

很多人会把“有 Planner”误解成“已经有动态规划能力”，这不够准确。真正的动态规划能力至少要回答三个问题。第一，第一次计划失败怎么办；第二，执行到一半发现路走错了怎么办；第三，同样的错误再次出现时，系统会不会还按原路硬撞。如果这些问题答不上来，未知问题下的灵活性确实会很低。

选型

这里至少有两个方案。方案 A 是固定流程加局部兜底，例如 Search 失败就换 Browser，或者 Planner 失败就直接终止。优点是实现简单，缺点是备选路径是预写死的，面对未知问题很快失效。方案 B 是 Planner 动态生成 DAG，执行层保留 fallback DAG，质量层再通过 Reflection 和 replan 形成带记忆的闭环。这里必须选方案 B，因为研究型任务的关键不是提前枚举所有备选方案，而是让系统在运行中根据失败和证据情况重组路径。

如何做

当前这套实现里，第一次计划失败会退化到 fallback DAG，保证最小可执行能力不丢。执行到一半如果 Reflection 认为证据不足、引用覆盖不够或者整体置信度低，系统不会只把失败节点重置成 pending，而是会进入 replan，重新调用 Planner 生成一张新 DAG。更重要的是，replan 不是在真空中发生的，它会把本会话已经出现过的失败模式整理成 failure memory 注入新的规划提示，明确告诉 Planner 哪些工具模式、查询模式或错误类别已经踩过坑，优先换工具、缩小查询范围或改变取证路径。

效果

这样处理以后，系统的灵活性不再依赖预先写死多少分支，而是依赖三种可组合能力：动态拆图、保底退化、带记忆重规划。它解决的不是“Planner 永远正确”，而是“Planner 不正确时任务还能继续，而且第二次规划会比第一次更少重复犯错”。

4.6 DAG 的正确性如何保证

- 数据结构层：Planner 输出被解析成 PlanNode / PlanEdge / DAGDefinition。
- 执行层：get_executable_order() 用拓扑思路分批求可执行节点。
- 容错层：非法节点会被过滤，Planner 失败时回退到 fallback DAG。
- 策略层：系统还会强制内部 RAG 先于 Web 节点执行，形成 internal-first 检索策略。

深入分析

DAG 正确性不能只理解成“有没有环”。在这个系统里，它至少有三层含义。第一层是结构正确，即节点类型合法、依赖存在、拓扑可排序。第二层是执行正确，即调度器不会把失败节点当成完成，也不会在依赖未满足时提前放行下游。第三层是策略正确，即图虽然形式上可跑，但是否符合平台要求，例如 internal-first、预算优先、审批前不得执行高风险写操作。

这也是为什么单靠 Planner 输出校验不够。很多人会说“把 JSON schema 校验做好就行”，这只能保证结构层。真正难的是执行层和策略层，因为这些正确性是运行时才能暴露的。比如某个 Browser 节点虽然结构合法，但它连续超时后是否应该进入 retryable failed，还是被预算 breaker 直接跳过；某个 MCP 写工具虽然在图里合法，但如果缺审批，运行时就必须被挂起。这些都不是静态 schema 能解决的问题。

所以这里的正确性设计本质上是“静态约束 + 动态约束”的组合。静态部分让 DAG 至少能被解释，动态部分保证图在真实世界里按治理要求运行。面试里如果你能明确这两层，说明你理解的是工程正确性，而不是算法课里的 DAG 定义。

4.7 为什么要先 internal RAG，再决定要不要联网

设计理由

研究型系统不应该把互联网搜索当作唯一真相源。内部知识库通常更贴近企业语境，也更可控，所以系统会优先执行内部 RAG，再根据命中情况和策略判断是否继续走 Web。这样既减少无意义联网，也能降低把内部问题完全外部化的风险。

4.8 工具节点为什么必须返回显式结果契约

Listing 2: NodeOutcome 契约

```
NodeOutcome = TypedDict("NodeOutcome", {
    "node_id": str,
    "tool_call_id": str | None,
    "tool_name": str,
    "status": Literal[
        "success",
        "retryable_error",
        "terminal_error",
        "awaiting_approval",
        "skipped",
    ],
    "error_category": str | None,
    "error_message": str | None,
    "retry_count": int,
    "tokens_used": int,
    "cost_usd": float,
    "result_count": int,
    "approval_request_id": str | None,
})
```

为什么不能只靠 tool_histories

tool_histories 更适合审计，不适合做唯一状态源。因为审计记录的目标是“发生了什么”，而路由的目标是“下一步怎么办”。如果失败语义只藏在 tool history 里，聚合器很容易在语义不明确时把节点误标成完成。显式的 NodeOutcome 正是为了解决这个问题。

深入分析

这道题是整个治理改造里最有代表性的一个点，因为它直接说明“记录”和“控制”不能混为一谈。tool_histories 天然面向审计的结构，它关注的是调用了什么工具、传了什么参数、耗时多久、结果摘要是什么。它当然很重要，但它不是为路由设计的。路由真正需要的，是一组最小而明确的控制语义：这次调用是成功、可重试失败、终止失败、等待审批，还是被跳过；如果失败，失败类别是什么；如果等待审批，审批请求 ID 是什么。

如果没有 NodeOutcome，聚合器通常只能从一堆宽泛的历史记录中“推断”当前节点是不是算完成。这种推断最危险的地方在于，一旦工具层记录不完整或者语义不稳定，执行层就会把错误状态误判成完成状态。历史上那个“工具失败但节点被标 done”的 bug，本质上就是因为控制语义没有一等建模。

所以 NodeOutcome 的价值不在于字段好看，而在于它把执行层真正需要的控制

信息抽了出来。你可以把它理解成工具层和调度层之间的一份契约：工具层负责把业务结果和失败语义翻译成一组最小控制信号，调度层再用这组信号决定下一步如何路由。只有这样，审计和控制才不会互相污染。

4.9 怎样避免单点故障把整条链路拖死

方案	做法	优点	缺点
方案 A	任意节点失败就整任务失败	语义简单	单点故障风险高，未知问题可用性差
方案 B	失败显式分类，临时故障指数退避重试，永久故障终止或重规划	可恢复性强，治理语义清晰	需要更多状态和策略维护

原因

如果系统不做错误分类，最典型的后果就是两头都错。第一类错法是把瞬时抖动当永久错误，明明一次 `retry` 就能恢复，却直接把整条链路打死。第二类错法是把永久参数错误当临时波动，结果系统拿着错误输入不断重试，把预算和 `wall clock` 一起烧掉。更进一步，如果可重试失败每次都立刻再打一次，上游正处在抖动窗口时，系统只会更快撞上限流和预算阈值。

选型

这里至少有三个可行方案。方案 A 是所有失败都统一成一个布尔值，只决定成功或失败，实现最简单，但治理语义几乎为零。方案 B 是显式区分 `retryable_error` 和 `terminal_error`，但失败后立即有限重试，优点是比方案 A 强很多，缺点是对上游抖动不够友好。方案 C 是显式失败分类，再把可恢复错误交给调度层做指数退避重试，同时把终止性错误直接停掉。这里要选方案 C，因为它既能区分错误性质，又能避免在抖动窗口里无意义地密集重试。

如何做

当前实现里，工具节点不会把所有失败都压成一个布尔值，而是显式区分 `retryable_error`、`terminal_error` 和 `awaiting_approval`。`timeout`、429、上游 5xx、网络抖动、临时 DB 故障会被归到可恢复类别；`schema` 校验失败、权限问题、审批拒绝、MCP server 不可信这类会被归到终止类别。对于前一类错误，系统不会立刻机械重试，而是根据错误类别给一个基础退避时间，再随着 `retry_count` 指数增长，计算出 `next_retry_at` 写进 `failure memory`，并受 `max_retries_per_tool` 约束。调度层只有在退避窗口到期后，才会把节点重新放回执行队列。对于后一类错误，节点会直接标成 `terminal`，不再重复消耗预算，同时失败模式会写入 `failure memory`，为后续 `replan` 提供负反馈。

效果

这样做以后，系统既不会因为一次网络抖动就把整条任务打死，也不会因为一个永久参数错误而在错误输入上疯狂重试。可恢复错误会在调度层自动退避并等待更合适的重试窗口，终止性错误会被尽快收口，避免继续烧 token、tool call 和 wall clock。

4.10 系统会不会从历史失败里吸取教训

原因

如果系统每次失败都把历史清空，就会重复踩同一种坑。对研究型任务来说，真正浪费预算的往往不是单次失败，而是同一个工具模式、同一个查询模式、同一种错误类别在一个会话里反复出现。这个问题如果不处理，重试会退化成“重复犯错”，replan 也会退化成“换个 prompt 再犯一遍”。

选型

这里至少有两个方案。方案 A 是完全不记失败历史，每次重试和 replan 都只看当前节点状态，优点是实现简单，缺点是系统没有记忆。方案 B 是先做会话级 failure memory，把本次任务里的失败模式变成控制面的一等状态，再用于重试调度和 replan 提示。这里选方案 B，而不是一上来做全局跨任务学习，是因为会话级记忆最贴近当前控制面，也最容易验证效果和控制风险。

如何做

当前已经支持会话级 failure memory，而不是全局跨会话学习。系统会总结本次任务里哪些工具模式、查询模式和错误类别已经失败过，并把这些信息记录成结构化状态。它至少做三件事。第一，重复失败阻断。同一个工具类型加同一个查询，如果已经达到最大重试次数，或者已经被判成终止性失败，系统不会继续把它当成普通 pending 节点调度。第二，退避调度。对于可恢复错误，系统会把下一次可重试时间写进 failure memory，按指数退避窗口再入队。第三，重规划提示。系统会把这些失败模式整理成简短的 planning hints 注入下一次 replan，明确告诉 Planner：哪些模式已经失败过、哪些已经 repeat blocked、优先换工具还是缩窄查询。这里也要守住边界：当前退避是图内调度层的短等待与再入队，不是 Harness 级定时唤醒系统。

效果

failure memory 的效果不是把系统吹成“会自我进化的大模型”，而是先把失败经验转化成可执行控制。它能减少会话内重复犯错，让自动重试更克制，让 replan 更有方向感。面试里最稳的说法是：我们已经实现了会话级失败记忆，能在本次任务里做到重复失败阻断、指数退避调度和带记忆重规划；但还没有把它做成跨

任务长期学习系统。

4.11 Search / Browser / RAG 三类节点如何分工

- Search: 适合快速事实、数据、新闻入口、网址发现。
- Browser: 适合深读文章、结构化页面提取、动态网页。
- RAG: 适合内部资料、历史文档、背景知识和企业语义上下文。

面试回答模板

这三类节点不是冗余，而是故意分层。Search 负责广撒网，Browser 负责深挖高价值页面，RAG 负责内部可控证据。真正有价值的研究系统不是任何问题都扔给浏览器，而是先判断哪种证据源最值得花成本。

5 第四轮拷打：关键机制是不是你真的做过

这一轮面试官在试什么

这一轮会专门挑几个最容易暴露“只懂概念、不懂实现”的点来问，例如 Reflection 有什么用、RRF 为什么合理、Checkpoint 到底帮你解决了什么。这里需要把抽象概念落回具体执行语义。

这一轮最常见的追问

- Reflection 是不是只是让模型再看一遍自己的答案。
- Analyst 和 Report 为什么不能合并成一步。
- Checkpointer 到底恢复了什么，没恢复什么。
- SSE 只是前端体验，还是系统治理的一部分。

5.1 Analyst 为什么单独存在

核心作用

Analyst 的意义是把多源证据从“片段集合”转成“结构化分析”。Search、Browser、RAG 拿回来的是证据，Analyst 负责做事实归类、冲突识别、分析组织和初步结论形成。

深入分析

这一题真正要回答的是，为什么不能让最终 Report 一步做完。答案是证据整合和结果表达虽然都要调用模型，但它们承担的是不同职责。Analyst 关注的是“证据之间到底是什么关系”，包括哪些事实互相印证、哪些来源互相冲突、哪些结论目前证据不足；Report 关注的是“如何把已经形成的分析组织成适合阅读的输出”。把两者合并，短期能省一个节点，长期会让分析过程变成黑箱。

分开设计至少有三个收益。第一，错误隔离。若报告写得不好，你可以判断是表达问题还是前面的分析就错了。第二，治理插点更清晰。Reflection 应该审的是分析质量和引用覆盖，而不是一篇已经高度修辞化的最终文案。第三，复用能力更强。未来如果要同时输出简版摘要、详细报告、内部 memo，Analyst 产出的结构化分析可以被多个 report renderer 复用。

所以 Analyst 不是为了“多一个 Agent”，而是为了把证据解释层从最终呈现层剥离出来。面试官如果继续追问“是不是过度设计”，你就可以直接回答：只有当系统目标是一次性回答时它才显得重；一旦目标是多源证据研究、复盘和质量控制，这个拆分就是必要的。

5.2 Reflection 为什么不能省掉

核心回答

如果没有 Reflection，整个系统就会退化成“拿到一些证据后直接写报告”。Reflection 的职责是把回答质量结构化成 `overall_confidence`、`citation_coverage`、`needs_revision` 等信号，再由图路由决定是否重规划。它不是装饰，而是把“能不能继续往前走”显式化。

深入分析

Reflection 的关键价值，是把“质量判断”从人脑直觉变成系统信号。没有它，系统虽然也能输出报告，但整个链路没有一个地方正式回答“这份报告到底靠不靠谱、证据覆盖够不够、是否需要再搜一次”。很多 Demo 都止步在这一步，因为只要模型能写出看起来像样的报告，就误以为流程闭环了。实际上那只是生成闭环，不是质量闭环。

备选方案至少有两个。方案 A 是完全不做 Reflection，相信 Analyst 和 Report 已经足够。优点是链路短、成本低，缺点是系统没有内生的质量回路。方案 B 是加入一个专门的反思节点，把答案质量、引用覆盖率和修订建议结构化，再把它转成路由条件。这里必须选方案 B，因为研究系统的价值不只是“能写”，而是“在证据不足时知道自己该停下还是重来”。

当然，Reflection 也不是银弹。它本身仍依赖模型判断，所以它给出的不是绝对真相，而是一个可操作的质量信号层。这一点要诚实讲清楚。它不是替代人工 reviewer，而是在自动链路里尽量提前发现低置信或低覆盖的问题。这样表达既不会夸大能力，也能体现你理解它的工程位置。

5.3 Reflection 和传统 reviewer 的区别

- Reviewer 更偏规则门禁或人工审查。
- Reflection 更偏模型辅助的自校验与质量评分。
- 在当前项目里，Reflection 和动作审批 reviewer 已经分开：Reflection 负责答案质量自校验，approval reviewer 负责高风险动作的暂停、审批和恢复入口。

5.4 RRF 融合检索为什么合理

Listing 3: RRF 的直观公式

```
score(d) = sum(1 / (k + rank(d)))
```

设计理由

RRF 不是为了追求算法花哨，而是为了在多路检索结果之间取得稳定折中。向量相似度、关键词匹配、重排结果的分值尺度往往不一致，直接线性加权很脆弱；RRF 只依赖 rank，更稳、更简单，也更容易工程落地。

深入分析

RRF 合理，不是因为它在论文里很经典，而是因为它非常适合工程环境里的“异构召回”。在真实系统里，向量检索、关键词检索、规则检索、重排模型往往来自不同模块，它们的分值空间根本不可直接比较。你如果做线性加权，就必须不断调分值归一化、权重比例、场景偏置，这在数据分布变化时会很脆。

RRF 退一步，只相信名次，不相信原始分数。这样做牺牲了一些极致调优空间，但换来的是稳定性和可维护性。对于一个研究型系统来说，这通常是正确取舍，因为它需要一个在数据源变化、召回器变化时依然不至于失控的融合策略，而不是一个只在离线集上分数最好、上线后难维护的复杂公式。

如果面试官追问为什么不用学习排序或者更复杂的融合，你可以直接回答：可以做，但前提是你已经有足够稳定的数据闭环和标注反馈。当前阶段更值得优先的是稳健和可解释，而不是把排序问题过早做成重机器学习工程。这个回答会显得很务实。

5.5 Checkpointer 的真实价值是什么

核心答案

`thread_id=session_id` 保证的是会话隔离，Checkpoint 提供的是状态恢复能力。它能让 LangGraph 在节点边界保存执行快照，使流程在中断后有机会从已知状态继续。但要注意，检查点本身不是完整治理方案，它不自动解决预算、审批、

审计和 supervisor 恢复控制。

深入分析

这一题最常见的误区是把 checkpointer 神化，好像一旦有了 checkpoint，恢复问题就全解决了。实际上 checkpointer 解决的是“图内状态快照”问题，不是“任务级治理闭环”问题。它很擅长保存某个 thread 在某个节点边界上的状态，让你不必每次都从头开始跑；但它不天然知道当前 worker 是否还持有租约，不知道预算是否已超，不知道某个动作是否还在等待人工审批，也不知道 supervisor 现在是否决定暂停整个任务。

所以 checkpointer 的正确定位是：它是恢复机制的重要组成部分，但不是完整的恢复控制面。你可以把它理解成恢复所需的“状态底座”，而 Harness 和数据库里的 runtime/approval/budget 状态则是在定义“到底能不能恢复、应该恢复到哪里、恢复后是否继续执行”。面试时如果你能把 checkpointer 的能力边界讲清楚，反而更显得你做过真实长任务系统，而不是把框架能力误当成系统能力。

5.6 为什么 SSE 对这个系统重要

核心理由

研究任务天然就是长任务，如果没有流式可观测性，用户只能看到一个很长的 loading。SSE 的价值不是“酷炫实时输出”，而是把 agent trace、tool 进度、状态变化、报告片段和错误事件透明推给前端，让任务变成可感知、可追踪、可中断观察的过程。

6 第五轮拷打：性能、成本和资源怎么扛

这一轮面试官在试什么

很多候选人在这里会只讲“做了并发、做了缓存、做了优化”。这不够。研究型 Agent 的资源问题至少包含三类：时延、token/cost、数据库与工具开销。面试官想看的是你有没有把它们视为同一个系统问题。

这一轮最常见的追问

- 这个系统最主要的耗时在什么地方，是模型、检索还是网页抓取。
- 你怎么避免 Browser 节点把 token 直接吃爆。
- 预算治理是软提示还是硬熔断，熔断之后系统怎么收口。
- 为什么数据库仍然选 PostgreSQL，而不是拆成更多专用组件。

6.1 为什么并行执行能显著降低时延

Listing 4: LangGraph Send API 的作用

```
dag_executor -> [Send("search"), Send("browser"), Send("rag")]
```

面试回答

如果 Search、Browser、RAG 串行执行，总时延接近三者之和；并行 fan-out 后，总时延更接近三者中的最大值。这类任务的瓶颈往往在 I/O，而不是 CPU，所以并行调度的收益很高。

深入分析

并行执行的收益不能只用一句“总时延取最大值”带过，面试官真正想看的是你是否知道什么场景适合并行、什么场景不适合。这个系统里 Search、Browser、RAG 的共同特点是高度 I/O 密集，而且很多时候相互独立。Search 等网络返回，Browser 等页面加载，RAG 等数据库检索，这些等待时间如果串行堆起来，就是纯浪费。所以把它们 fan-out，本质上是在回收等待时间。

但并行不是越多越好。第一，外部依赖会有限流和配额，Browser 开太多页面可能直接把系统拖死。第二，并行会放大预算消耗，如果 Planner 过度拆图，虽然时延下降了，但 token、工具调用次数和外部 API 成本会同时上升。第三，结果聚合会更复杂，失败语义、超时处理和 partial result 策略都必须更严谨。所以正确回答不是“并行一定更快”，而是“在 I/O 主导、节点相互独立、预算允许的前提下，并行能显著降低 wall clock；但必须配合批次调度、预算 breaker 和失败聚合语义，否则它只是把串行浪费变成并行失控”。这句话会让面试官感觉你真的踩过并发系统的坑。

6.2 Token 优化策略

- 输出长度分 short / medium / long 三档，不只是影响报告 token，也同步影响搜索次数、RAG 返回量和 citation 上限。
- 浏览器内容用 snippet / skim / deep 控制页面读取粒度。
- RAG 侧采用候选召回 + 排序 + 截断，避免把低价值片段灌入上下文。
- 最终报告阶段再根据预算控制输出规模，避免前面省下来的 token 最后一次性烧光。

深入分析

Token 优化真正难的地方，不在于“少输出一部分”，而在于你不能为了省 token 把证据链和答案质量一起省没了。研究型 Agent 和普通聊天模型的最大区别在于，很多 token 消耗不是出在最终回答，而是出在前面的证据获取和中间分析。网页

抓取太深、RAG 返回太多低质量片段、Planner 把 DAG 拆得太碎、Reflection 重复审查太多次，都会在最终报告生成前就把预算吃掉。

所以这里的优化必须是系统性的。第一层是输入压缩，即在进入模型前先控制证据粒度，例如浏览器的 snippet/skim/deep、RAG 的候选截断、citation 上限。第二层是路径裁剪，即并不是所有任务都值得走完整 Web 扩展链路，internal-first 的策略就是一种成本治理。第三层才是输出控制，即最终报告的长度、引用数量、展开程度。把这三层分开讲，面试官会更容易相信你理解的是“成本结构”，而不是只有“把 max_tokens 调小”这种表层操作。

6.3 数据库优化怎么看

- 文档层：向量索引承担 ANN，全文索引承担关键词检索。
- 业务层：research session、citation、tool audit、budget state 都在同一库里，降低同步复杂度。
- 可观测层：指标与 trace 可以按 session 维度做统计和聚合。

深入分析

数据库选型这题不能只答“PostgreSQL 通用、成熟”。真正的判断是：当前系统的复杂度是否值得拆成多套专用存储。至少有两个方案。方案 A 是 PostgreSQL 一库承载业务数据、向量检索、审计和预算状态，优点是事务边界清晰、查询方便、运维成本低，缺点是向量能力和高吞吐日志能力不如专用系统。方案 B 是把向量库、OLTP 库、日志库拆开，优点是各组件可以针对性优化，缺点是同步、幂等、一致性和运维复杂度都会上升。

当前阶段选方案 A 是合理的，因为系统最主要的矛盾还不是极致吞吐，而是治理闭环和状态一致性。审批、预算、runtime status、tool audit 这些关键控制数据如果分散在多种存储里，恢复和审计会先变复杂。把它们尽量放在同一个事务型主库里，反而更利于把控制面做稳。

当然，这个选择有边界。随着文档规模、检索 QPS、审计写入量上升，未来完全可能拆出专用检索或时序/日志存储。但那应该建立在当前单库模型已经成为明确瓶颈之后，而不是一开始为了“架构高级感”就过度拆分。面试时把这一层演进逻辑说清楚，会比直接喊“微服务 + 多存储”成熟得多。

7 第六轮拷打：你怎么证明它没胡来

这一轮面试官在试什么

会做流程不等于会做系统。面试官在这一轮通常会问：你怎么知道它真的在工作，而不是表面有输出？你怎么定位问题？你怎么区分是模型错了、工具错了、还是治理规则拦住了？

这一轮最常见的追问

- 你会盯哪些指标，而不是泛泛地说“看日志”。
- trace 为什么要拆成 agent、guardrail、tool audit 三条线。
- 出现错误时，怎么判断是模型输出坏了还是外部工具失败了。
- 这个系统能不能做法证级复盘，复盘数据现在放在哪。

7.1 我会看哪些指标

维度	代表指标	作用
工作流层	成功率、P95 时长、重规划次数	看流程稳定性
工具层	tool success rate、平均时长、错误分类	看节点可靠性
质量层	离线看 Hit@K、MRR、Faithfulness、Answer Relevancy；在线看 citation coverage、overall confidence、转人工率	分层定位检索问题还是生成问题
成本层	used tokens、used cost、tool calls、wall clock	看预算消耗
治理层	approval count、budget stop count、terminal failures	看可控性

面试里这题的标准说法

RAG 评估不能只看最终答案好不好，而要拆成两层。第一层先单独看检索层，常用 Hit@K 和 MRR。Hit@K 回答“正确 chunk 找到没”，MRR 回答“找到得靠不靠前”。第二层再看生成层，常用 Faithfulness 和 Answer Relevancy，前者看有没有照着资料说，后者看有没有回答用户真正的问题。线上再用点击率、转人工率这类业务指标做最终验收。这样一拆，问题才能定位得准。

深入分析

如果只盯最终答案质量，很容易把检索问题和生成问题混在一起。比如答案不好，可能是检索根本没把正确 chunk 召回来，也可能是 chunk 明明召回了，但排序太

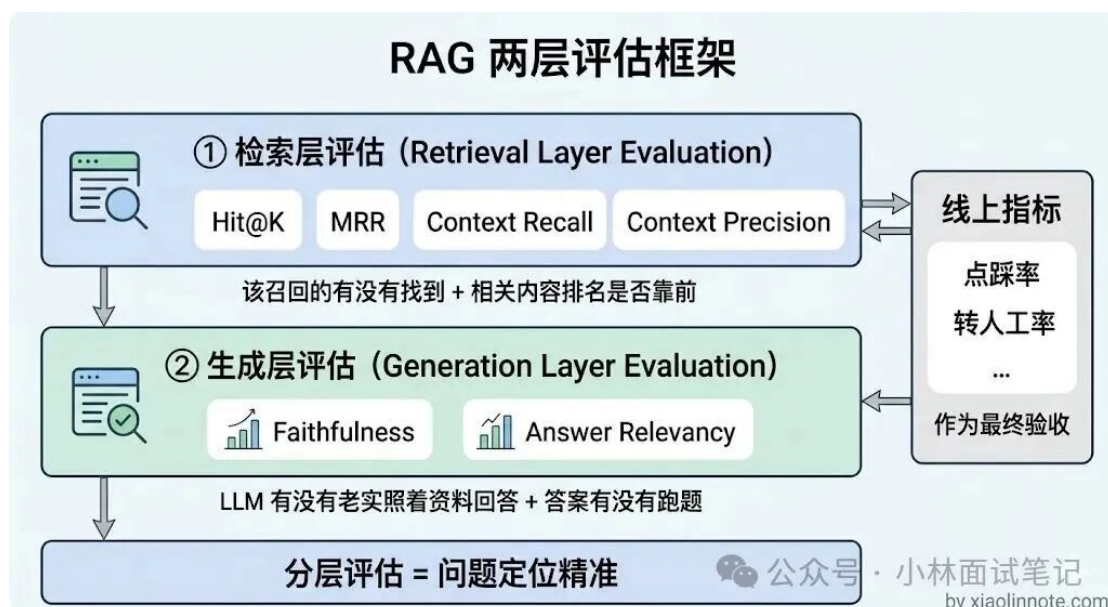


图 1: RAG 两层评估框架：离线先拆检索层与生成层，线上再看业务验收指标

靠后，模型没好好用；还可能是上下文没问题，但模型自己跑题或者幻觉。把评估拆成检索层和生成层，诊断路径就会清晰很多。

检索层里，Hit@K 看的是“有没有找着”。它不关心正确 chunk 是第一名还是第五名，只关心在 Top-K 里有没有出现。所以 Hit@5 高，说明召回大体没丢。MRR 看的是“找着得有多快”。如果 Hit@5 很高，但 MRR 很低，通常表示正确内容虽然被召回了，但排位靠后，这时候优先怀疑 rerank、融合排序或者 chunk 粒度不合适。

生成层里，Faithfulness 和 Answer Relevancy 不能混为一谈。Faithfulness 问的是“说的是不是真的，有没有证据支撑”，Answer Relevancy 问的是“说的是不是用户想要的”。一个答案可以非常忠实于资料，但完全答非所问；也可以回答了问题，但中间夹杂很多编造内容。面试时把这两个维度分开讲，面试官通常会认为你是真的做过质量诊断，而不是只背了几个名词。

工程上我会把这套体系分成离线和在线两条线。离线用小规模标注集跑 Hit@K、MRR，以及基于 RAGAs 思路的 Faithfulness、Answer Relevancy、Context Recall、Context Precision，用来快速定位是哪一层出了问题。在线则继续看 citation coverage、overall confidence、点击率、转人工率，把它们作为最终业务验收。这样既能做研发调参，也能做真实效果闭环。

怎么解释 Hit@K 和 MRR 的差异

很多人会把 Hit@K 和 MRR 讲成一回事，这不够准确。Hit@K 只回答“Top-K 里有没有命中正确 chunk”，所以它更像召回覆盖指标；MRR 则会惩罚排名靠后，正确内容排第一名得 1 分，排第五名只有 0.2 分，更适合判断 rerank 和融合排序有没有把关键内容顶到前面。面试时最好直接举一个例子：Hit@5 很高但 MRR 很低，说明“找到了，但排得不够前”。



图 2: Hit@K vs MRR: Hit 看“有没有找到”，MRR 看“找到得有多靠前”



图 3: Faithfulness vs Answer Relevancy: 忠实于资料不等于回答了用户问题

怎么解释 Faithfulness 和 Answer Relevancy

Faithfulness 和 Answer Relevancy 不能混着说。Faithfulness 关注答案是否有证据出处，本质上是在看幻觉；Answer Relevancy 关注答案是不是在回答用户真正的问题，本质上是在看跑题。一个回答可以字字有据但完全答非所问，也可以看上去回答了问题但夹杂大量编造内容。把这两个维度拆开，生成层的问题才能定位得准。



图 4: RAG 指标诊断与优化映射：不同指标低，对应不同病灶和优化方向

指标低时怎么诊断

这张图在面试里特别好用，因为它能把“会看指标”和“会做优化”连起来。如果 Context Recall 低，优先怀疑 embedding、chunking、多路召回不够；如果 Context Precision 低，优先怀疑 rerank 或 Top-K 截断不合适；如果 Faithfulness 低，优先怀疑 prompt 约束不足或检索上下文质量差；如果 Answer Relevancy 低，则更多是指令聚焦不够、问题重写有偏差，或者回答模板没有把用户目标钉死。真正成熟的回答不是背指标定义，而是能顺着指标讲出下一步怎么改。

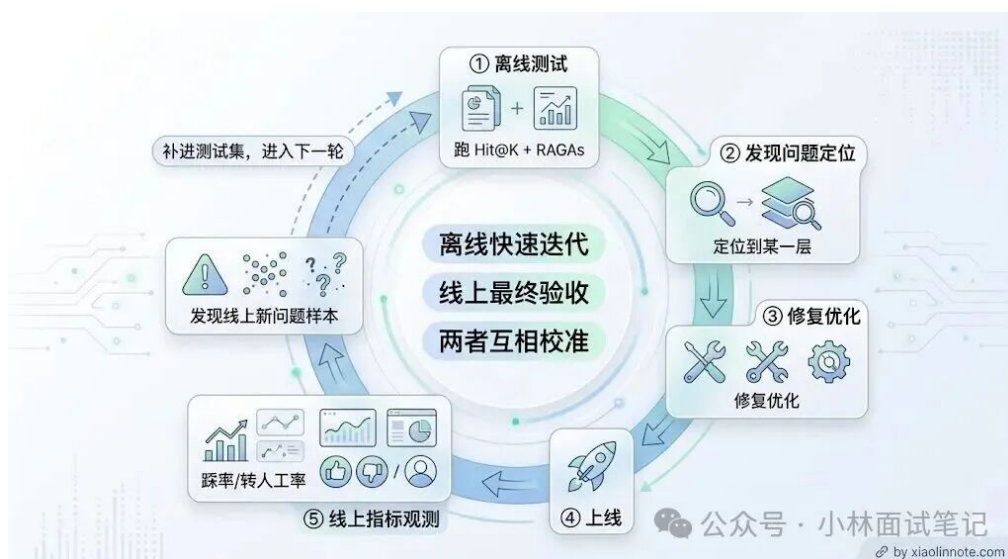


图 5: 离线快速迭代、线上最终验收、两者互相校准

怎么讲离线和线上闭环

工程上不能只做离线 benchmark，也不能只盯线上点击率。更合理的做法是：先在离线测试集上跑 Hit@K + RAGAs，定位检索层还是生成层出了问题；修完后再看线上点击率、转人工率、用户满意度有没有一起改善；如果线上暴露了新的坏样本，再把这些样本补回离线测试集，进入下一轮。这样离线是诊断工具，线上是最终验收，两者互相校准。

7.2 Embedding 怎么评估，光看 MTEB 榜单行不行

这一题面试官在试什么

表面问的是“你们 RAG 用什么 embedding、怎么评估好不好”，实际考的是：你能不能把“把问题和文档变成向量”这一步放回真实业务里判断，而不是报一个公开榜单名次就完事。会背 MTEB 和真正做过选型，差距就在这一题。

指标	属于哪层	衡量什么	低了说明什么
Hit@K	检索层	正确 chunk 是否被召回	Embedding 或 Chunking 有问题
MRR	检索层	正确 chunk 的排名是否靠前	Rerank 效果差
Context Recall	生成层输入	检索内容覆盖了多少正确信息	多路召回不足
Context Precision	生成层输入	检索内容里噪音多不多	Rerank 没过滤掉无关内容
Faithfulness	生成层	答案有没有幻觉	Prompt 约束不足或检索质量差
Answer Relevance	生成层	答案和问题相不相关	Prompt 写法问题
踩率 / 转人工率	线上	用户实际满意度	整体系统效果，综合反映

图 6: RAG 指标总表：指标所属层级、衡量对象与低分时的优先排查方向

面试里这题的标准说法

先拿 50 个真实用户问题，再为每个问题标出“标准应该找到哪份资料”（gold doc）。然后看系统前 10 个候选里有没有这份资料、它排在第几位，专业点说就是用业务 query + gold doc 跑 Hit@K / Recall@K 和 MRR。我的判断是：没有自家测试集的 embedding 选型，本质上只是换模型抽盲盒。MTEB / C-MTEB 只能做第一轮粗筛，把候选从几十个砍到三五个；最终选择必须由业务测试集定。

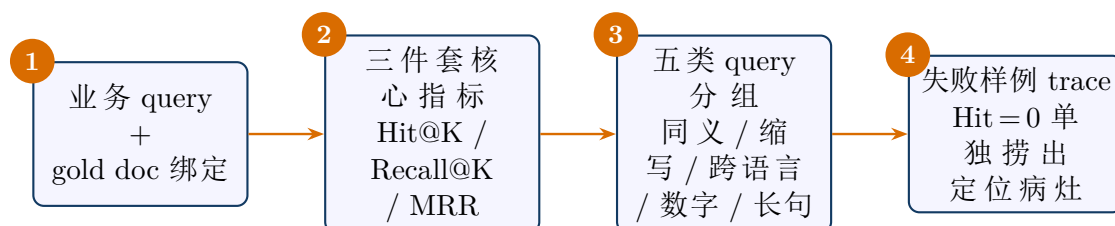


图 7: 最小 embedding eval 的四步：绑定 gold doc、跑三件套指标、按五类 query 分组、再对失败样例做 trace

为什么不能只看榜单，也不能只“换更大的模型”

MTEB 测的是“通用考试题”，而你的系统面对的是“公司自己的题”：内部工单、合同、代码库、客服记录。公开榜单靠前的模型换到法务 PDF、内部缩写场景上，Top-10 命中率反而可能掉十几个点。“换更大的模型”同样靠不住——不同领域上 text-embedding-3-large、Voyage、bge-m3 这类模型的胜负关系是交叉的，把“大等于准”当标准，恰好跳过了最关键的一步：它在你的资料库里到底找不找得到正确资料。

真正的分水岭不是你报出哪个模型名，而是你能不能拿出一张“哪些问题找得到、哪些找不到”的失败样例表。判断顺序我会固定成三步：先看找不找得到（Hit@K / Recall@K），再看排得靠不靠前（MRR），最后看失败集中在哪类问题上。不要只看 cosine 均值，它更像平均相似度，不能直接说明用户能不能拿到正确资料。

单个标准答案和多个标准答案，指标怎么选

单个 gold doc 的场景先看 Hit@K：前 K 个候选里有没有命中那条正确资料；多个 gold doc 的场景再看 Recall@K：正确资料被找回了多少比例。所以工程上我会把 Recall@K 和 Hit@K 一起记录，而不是只留一个。代码里 `record_retrieval_eval` 对每条样本同时算 hit、recall@k、reciprocal_rank，正是为了支撑这个判断顺序。

判断核心：按 query 类型分组，比平均分更值钱

为什么必须按 query 类型分组

平均分会把问题盖住。企业 RAG 最容易翻车的不是标准问法，而是五类“麻烦问题”：同义改写、公司内部缩写与术语、中英混着问的跨语言、订单号/错误码这类精确字符、以及很长很口语的长 query。这些在公开榜单里不一定多，但在你的知识库天天出现。

工程动作就是：给每条问题打一个类型标签，跑完后按标签分组看 Recall@10；再把失败问题单独拿出来，看是资料切得不对、模型不认识内部词，还是 gold doc 本身标错了。这套数据沉淀下来就是回归集，每次换模型、换切分策略都重跑。代码里 `RAGEvalExample.query_type` 把类型带进每条样本，collector 的 `by_query_type` 做分组聚合，failures 把 Hit 为 0 的样本单独 trace 出来，正是这套方法的落地。

跑完后不要只看平均分，按类型聚合，弱在哪一眼就能看出来。下面是一份典型的分组诊断表，结论不是“换更大的 embedding”，而是分类型对症下药：

Query 类型	Recall@10	MRR	诊断方向
同义改写	0.92	0.71	先不动
缩写术语	0.34	0.18	加 BM25 关键词召回
跨语言	0.61	0.42	先看失败样例再定
数字代码	0.28	0.14	关键词精确兜底
长 query	0.78	0.55	加 rerank

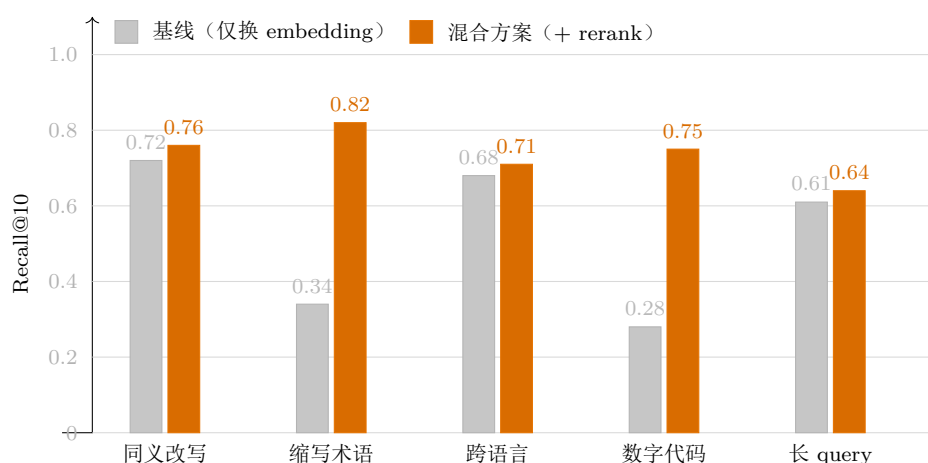


图 8: 仅换 embedding vs hybrid+rerank: 按 query 类型分组的 Recall@10 对比。缩写术语与数字代码两类收益最大 (+0.48、+0.47)，同义改写几乎无差别——印证“分类型对症下药”而非笼统换大模型。

为什么术语和数字代码要走 hybrid，而不只是换 embedding

内部缩写、产品型号、订单号不像自然语言，更像精确字符，比如 SKU-X1932、ORD20260418，纯语义检索反而不可靠，关键词检索更稳。更值得做的是 hybrid：语义搜索找一批、关键词搜索找一批，合并后再 rerank。一份匿名复盘可以印证方向：仅换 embedding，整体 Recall@10 从 0.74 到 0.78；加关键词兜底 + 重排后整体 0.74 到 0.91，缩写类从 0.34 拉到 0.82。这不是公开基准，只说明诊断方向——收益最大的恰恰是缩写类和数字代码类。

追问：Recall@10 高但答案仍然差，是 embedding 的问题吗

大概率不是。前 10 个候选里已有正确资料，说明“找资料”这一步不算坏；问题多半在后面：rerank 没把它推到前三、提示词没要求优先看高排名片段、或引用被裁断。处理顺序是先看 rerank，再调提示词，最后才动 embedding。这也对应生成层诊断图里的结论：Context Recall 不低却 Faithfulness/Answer Relevancy 低，病灶在排序与生成约束，而非召回。

追问：怎么用 50 条 query 做一个最小 embedding eval

五步就够：抽 50 条真实问题；给每题标出应该找到的 1-3 篇 gold doc；跑两个候选模型；看前 5 / 前 10 是否命中、排第几；最后按问题类型分组诊断。50 条不是上限而是起跑线，跑通后再扩到 200。关键是先把评估闭环跑通——同一份 200 条样本上跑过 3 个 embedding，比只看 MTEB 排行榜有用得多。

7.3 生产追问：Agent Eval 怎么拆到可定位故障

工程结论

业务 Agent 评估不能只给最终答案打分，而要拆成“任务完成、证据召回、工具执行、答案可信、成本受控、恢复能力”六层。这样一旦分数下降，能定位是 retrieval、rerank、generation、tool policy、budget breaker 还是 harness recovery 的问题。

按流程讲最清楚

第一步先定义评估单元。不是所有任务都该用同一把尺子，至少要区分事实问答、研究总结、流程查询、开放分析这几类任务，并给出 gold answer、gold doc 或最小验收标准。第二步跑离线基准。对 RAG 问题先测检索层的 Hit@K / Recall@K / MRR，再测生成层的 Faithfulness / Answer Relevancy / Context Recall / Context Precision。第三步看执行层。也就是工具调用次数、成功率、失败类别、是否触发审批、是否被 budget breaker 截断。第四步再接线上指标，例如点击率、采纳率、转人工率、用户追问率。

每个指标最好直接讲语义。Hit@K 是“找没找到”；MRR 是“找得靠不靠前”；

Faithfulness 是“有没有按证据说”；Answer Relevancy 是“有没有答到问题”；Context Recall 是“该给模型看的关键上下文有没有进来”；Context Precision 是“塞给模型的上下文里噪音多不多”。再往执行侧，tool success rate 是“工具面是否稳定”，approval rate 是“高风险动作触发频率”，budget overrun rate 是“系统是否经常失控”。

技术选型上也有两种路。方案 A 是只做 LLM-as-a-judge，优点是搭得快，缺点是会把检索、排序、生成、工具问题混成一锅。方案 B 是分层评估，把 retrieval、generation、execution、business acceptance 拆开。DeepIntel 已经明显走向方案 B，因为项目里既有 record_retrieval_eval，也有 tool history、node outcome、budget state 这些运行时观测面。

7.4 生产追问：无增益 tool call 怎么量化和治理

工程结论

这类问题不能只看 answer quality，因为答案可能没变，但效率已经变差了。最直接的做法是补一组“过程效率指标”，例如 tool efficiency、redundant tool call rate 和 marginal utility per call。

推荐的评价口径

至少有两种口径。方案 A 是只看最终答案对不对。优点是简单，缺点是会纵容“结果没坏但过程很浪费”的行为。方案 B 是把结果指标和过程指标拆开：结果层继续看正确率、Faithfulness、任务完成率；过程层新增冗余调用率、平均每次成功任务的 tool calls、同类查询重复调用率，以及“去掉某次调用后结果是否变化”的边际贡献评估。这里显然更该选方案 B，因为 Agent 系统的成本和稳定性本来就是产品质量的一部分。

面试时可以给一个很稳的表述：如果多调用工具但最终答案完全不变，我会把它定义成“无增益调用”。评估上不改最终正确率分，但会拉低过程效率分。更细一点，可以把一次任务的总分写成：quality score 和 efficiency penalty 分开，前者奖励答对，后者惩罚多余调用、重复检索和无效重试。这样既不误伤答案质量，也不会让资源浪费被平均分盖住。

7.5 生产追问：Chunking 如何避免破坏证据边界

项目内实现

DeepIntel 当前采用的是偏工程稳妥的段落优先切分：先按空行切段，尽量把相邻段落拼到 chunk_size 上限内；如果单段过长，再按字符窗口切开，并保留 chunk_overlap 重叠。

为什么这么做

当前仓库里的实现比较明确：先把文本做换行归一，再按段落分块；短段落尽量合并，长段落才退化到窗口切分。这种方案的优点是实现简单、稳定、对 markdown / txt / json 转文本后的内容都能工作；缺点是它主要按表面结构切，不理解标题层级、表格语义和代码块边界。

至少有两种方案。方案 A 是固定窗口切分，优点是简单、吞吐高，缺点是容易把语义句子和段落拦腰切断。方案 B 是结构感知切分，例如先按标题、列表、表格、章节切，再在局部做窗口化。优点是语义完整度更高，缺点是要先把文档标准化并做更复杂解析。DeepIntel 当前实际落地更接近 A 和 B 的中间态：不是裸滑窗，而是“段落优先 + 超长回退滑窗”。

7.6 生产追问：多格式文档如何做 canonical representation

结论

先不要急着切 chunk，先做文档标准化。核心原则是把不同来源先转成统一的中间表示，再按文档类型决定切分策略，而不是所有输入都直接按字符切。

处理顺序

更稳的设计至少分两层。第一层是解析归一化，把 pdf、docx、json、txt 先抽成统一文本或中间结构；项目里现在已经支持 txt、md、json、docx、pdf，并且 json 会递归展开成可读文本。第二层是类型化切分。制度文档适合按标题和条款切，FAQ 适合按问答对切，代码文档适合按函数 / 类切，工单记录适合按时间段或会话轮次切。

方案 A 是所有内容都 flatten 成纯文本，再统一 chunk。优点是实现最快，缺点是结构信息损失大。方案 B 是先做 canonical representation，例如 title / section / body / table / metadata 这样的中间层，再按类型路由切分器。长期一定更应该走方案 B，因为“先统一表示，再分类型处理”才有机会把不同文档类别都做好。当前项目已经做了第一步的文本归一化，但严格说，按文档类型分策略这一步还没有完全展开，面试里要诚实讲成“现阶段边界”。

7.7 生产追问：Embedding、稠密召回和 Rerank 的取舍边界

Embedding 选型原则

Embedding 选型我会固定讲四件事：语言覆盖、领域匹配、向量维度与成本、和现有检索链路的兼容性。公开榜单只做粗筛，最终一定回到业务测试集。

选型对比

至少有两个方向。方案 A 是选通用大模型 embedding，优点是多语言和泛化通常更强，缺点是成本高、维度大、在内部术语场景未必稳赢。方案 B 是选开源可本地化模型，例如项目里现在配置的 BAAI 系列，优点是可控、成本低、便于私有化，缺点是要自己做业务评估和运维。DeepIntel 当前选的是方案 B，仓库里默认 embedding 是 BAAI/bge-zh-qwen2-int8，rerank 是 BAAI/bge-reranker-v2-m3，这是偏中文和可控性的取舍。

判断标准不要只背模型名。语言覆盖看中英混查和多语言资料；领域匹配看内部术语、缩写、数字代码；工程成本看维度、吞吐、部署资源；链路兼容性看它和 BM25、rerank、pgvector 的组合效果。这才是成熟回答。

稠密召回的失效模式是什么

稠密检索的核心是把 query 和 document 映射到同一个向量空间里，再按 cosine similarity 或 inner product 做近邻搜索。它的强项是语义改写，弱项是精确编号、罕见术语、权限过滤和结构化字段，所以生产 RAG 通常要 hybrid search、metadata filter 和 rerank 共同兜底。

原理版回答

如果面试官继续追问，可以这样展开：训练阶段希望相关 query-doc 对在向量空间更近，不相关对更远；推理阶段把 query 编成向量后，在向量库里做近邻搜索，拿回 Top-K 候选。它擅长语义改写、同义表达和自然语言问题，但对精确字符串、编号、缩写不一定稳，所以企业场景常常要和 BM25 做 hybrid。

Rerank 为什么不能替代召回

Rerank 是第二阶段排序，只能重排已召回候选，不能补回第一阶段没找到的证据。因此如果 Recall@K 本身很低，换更强 reranker 不会根治；先要修 chunking、hybrid recall、query rewrite 和 metadata filter。

怎么讲算法原理

至少有两大类。方案 A 是 bi-encoder 召回，也就是 embedding 检索，优点是快，缺点是相关性判断较粗。方案 B 是 cross-encoder rerank，把 query 和 candidate 一起送进模型做交互打分，优点是排序更准，缺点是慢。工程上最常见的组合就是“bi-encoder 召回 + cross-encoder 重排”，DeepIntel 现在就是这个方向，所以它前面要先做 Top-K 截断，后面再把更贵的 reranker 用在少量候选上。

7.8 生产追问：RAG 事故如何定位到 retrieval、ranking 还是 generation

固定判断顺序

我会先看“正确 chunk 有没有进 Top-K”，再看“有没有被排到足够靠前”，最后才看“模型有没有按证据说”。也就是先查 retrieval，再查 ranking，再查 generation。

诊断路径

如果 Hit@K / Recall@K 很低，大概率是检索层问题，优先怀疑 chunking、embedding、BM25 兜底、多路召回。若 Hit@K 不低但 MRR 很低，说明“找到了但排得不前”，优先查 rerank、RRF 融合和 Top-K 截断。若 Context Recall 不低，但 Faithfulness 或 Answer Relevancy 低，就更像生成层问题，通常是提示词约束、引用组织、回答模板或模型输出跑偏。

这里也有两个常见误区。第一是看见答案错就立刻换模型；第二是看见召回高就以为检索没问题。更成熟的判断是：召回、排序、生成分层排查。DeepIntel 的离线评估代码本身就是按这个思路拆的，检索层和生成层分开记录，而不是只留一个总分。

7.9 trace 为什么要分成几条线

- agent_trace: 高层流程事件，适合 UI 和时间线回放。
- guardrail_trace: 安全和路由决策，适合解释“为什么被拦”。
- tool_call_audit: 单次工具调用明细，适合法证级追踪。

设计观点

把所有东西都塞进一条 trace 里，看起来简单，实际上后面既不好查，也不好给不同角色消费。高层事件、护栏决策、单次调用明细这三类数据天然就是不同层次，拆开才有意义。

深入分析

很多系统刚开始都会把“可观测性”理解成“有日志就行”，再进一步可能是“有一条 timeline 就行”。但当系统同时涉及业务流程、治理决策和外部工具时，单一时间线很快就会失效。原因不是数据量大，而是语义层次不同。产品和前端更关心用户可见的高层进度，例如 Planner 开始了、Report 生成了、任务完成了；安全和治理更关心“为什么这个动作被拦了”“为什么这个节点进入 awaiting_approval”；法证和排障则需要知道单次工具调用的参数、结果摘要、错误类别和耗时。

如果不拆线，后果通常有两个。第一，查询和展示成本会上升，因为每个消费者都得从同一条混杂日志里自己再切语义层。第二，边界会变模糊，尤其是当你想

回答“这个错误到底是业务失败、治理阻断还是外部工具异常”时，很难快速切干净。

所以拆 trace 的核心不是“多存几份”，而是让不同层次的问题有不同层次的数据承载。面试时最好直接说：`agent_trace` 服务于高层流程可见性，`guardrail_trace` 服务于治理解释，`tool_call_audit` 服务于单次调用法证。这样表达会非常清晰。

8 第七轮拷打：哪些能吹，哪些不能吹

这一轮面试官在试什么

这轮最考验真实性。很多项目不是死在技术上，而是死在表述上。你如果把设计稿说成已经上线，或者把半成品说成生产级，懂行的面试官一追就穿。这里必须把“已做完”“已部分落地”“还在设计”分开讲。

这一轮最常见的追问

- 你现在到底能不能说自己做了治理闭环。
- 哪些能力已经在代码里落地，哪些只是详细设计。
- 历史上最关键的 bug 是什么，修复后语义有什么变化。
- 如果我现在把系统部署到生产，会先卡在哪几个现实问题上。

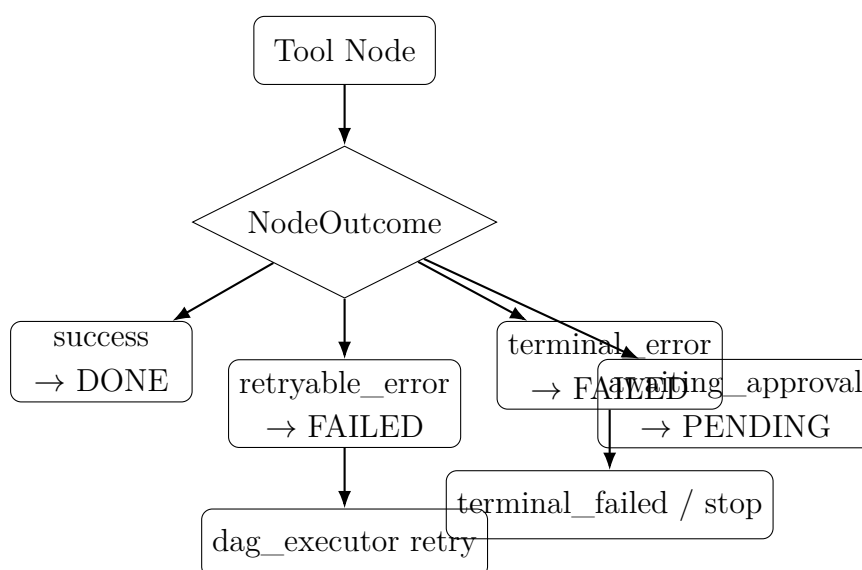
8.1 当前真实完成度应该怎么诚实表达

建议表述

当前系统已经具备研究主链路、DAG 编排、反思校验、SSE trace 和基本预算治理能力，但完整治理闭环仍在演进中。更准确的说法是：它已经是一个能工作的研究型 Agent 系统，但离严格意义上的生产级执行治理系统还有距离。

能力项	当前状态	面试表述建议
主任务生命周期	已打通	可以说“已实现主链路”
失败语义	已从历史 bug 修到显式 outcome	可以说“失败语义已显式化”
工具审计	已有独立 <code>tool_call_audit</code> 并改为运行中持续落库	可以说“审计已具备 crash-safe 强化版，但仍不是全量法证平台”
会话级预算 breaker	已有硬熔断与中途预算状态持久化	可以说“已支持会话级硬熔断，但 token/cost 仍部分依赖估算”
动作级审批	已接入独立 reviewer 分支与 approval API	可以说“已实现动作级审批主链路，但恢复粒度仍以批次边界为主”
Harness supervisor	已有图外 supervisor 文件协议与恢复快照	可以说“已实现最小可用任务级恢复控制，但还不是多 worker 生产调度器”
MCP 安全接入	已有 registry + policy proxy + allowlist 闸门	可以说“已接入安全通路，但默认业务图还未大规模使用 MCP 节点”

8.2 失败语义为什么是治理第一优先级



历史 bug 与当前状态

这个系统历史上最关键的问题，是工具节点失败后聚合器可能仍然把节点当成完成。这个 bug 现在已经修掉了，工具节点通过 `NodeOutcome` 回传显式状态，聚合器按 `success` / `retryable_error` / `terminal_error` / `awaiting_approval` / `skipped` 更新节点。它之所以值得在面试里重点讲，是因为它直接体现了“为什么失败语义必须显式建模”。

原因

为什么失败语义是治理第一优先级，而不是审批、预算或者 MCP？因为只要失败语义不对，后面所有治理都建立在错的状态之上。审批系统的前提是你知道当前动作到底是在成功、失败、等待审批还是已经终止；预算系统的前提是你知道当前节点是否还应该继续消耗资源；恢复系统的前提则是你知道上一次中断时到底停在了哪种失败态。历史上那个“工具失败但节点被标 done”的 bug，本质上就是完成态被污染。

选型

这里至少有两个方案。方案 A 是继续把失败信息藏在 tool history 或异常文本里，由聚合器自己猜当前节点到底算成功还是失败。优点是改动少，缺点是语义脆弱。方案 B 是让工具层显式回传一份最小控制契约，也就是 `NodeOutcome`，把成功、可重试失败、终止失败、等待审批和跳过这些状态直接告诉调度层。这里必须选方案 B，因为控制不能建立在猜测审计日志的基础上。

如何做

当前实现里，每个工具节点执行后都会显式回传 `NodeOutcome`，其中至少包含 `success` / `retryable_error` / `terminal_error` / `awaiting_approval` / `skipped` 这些控制语义，以及错误类别、重试次数、工具调用 ID 等最小信息。聚合器不再根据异常文本猜测结果，而是按这份显式契约更新 DAG 节点状态和 runtime status。与此同时，Browser 的本地 fallback 也不再被上层误记成成功页面，而是会把真实错误向上转换成 `retryable` 或 `terminal` 失败。

效果

这样做以后，失败语义从审计层附带信息升级成了调度层硬约束。工具失败不再会被吞掉，后续路由、预算治理、审批暂停和恢复决策都有了可信状态基础。面试时可以很明确地说：失败语义已经显式化，这个历史 bug 已经修掉，不应继续被当成现存问题对外表述。

8.3 会话级预算治理现在到了什么程度

原因

预算治理最容易被答成“有个 `max_tokens` 就行”，这其实远远不够。研究型 Agent 的成本不是单一来源，至少包括模型 token、外部工具调用、网页抓取时长、重试次数和整体 wall clock。只盯 token，会漏掉 Browser 一直超时、MCP 工具反复重试、或者 Planner 把图拆得过深这类真实成本问题。

选型

这里至少有两个方案。方案 A 是 observability-only，只打 warning，不改系统行为。优点是实现最简单，对现有流程侵入小；缺点是出了预算事故你只能复盘，不能止损。方案 B 是 breaker 模式，让预算阈值真正参与路由决策。比如超过某个阈值后停掉可选 Web 扩展、禁止新的 replan、必要时提前进入部分报告。这里必须选方案 B，因为治理的定义就是让规则能改变执行，而不只是描述执行。

如何做

当前项目已经不是只有长度档位了，图内已经接入 session-level budget breaker，包括 max_tool_calls、max_wall_clock_seconds，以及 max_total_tokens / max_cost_usd 的运行时累计通道。预算治理分三层。第一层是计量层，尽可能记录 usage、cost、tool call 和耗时；拿不到精确值时明确标估算，并把 estimated 与 usage_source 写入审计。第二层是策略层，在 80% 左右给 warning，在硬阈值时停止可选扩张，在更高风险时禁止 replan。第三层是收口层，即使被硬熔断，也不会让任务直接烂尾，而是跳过剩余工具节点，进入分析与部分报告，同时把预算中间态持续落到 session_budget_state。

效果

这样做以后，预算治理不再只是一个提醒系统，而是真正能改变执行路径的硬约束。系统会在预算接近危险区时发出告警，在硬阈值时主动止损并优雅降级。这里也必须诚实说明边界：当前 token/cost 通道虽然已经接到 LLM usage 和 budget state，但它仍不是绝对精确计费系统，因为上游 provider 不总返回 usage，tool 侧也未必暴露真实账单，所以系统只能做到可精确则精确，不可精确则显式标记估算。

8.4 法证级审计现在到了什么程度

当前结论

现在已经不只是 tool_histories 运行时累积状态了，系统已新增独立的 tool_call_audit 表，把单次工具调用拆成独立记录，并在 workflow 运行过程中持续 upsert。这样 session 结果查询和审计查询不再混在一行 JSON 里，中途崩溃时也不会像以前那样整批丢光。

仍未闭环的点

现在工具审计和预算状态已经前移到运行中持续落库，所以 crash-safe 能力比之前强很多。但它仍不是“无限细粒度恢复”：当前最可靠的恢复语义仍是批次边界恢复，不承诺单节点执行到一半的原地续跑。

深入分析

“法证级审计”这四个字不能乱用。真正的法证关注的是：系统在某个时刻为什么做了某个动作、当时依据是什么、参数是什么、结果是什么、谁批准的、是否经过脱敏、崩溃前最后一条可确认事实是什么。单靠最终 session JSON 或一条高层 trace，是达不到这个要求的。

当前系统之所以比早期状态前进了一大步，是因为工具调用和预算状态已经从“流程结束后统一汇总”前移成“运行中持续落库”。这意味着进程中途崩掉时，至少已经完成的调用明细、预算累计和关键状态切换不会整批蒸发。这就是 crash-safe 的核心价值，不是恢复得有多优雅，而是证据先别丢。

但也必须守住边界。现在的强项是法证记录比以前完整得多，不是说恢复已经精确到单条工具流内部。批次边界恢复依然是主语义，所以更准确的表述应该是“已经具备 crash-safe 的实时审计闭环雏形，并支持批次边界恢复”，而不是“已经完成全量执行回放平台”。这类表述差异，面试里非常关键。

9 第八轮拷打：邪恶 Agent 怎么管住

这一轮面试官在试什么

这轮不是问“你会不会做 Agent”，而是问“你敢不敢把 Agent 放进真实系统”。如果面试官开始追问审批、预算、审计、MCP、恢复、权限边界，说明他在看你有没有治理视角，而不只是能力视角。

这一轮最常见的追问

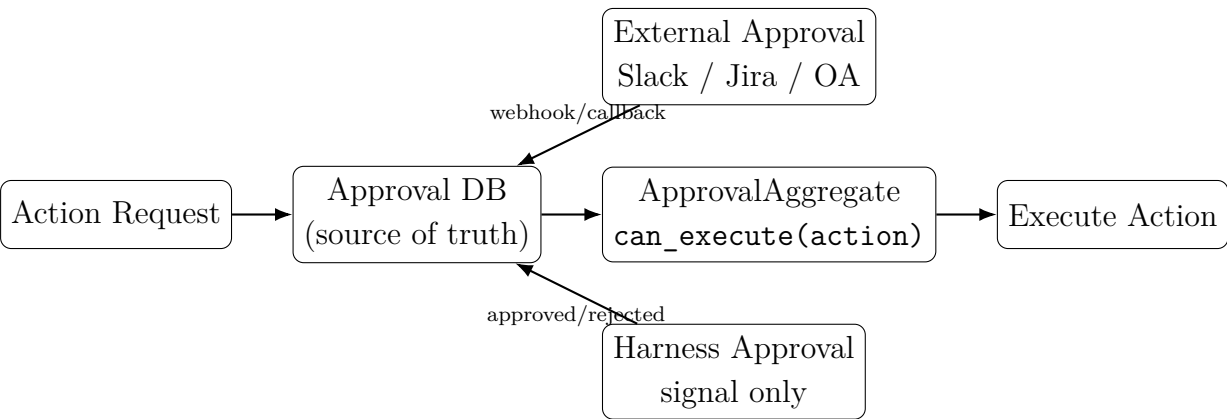
- 失败后到底是重试还是终止，谁来裁决。
- 高风险动作要不要审批，审批状态到底谁说了算。
- tool audit 能不能支撑事后法证，而不只是页面展示。
- 如果明天接 MCP 工具，为什么不能直接把它们当普通 Tool 放进图里。

9.1 治理闭环现状判断

一句话

现在系统已经有 workflow、trace、tool audit、budget breaker、动作级审批、Harness supervisor 和 MCP policy proxy 的主干治理链路，但恢复粒度、MCP 默认用量和计费精度仍有边界。

9.2 审批状态源（Approval Source of Truth）



固定立场

动作级审批的最终权威源必须是本地数据库审批单。Harness、Slack、Jira、ServiceNow 这类系统都可以作为审批信号源，但最终 `can_execute(action)` 的判断必须由本地审批聚合根负责。

深入分析

这道题背后其实是在问“source of truth 到底怎么定”。很多团队一开始会误以为，用现成平台做审批最省事，所以直接把外部状态当执行依据就行。但一旦动作进入自动化执行流，审批不再只是一个页面状态，而是一个会影响幂等、恢复、重试和危险动作下发的强控制条件。这时如果没有本地单一真相源，系统几乎必然会出现状态漂移。

为什么数据库审批聚合根最适合做最终裁决？因为动作执行发生在你的系统里，执行前最后一跳必须依赖一个你能稳定查询、能事务更新、能做版本控制、能和审计关联的本地状态。如果直接去问 Harness、Slack 或 Jira 当前是否 approved，你会立刻遇到几个问题：远端状态变化有延迟，回调可能丢失，远端 UI 和本地执行记录不在同一事务里，失败重试时也缺统一幂等点。

所以最稳的架构不是拒绝外部审批，而是重新定义它们的角色：外部系统负责承载人类交互和审批信号，本地数据库负责承载最终执行裁决。只要把这层权责讲清楚，面试官通常会认可你不是在排斥外部平台，而是在保护动作执行的一致性边界。

来源	谁是权威	优点	问题
LOCAL DB	系统自己	单一真相源、查询直观、审计强	工程量最大
HARNESS	Harness 平台	UI、RBAC、流程现成	易和业务状态漂移
EXTERNAL	Slack/Jira/OA 等	贴合企业习惯	远端状态和本地状态同步复杂

Listing 5: ApprovalAggregate 的建议最小字段

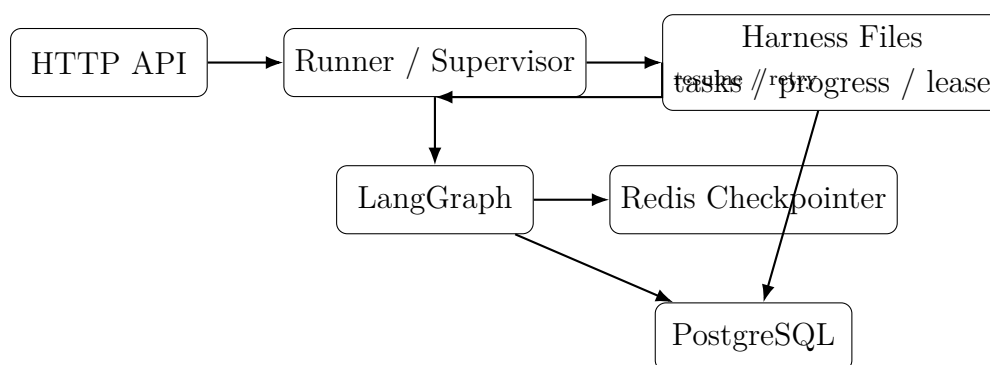
```
approval_id
action_id
approval_source
external_id
status
version
approved_by
expire_time
```

9.3 为什么外部审批只能是信号，不能是唯一裁决

原因

因为只要允许多个 source of truth 并存，就一定会遇到状态漂移：例如 Harness 已经批了，但 DB 仍是 pending；或者 Slack 上已经通过，但 agent 因为没看到本地状态变化再次发起执行。真正要避免的不是“查不到谁批了”，而是“重复执行危险动作”。

9.4 Harness 外挂设计



架构答案

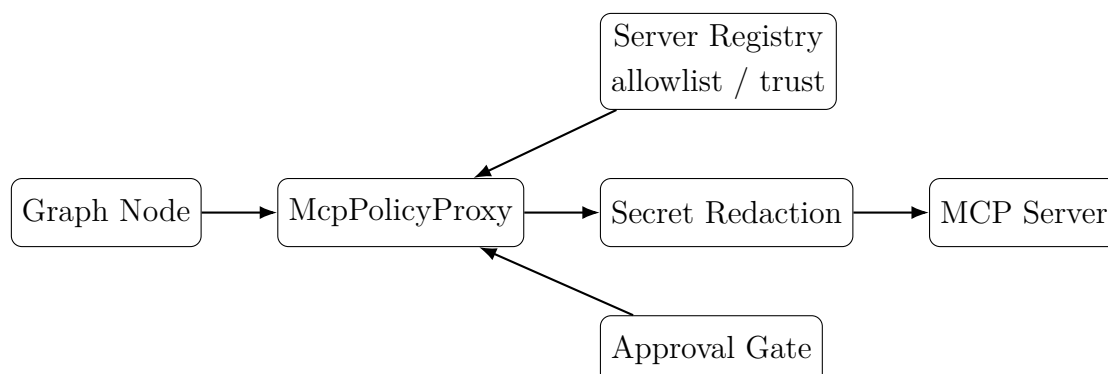
Harness 应该放在 LangGraph 外层。LangGraph 负责编排业务步骤，Harness 负责任务租约、失败恢复、暂停恢复、预算调度和 supervisor 语义。数据库负责审批、审计和预算事实，Harness 负责 lease、checkpoint 选择和 worker 协调。当前仓库已经落了最小可用实现：有 `harness-tasks.json`、progress log 和 state snapshot，但还不是完整的分布式 worker 调度系统。

深入分析

Harness 设计的关键不是“多一份文件”，而是明确任务级控制面和业务执行面的解耦边界。一个长任务系统真正难的，不是把 DAG 跑完，而是在失败、暂停、恢复、多 worker、审批等待这些现实条件下仍然能保持状态一致。Harness 文件之所以重要，是因为它把这些任务级信息从短生命周期的 HTTP 请求和进程内存里抽了出来，变成跨会话可恢复的外部上下文。

这也是为什么 progress file 和 tasks file 在这套设计里不是附件，而是恢复上下文本身。任务一旦跨会话执行，agent 的上下文窗口不再可靠，单靠口头说明或者控制台日志也不可靠。真正能支撑恢复的，是结构化任务状态、最近 checkpoint、当前 batch、审批等待点、预算使用量这些可以被下一次执行直接消费的数据。当然，这里也要诚实讲代价。外置 Harness 以后，一定会引入双写和状态同步问题，所以必须分域定权：审批、预算、审计以 DB 为准；worker lease、checkpoint 选择、恢复编排以 Harness 为准。只要把这个边界说清楚，面试官反而会认为你想清楚了复杂度，而不是机械套了一个 supervisor 名词。

9.5 MCP 安全接入原则



固定表述

MCP 是协议，不是治理系统。真正安全的接法不是把 MCP Tool 直接映射成 LangChain Tool，而是先过 McpPolicyProxy，把 server allowlist、tool allowlist、只读默认和 secret redaction 做成强闸门。当前仓库里这条代理通路和 mcp_server_registry 已经落地，但 Planner 还不会默认大量生成 MCP 节点，所以它更适合表述成“安全接入通路已存在”，而不是“业务主流量已全部走 MCP”。

深入分析

MCP 的危险点在于它天然会给人一种错觉：协议既然统一了，接上就能安全使用。实际上协议只解决了“如何发现和调用工具”，没有解决“这个工具该不该被信任、这组参数会不会越权、返回结果有没有敏感字段、写操作需不需要审批”。也就是说，MCP 统一的是接口，不是治理。

如果你把 MCP server 暴露出来的工具直接映射成普通 Tool，有三个高风险后果。第一，server 信任边界缺失。你可能根本不知道这个 server 是谁提供的、是否可信、tool 列表有没有变更。第二，能力边界缺失。同一个协议既可以暴露只读查询，也可以暴露写文件、发消息、调内部系统，如果没有 allowlist 和只读默认，agent 会天然尝试更强能力。第三，数据边界缺失。很多 MCP 返回值会带敏感字段、token、内部 ID，如果不在代理层做 redaction，这些数据会直接进入审计和上下文。

所以 policy proxy 的核心价值，是在业务图和外部工具生态之间插入一层强治理边界。它不是为了让调用更麻烦，而是为了把“能调用”变成“在规则内可调用”。这一层如果讲得足够清楚，面试官通常会把你归到真正考虑过安全接入的人，而不是只会接协议的人。

9.6 生产追问：MCP 协议层、传输层和治理层怎么拆

工程结论

MCP 底层消息是 JSON-RPC 2.0，本地常用 stdio，远程推荐 Streamable HTTP。工具 schema 要同时关注 inputSchema 和 outputSchema：输入决定能不能执行，输出的 structuredContent 决定能不能进入后续上下文和审计。

怎么回答既不空泛也不吹过头

如果面试官问协议，最容易答错的点就是把“传输方式”和“消息格式”混在一起。更准确的说法是：MCP 不管跑在本地还是远程，底层消息格式统一都是 JSON-RPC 2.0。它解决的是“client 调 server 的 method，server 返回 result 或 error”这件事，本质上就是一个轻量 RPC 规范。之所以选它，不是因为花哨，而是因为 JSON 可读、跨语言实现简单、调试成本低，Python 和 TypeScript 写的 server 都能讲同一种消息语言。

本地传输：stdio

本地场景下，MCP 最常见的不是网络，而是 stdio。Client 启动 MCP Server 这个子进程，然后通过标准输入发 JSON-RPC 请求、从标准输出读 JSON-RPC 响应。两个进程在同一台机器上运行，走的是操作系统管道，不经过网卡，也不需要 TCP/IP 协议栈，所以延迟低、配置简单、没有额外的网络暴露面。

远程传输：Streamable HTTP

远程场景下，现在推荐的是 Streamable HTTP。核心不是 WebSocket，而是单个 HTTP 端点同时承接请求和响应。Client 通过 POST 发送 JSON-RPC 消息；Server 可以直接返回普通 JSON，也可以在需要流式输出时返回 SSE 流。关键点在于：这

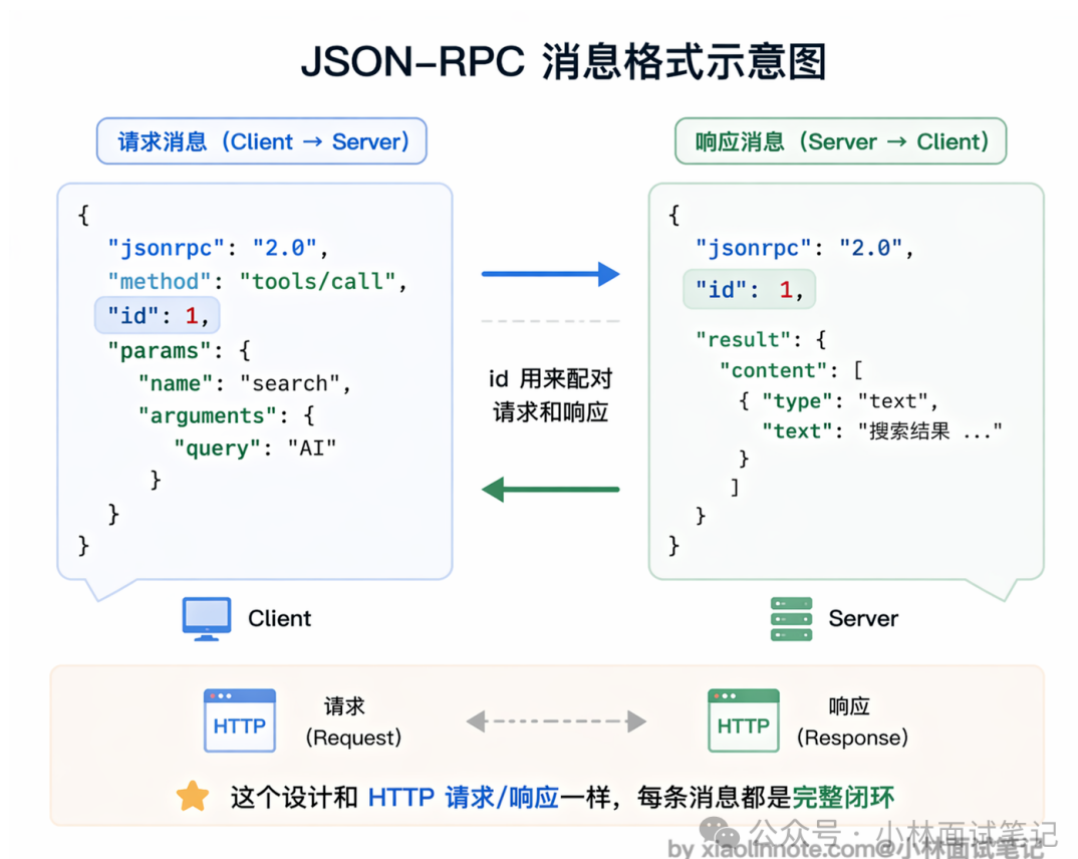


图 9: MCP 底层消息格式统一使用 JSON-RPC 2.0: 传输方式可变，消息结构不变

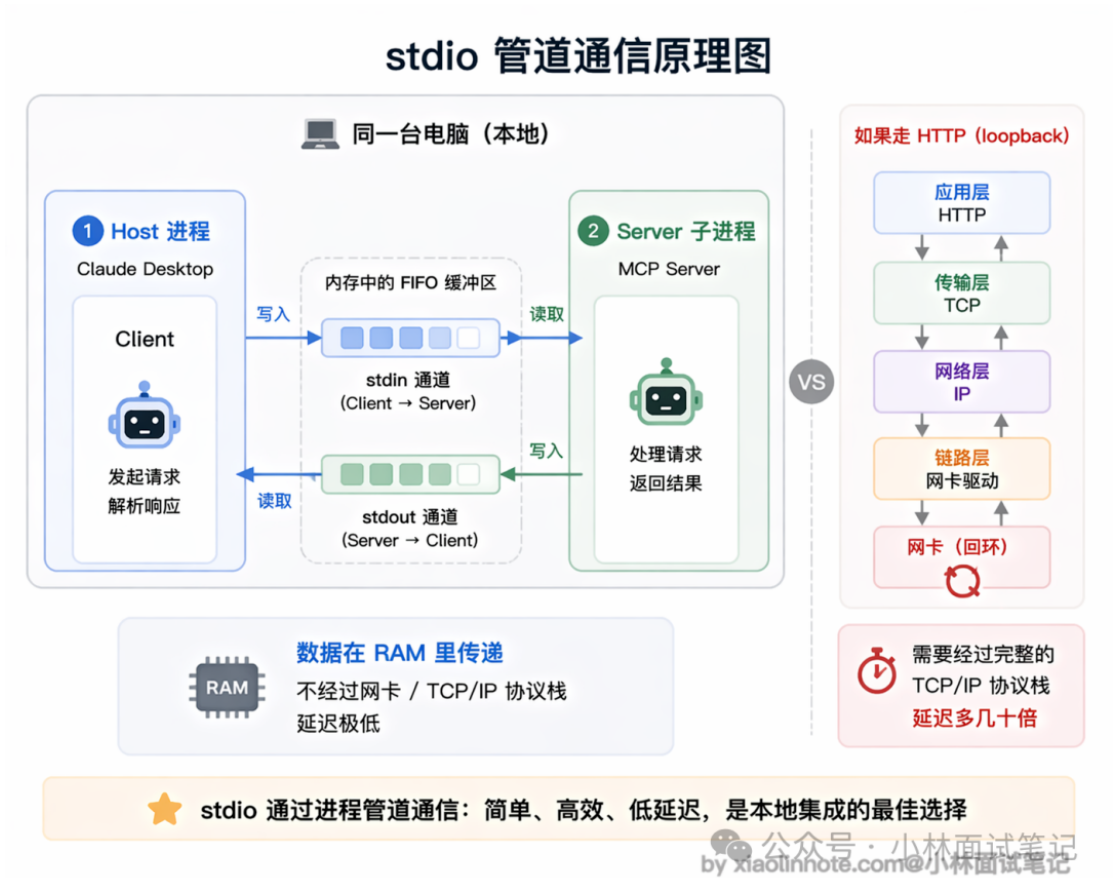
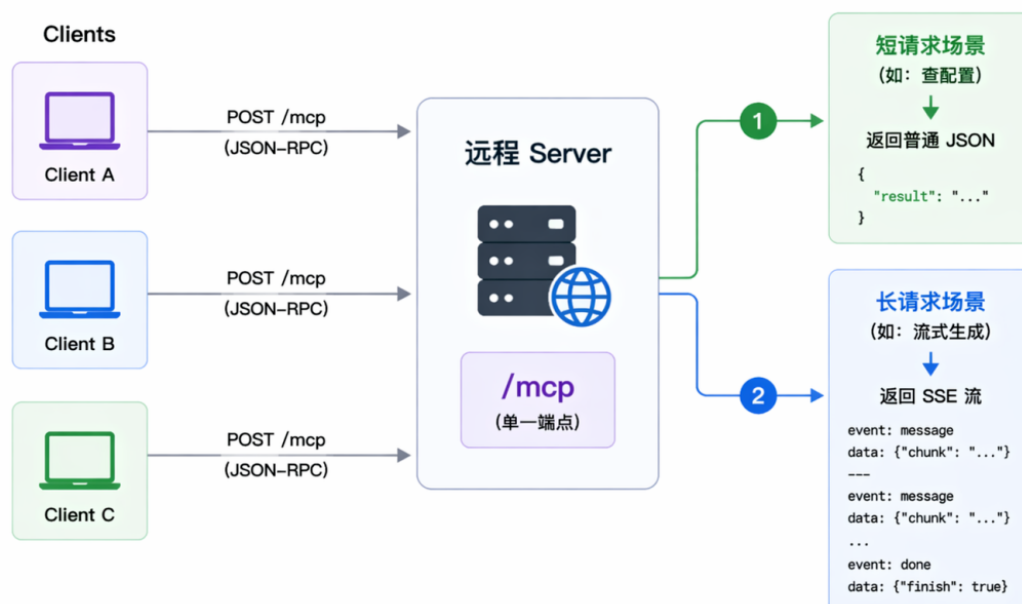


图 10: stdio 本地通信：Client 把 Server 当作子进程启动，通过 stdin/stdout 交换 JSON-RPC 消息

里并没有抛弃 SSE，而是把它收编进单端点架构里，变成“需要流式时的返回形式”，而不是早期那种额外维护一条独立 SSE 长连接。

Streamable HTTP 单端点架构图



一个端点处理两类场景，按需选择

公众号：小林面试笔记
by xiaolinnote.com / 小林面试笔记

图 11: Streamable HTTP: Client 用 POST 发 JSON-RPC，请求返回普通 JSON 或 SSE 流

为什么早期 SSE 双端点方案被弃用

这里要再补一个时间线。早期 MCP 远程方案是 HTTP + SSE 双端点：一条 POST 端点负责发请求，另一条 SSE 长连接负责收流。这套方案在早期规范里存在过，但在后续规范演进里已经被标记为 deprecated，新项目更应该直接用 Streamable HTTP。原因并不玄学，就是双通道让状态管理、排障和部署都更复杂；而 Streamable HTTP 把同一轮交互收拢回一个端点，语义更简单，也更适配负载均衡和 serverless 场景。

schema 这一层应该怎么补

如果面试官问的是 schema，不要答成“就是一个 JSON”。更准确的说法是：MCP 工具暴露结构化输入规范，本质上接近 JSON Schema，用来声明参数类型、必填项、枚举和对象结构；2025-11-25 规范还把 outputSchema 和 structuredContent 的关系讲得更清楚。DeepIntel 本轮已经把 policy proxy 从只校验输入扩展到校验

HTTP + SSE 双端点问题示意图（老版 MCP 远程传输）

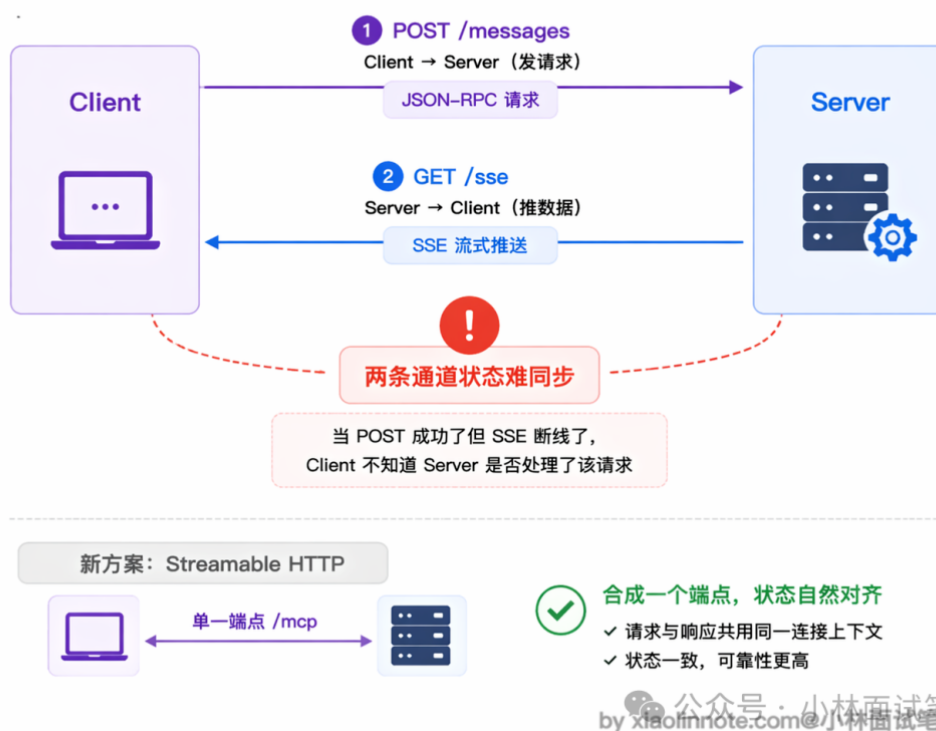


图 12: 早期 MCP 远端传输: POST 与 SSE 双端点分离, 链路更复杂



图 13: MCP 远端传输演进: 从双端点 SSE 过渡到单端点 Streamable HTTP

输出结构，坏的 `structuredContent` 不再直接进入 `summary`、`hash` 和后续 Agent 上下文。

最后再落回 DeepIntel

这里也至少有两个接法。方案 A 是把 MCP 当成“统一工具接口”，接上就直接让 Agent 调；优点是快，缺点是把协议误当治理。方案 B 是像本项目这样，把 MCP 当成外部能力暴露协议，但前面一定要加 `policy proxy`，先做 `server trust`、`tool allowlist`、只读默认、审批和 `redaction`。DeepIntel 明显选的是方案 B，所以面试里最稳的表述不是“我们已经全面 MCP 化”，而是“我们已经把安全接入通路搭好了”。

9.7 为什么 Agent 调工具时更容易出事

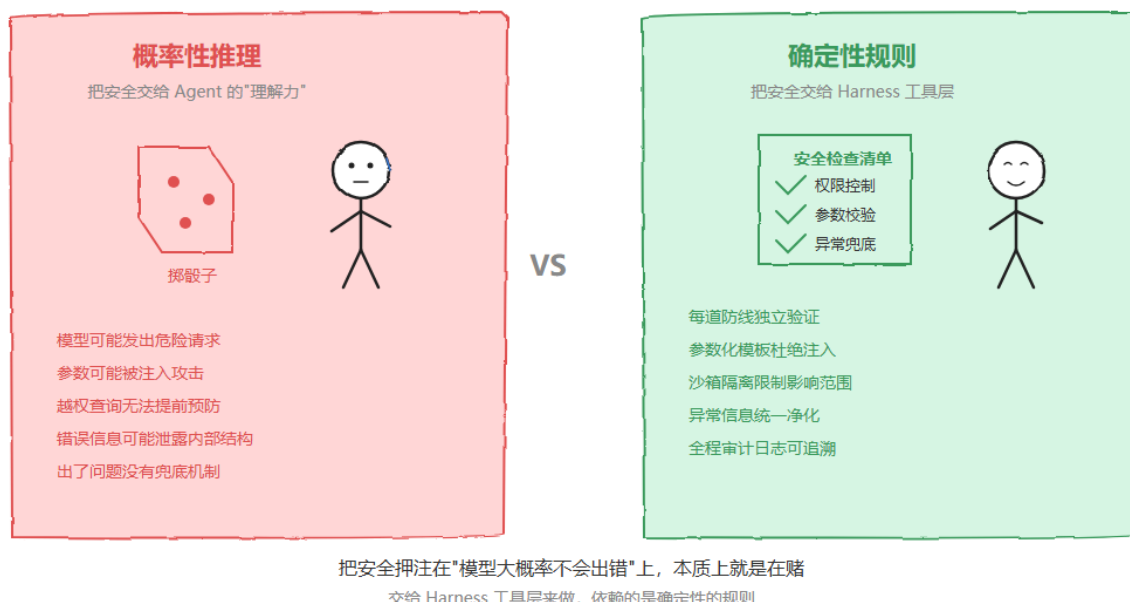


图 14: 概率性推理 vs 确定性规则：安全不能押在“模型大概率不会出错”上

核心判断

工具调用的风险不在于 Agent 会不会说错话，而在于它一旦带着错误参数碰到真实资源，后果会立刻变成工程事故。所以安全不能建立在“模型大概率不会犯错”上，而必须建立在 Harness 工具层的确定性规则上。

原因

面试官问“Agent 调工具，风险到底在哪里”，本质不是在考你会不会调 API，而是在考你有没有把工具调用链路当成高风险执行面来治理。数据库查询、内部检索、发消息、写文件，这些动作一旦从“语言输出”变成“真实执行”，风险类型

就从幻觉升级成越权、注入、数据外泄和副作用失控。

选型

这里至少有两个方案。方案 A 是把安全主要押在模型理解力上，相信模型足够聪明就不会发危险请求。优点是接入快、系统侵入小；缺点是它本质上是概率控制，遇到 prompt injection、越权诱导、异常路径时没有硬约束。方案 B 是在 Agent 和真实资源之间加一层 Harness 工具层，把每次调用都当成独立请求，串行经过权限控制、参数校验和异常兜底。这里必须选方案 B，因为安全要求是确定性的，不能建立在“模型大部分时候应该没问题”这种判断上。

深入分析

很多系统早期会误以为，只要把 system prompt 写得更严格一点，或者让模型“理解不要乱查数据库”，安全问题就差不多解决了。这种做法能降低一部分误用，但解决不了根本矛盾：模型的推理输出是概率性的，而权限、参数合法性和敏感信息边界是确定性的。两者不是一个层级的问题。

也正因为如此，真正成熟的工具接入层不会假设 Agent 可信，而是把 Agent 当成一个可能犯错、可能被诱导、也可能被脏输入污染的请求发起者。工具层的职责不是“帮 Agent 把请求转发得更漂亮”，而是先判断这个请求到底有没有资格执行、参数是否合法、失败后会不会把内部结构漏出去。把这层说清楚，面试官通常会意识到你理解的不是 prompt engineering，而是执行面治理。

9.8 Harness 工具层怎么挡住越权和带毒参数

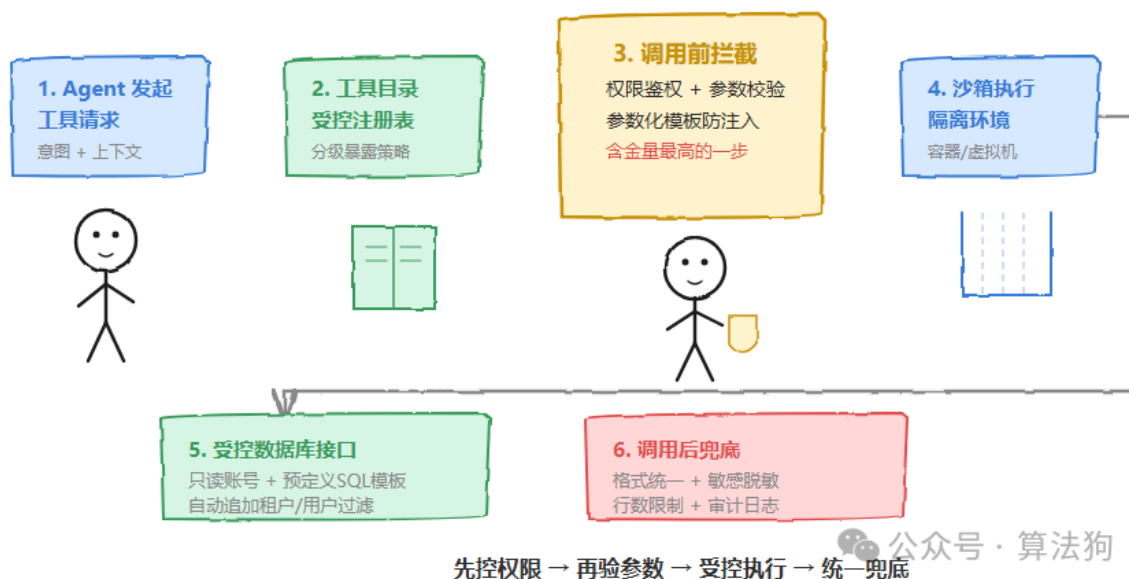


图 15: 先控权限，再验参数，受控执行，统一兜底

架构答案

最稳的设计是把 Harness 放在 Agent 和真实工具之间，所有工具请求先进入受控目录，再经过调用前拦截，最后进入沙箱和受限数据库接口。整个链路至少有三道串行防线：权限控制回答“你有没有资格调这个工具”，参数校验回答“你传进来的参数是不是安全合规”，异常兜底回答“执行失败或返回敏感内容时怎么安全收口”。

选型

这里也至少有两个方案。方案 A 是只在工具函数内部各自写保护逻辑，让每个工具自己判断权限、自己处理异常。优点是实现快，局部改动小；缺点是规则分散、容易漏判，而且不同工具的行为不一致。方案 B 是把权限、schema、审批、脱敏、审计收拢到统一 Harness 工具层，工具本体只负责业务能力，治理规则由中间层集中裁决。这里更应该选方案 B，因为统一治理才能做到一致审计、集中升级和纵深防御。

流程拆解

整条链路可以概括成八个字：先控权限，再验参数，受控执行，统一兜底。第一步，Agent 发起工具请求，但它表达的只是意图，不应该直接拥有数据库或外部系统访问权。第二步，请求进入 Harness 工具目录，每个工具都要提前声明能力描述、参数 schema、权限级别和是否只读。Agent 能看见哪些工具，由暴露策略决定，而不是想调什么就调什么。

第三步是调用前拦截，也是含金量最高的一步。权限侧要结合身份、角色、租户、会话上下文做综合鉴权；参数侧要做类型、长度、枚举、必填项校验，并且强制数据库访问走参数化模板，禁止自由拼接 SQL。第四步是沙箱执行层。就算前面都通过了，也不能假设工具实现一定安全，所以仍要限制网络、文件系统写入、CPU/内存和执行时长，把副作用约束在小范围里。

第五步是受控数据库接口。这里不应该给工具完整读写权限，而是给只读账号、只读视图和预定义 SQL 模板，并在底层自动追加租户或用户范围过滤条件。第六步是调用后兜底。结果要先统一结构，再做敏感字段脱敏、返回行数限制和错误净化，最后把谁调用了什么工具、查了什么范围、为什么被拦或被放行写入审计日志。这样下游拿到的始终是规范化、安全化的数据，而不是裸结果。

两类典型风险怎么回答

第一类是参数带毒，也就是注入风险。工程判定口径是：先过 schema，不合法参数直接拒绝；数据库访问强制参数化模板，堵死自由拼接；执行异常统一包装，不能把原始 SQL 报错和内部结构直接暴露给模型。第二类是调用越权和数据越界。工程判定口径是：第一层在工具目录和鉴权器卡调用资格，第二层在数据库接口自动追加租户/用户过滤条件。前一层漏判了，后一层仍然能兜住。

Listing 6: Harness 工具层统一输出结构示意

```
{
  "status": "success | denied | safe_error",
  "source": "harness_tool_layer",
  "data": { "...": "masked business payload" },
  "meta": {
    "tool_name": "query_transactions",
    "row_count": 20
  },
  "safety": {
    "masked": true,
    "sanitized_error": true,
    "audited": true
  }
}
```

9.9 五类幻觉怎么治理

- 知识幻觉：先检索，证据不足就拒答，不能拿模型记忆补业务事实。
- 工具幻觉：Observation 必须来自真实工具返回，不能让模型伪造 tool result。
- 参数幻觉：schema 校验不过就不执行，工具调用是接口，不是聊天。
- 证据幻觉：分析和报告必须能回溯到证据和 citation。
- 行动幻觉：高风险工具默认不执行，需要动作级审批门。

10 第九轮拷打：高压追问怎么接

这一轮面试官在试什么

到这一轮，面试官往往已经默认你做过一些东西了。他接下来要看的是：当问题被压得很细、很尖锐时，你能不能既守住事实边界，又把架构判断讲透。下面这些问题适合直接背熟成口头版。

高压回答原则

高压追问时，不要急着追求“说满”。最稳的做法是先给结论，再补理由，最后补边界。也就是先说“我为什么这样做”，再说“备选方案为什么没选”，最后说“哪些部分我现在还没做完”。这样能显著降低被继续追着打穿的概率。

10.1 为什么选择 LangGraph，而不是直接做多 Agent 对话

答案

多 Agent 对话更适合开放式协作，不适合强流程研究任务。研究任务要求节点、依赖、预算、审计和恢复都可显式表达，LangGraph 这种状态机编排比纯消息流更适合工程化落地。

10.2 为什么动作审批的权威源必须是 DB

答案

因为动作执行是高风险事实，不能依赖多个远端状态拼凑出来。数据库里的审批聚合根必须是唯一裁决者，否则系统会出现“远端已批、本地未落”“远端已拒、本地重试继续跑”这类危险分叉。

深入分析

这个问题本质上是在问 source of truth。动作审批如果没有单一权威源，系统迟早会出现状态漂移。比如 Harness 那边显示 approved，Slack 消息里也有人点了批准，但本地任务状态还停在 pending；或者外部系统已经拒绝，本地因为网络抖动没收到回调，任务恢复时又把动作继续发了出去。对于读操作，这类漂移已经麻烦；对于写文件、发消息、调用内部系统的高风险动作，这就是事故入口。至少有两个方案。方案 A 是直接以外部审批系统或 Harness 状态作为最终依据，本地只做被动查询。优点是接入快，缺点是幂等、审计、一致性和恢复都会依赖远端可用性。方案 B 是数据库审批单作为聚合根，外部系统只提供信号，本地统一落库后再由本地状态决定是否 can_execute。这里必须选方案 B，因为动作执行最终发生在你的系统里，责任也落在你的系统里，裁决权就不能外包。再往深一点看，DB 作权威源的意义不只是“查起来方便”，而是让审批和执行共享一个可事务化的判断平面。这样才能把 approval_id、action_id、approved_by、expired、decision_version 这些信息和执行行为绑定起来，形成真正可追责的闭环。面试时把这一层讲明白，会显得你不仅懂 workflow，还懂高风险动作系统的责任边界。

10.3 为什么外部审批不能直接作为执行依据

答案

外部审批更适合作为输入信号，不适合作为最终执行前判定。标准做法是外部系统回调统一落库，更新本地审批状态，再由本地状态决定是否允许动作执行。这样幂等、审计和重试逻辑才有单一依赖点。

深入分析

这题和上一题相邻，但重点不同。上一题强调谁是最终裁决者，这一题强调为什么远端状态不能直接驱动执行。核心原因是执行系统需要的是稳定、可重放、可幂等的本地判断，而外部审批系统通常只保证“给你一个事件”，不保证这个事件会在你最需要的时候仍然可查询、可事务绑定、可与本地恢复流程完全同步。

如果直接以外部状态做执行前判断，会出现三类风险。第一是时序风险，回调到了但本地还没持久化，worker 就先继续跑了。第二是幂等风险，同一个 approve 事件重放两次，系统没有本地版本控制时可能执行两次。第三是一致性风险，外部系统后续撤销、过期或权限变更，本地没有统一聚合层就很难判断该如何处理已有任务。

所以外部审批最适合的角色是“信号源”，不是“执行开关”。你的系统应该消费这个信号，更新本地审批聚合，然后只相信本地聚合状态。这样一来，重试逻辑只看本地，恢复逻辑只看本地，审计逻辑也只需解释本地状态如何变化。对工程系统来说，这种单点依赖远比每次执行前都去问一圈外部世界稳得多。

10.4 为什么要在 LangGraph 外再加 Harness

答案

因为如果把失败恢复、预算熔断、审批暂停、worker 抢租约和任务恢复都塞进图里，图就会从“业务 workflow”膨胀成“超级控制器”。Harness 的价值在于把任务级控制面从业务级编排面剥离出去。

10.5 为什么 MCP 必须先过 policy proxy

答案

因为 MCP 只定义了如何暴露工具，不定义工具是否可信、参数是否越权、结果是否含敏感信息、危险写操作是否需要审批。没有 policy proxy，MCP 接入越多，系统越接近裸奔。

10.6 预算治理为什么不能只做软提示

答案

软提示只能告诉你“快超了”，不能真正止损。对长任务 Agent 来说，真正需要的是硬熔断加优雅降级：达到阈值后停止可选工具节点，禁止进一步 replan，最后基于当前证据输出部分报告。这才叫治理，不叫提醒。

深入分析

这道题的关键，不在于预算提醒有没有价值，而在于提醒本身不具备控制力。软提示适合人类在控制台里观察，例如“这个任务已经用了 70% 预算”；但 Agent 是自动执行系统，如果它收到提示却没有强制约束，执行路径不会自动收缩，成本照样继续增长。

从治理角度看，预算问题至少有两种处理模式。方案 A 是 observability-only，只打 warning，不改系统行为。优点是实现最简单，对现有流程侵入小；缺点是出了预算事故你只能复盘，不能止损。方案 B 是 breaker 模式，让预算阈值真正参与路由决策。比如超过某个阈值后停掉可选 Web 扩展、禁止新的 replan、必要时提前进入部分报告。这里显然要选方案 B，因为治理的定义就是让规则能改变执行，而不只是描述执行。

另外还要强调“优雅降级”这半句。硬熔断不是粗暴失败，尤其对研究任务来说，很多时候当前证据已经足够生成一个部分可用的结果。真正成熟的系统不是预算一到就直接报错，而是知道什么时候该停、停了之后怎样最小化价值损失。把这一点讲出来，会让你的预算治理显得完整，而不是只有一个 kill switch。

11 附录：Agent 岗底层推理拷打

11.1 底层题：不用框架写 multi-head attention，如何证明 shape 和 mask 没错

回答策略

这题不是为了背 Transformer，而是考你能不能把模型底层算子、张量形状、mask 语义和数值稳定性讲清楚。回答时先给单头 scaled dot-product attention，再扩到 multi-head，最后说明 causal mask、padding mask、softmax 稳定化和复杂度。

手撕时最稳的讲法

multi-head attention 的核心不是“复制很多个 attention”，而是把输入先线性投影成多组 Q 、 K 、 V ，每一组 head 在自己的子空间里算一次 scaled dot-product attention，再把多个 head 的结果拼接起来，最后过一层输出投影。公式可以压成四步：一， $Q = XW_Q$ ， $K = XW_K$ ， $V = XW_V$ ；二，按 head 数把最后一维 reshape 成 $[B, H, T, D_h]$ ；三，算 $\text{softmax}(QK^T/\sqrt{D_h})V$ ；四，把所有 head concat 回去再乘 W_O 。

至少有两个实现口径。方案 A 是纯 Python / 原生列表手写，优点是方便解释流程，缺点是代码长、性能差、容易写错维度。方案 B 是 NumPy 版本，优点是更接近工程实现，也更容易说明矩阵乘法、transpose 和 broadcasting。面试里更建议先给 NumPy 版骨架，再口头说明如果不用 NumPy 就是把矩阵乘法拆成三层循环。

Listing 7: NumPy 版 multi-head attention 最小骨架

```
import numpy as np

def softmax(x, axis=-1):
    x = x - np.max(x, axis=axis, keepdims=True)
    e = np.exp(x)
    return e / np.sum(e, axis=axis, keepdims=True)

def multi_head_attention(x, Wq, Wk, Wv, Wo, num_heads, mask=None):
    # x: [B, T, D]
    B, T, D = x.shape
    Dh = D // num_heads

    Q = x @ Wq
    K = x @ Wk
    V = x @ Wv

    Q = Q.reshape(B, T, num_heads, Dh).transpose(0, 2, 1, 3)
    K = K.reshape(B, T, num_heads, Dh).transpose(0, 2, 1, 3)
    V = V.reshape(B, T, num_heads, Dh).transpose(0, 2, 1, 3)

    scores = np.matmul(Q, K.transpose(0, 1, 3, 2)) / np.sqrt(Dh)
    if mask is not None:
        scores = np.where(mask, scores, -1e9)

    attn = softmax(scores, axis=-1)
    out = np.matmul(attn, V)
    out = out.transpose(0, 2, 1, 3).reshape(B, T, D)
    return out @ Wo
```

高级追问

最常见追问有五个。第一，为什么除以 $\sqrt{D_h}$ ：维度大时点积方差会变大，不缩放 softmax 容易过饱和。第二，mask 为什么放在 softmax 前：因为要从概率归一化前移除非法位置。第三，padding mask 和 causal mask 如何广播到 [B,H,T,T]。第四，长上下文复杂度为什么是 $O(T^2)$ ，工程上怎么用 KV cache、chunk attention 或检索外存缓解。第五，为什么这个底层题和 Agent 岗有关：因为 Agent 长上下文、工具轨迹压缩和证据窗口管理都依赖你理解 attention 的成本与边界。

12 第十轮拷打：优化记录总表与实施证据

这一章为什么必须单独写

这一章不是为了显得内容多，而是为了把“设计想法、代码落地、测试证据、面试话术、未完成边界”放在同一个地方。面试里最怕的不是不会讲概念，而是把设计稿讲成生产能力。这里的表格就是为了防止这个问题：每一项都必须能回答“现在到底做到哪一步，证据是什么，没做到的边界在哪里”。

真实性原则

本章的状态只有三类：已落地、设计已定、未完成。已落地必须有代码或测试证据；设计已定表示方案已经明确但代码还没完整接入；未完成表示面试时只能讲路线图，不能讲成当前能力。

主题	当前状态	证据	面试表达	边界
Harness v2 状态	已落地	app/governance/harness.py, tests/governance/test_harness.py	Harness 不再只是最小快照器，已经有 state、session config、backup、lease 和 summary。	仍不是完整分布式调度平台。
Harness 原子写入与状态锁	已落地	harness-tasks.json, .harness-state.lock 测试	长任务状态文件写入必须避免半写入；写前备份、临时文件、原子替换和轻量 transaction lock 是最低治理线。	仍不是完整多 worker scheduler。
Harness JSON 恢复	已落地	corrupt JSON backup recovery 测试	state 文件是恢复上下文，损坏时必须有明确恢复路径，不能静默失败。	backup 也损坏时会停止，不能自动重建任务语义。
Harness active marker	已落地	marker 创建和 idle 清理测试	active marker 用来区分 harness 是否接管任务生命周期，避免无任务时 hook 误触发。	marker 不是租约锁，不能替代 worker coordination。
Harness lease	已落地	expired lease 测试	lease 解决“谁正在做”的事实问题；没有 lease，恢复时无法区分正常执行和中断。	还没有多 worker 抢租约和工作目录隔离。

主题	当前状态	证据	面试表达	边界
Harness summary 与 依赖校验	已落地	summary、cycle、blocked propagation 测试	supervisor 必须能给出任务总览，并在循环依赖或失败依赖阻塞时显式标记 terminal failed。	依赖校验是安全网,不是完整任务调度器。
MCP allowlist	已落地	McpPolicyProxy 与 untrusted/tool 测试	MCP 只解决协议,不解决信任; server 和 tool 必须先 allowlist。	Planner 还未默认大量生成 MCP 节点。
MCP schema 校验	已落地	nested schema 与 array item 测试	tool inputSchema 是执行前最后一道结构化约束,不能只靠模型理解参数;当前已覆盖 object、array、enum、required 和 additionalProperties 子集。	仍不是完整 JSON Schema 引擎。
MCP 写操作 审批	已落地	write tool approval 与 raw args hash 测试	非只读或写工具必须返回 approval request; approval id 要绑定真实参数指纹,避免不同写入意图复用旧审批。	审批最终裁决仍要落到本地 DB 聚合根。
MCP redaction 与 稳定 审计 hash	已落地	configured redaction keys、stable result hash 测试	MCP 返回值进入上下文前必须递归脱敏,并记录稳定 result hash 与 fingerprint; hash 不应受字典字段顺序影响。	redaction key 需要随业务补充。
Skill scope 保护	已落地	scope metadata binding 与 manual enable scope 测试	手动启用不能绕过 tenant/project scope; metadata 也必须强约束 global/tenant/project 字段绑定。	还没有独立的企业级 RBAC UI。
Skill top-k	已落地	top-k truncation 测试	Skill 不是匹配越多越好,最终进入 prompt 的能力必须有上限。	top-k 仍是规则配置,不是学习排序。
Skill snapshot	已落地	resolved snapshot version 测试	session 必须绑定 skill version 和 prompt snapshot,避免运行中行为漂移。	还没有版本回收生命周期管理 UI。

主题	当前状态	证据	面试表达	边界
Agent tracing	设计已定	现有 agent trace 与 tool histories	面试时讲 tracing 是为了解释“为什么走到这里”，不是为了展示日志很多。	仍需更完整的 trace correlation。
Agent eval	设计已定	metrics 目录与 RAG/browser collectors	Agent eval 要同时看任务成功、证据质量、工具成本和恢复能力。	尚未形成统一 eval dashboard。
多 worker Harness	未完成	无完整调度测试	多 worker 需要 state lock、lease、独立 work-tree、rollback 隔离，不是简单开多个进程。	当前只能讲路线图。
Skill 自定义执行器	未完成	无 DAG node template 实现	Skill 当前影响 prompt 和工具边界；更高阶是自定义节点模板与执行策略。	面试不能说已经具备。
MCP 主流量化	未完成	Planner 仍不大量生成 MCP 节点	当前 MCP 是安全接入通路；主业务默认流量仍以 search/browser/rag 为主。	需要更多 server registry 和 planner policy。

13 第十一轮拷打：Harness 每一处细节怎么拆

一句话定位 Harness

Harness 是长任务 Agent 的任务级控制面。它不替代 LangGraph 的业务编排，也不替代数据库的审批、预算和审计事实源；它负责把跨上下文、跨进程、可恢复、可重试、可诊断的任务状态从短生命周期请求里抽出来。

13.1 为什么 Harness 不是“再加一个日志文件”

核心判断

如果一个 Agent 任务能在一次请求、一次上下文窗口、一次进程生命周期里完成，那么普通日志和内存状态可能够用。但深度研究任务不是这样：它会搜索、浏览、RAG、分析、反思、重规划，还可能等待审批、遇到预算熔断、发生浏览器超时、跨上下文恢复。此时日志不是附属物，任务状态也不是 UI 展示字段；它们是下

一次执行能不能继续的事实依据。

Harness 的价值就在这里：它把“任务现在在哪里、谁在执行、执行到哪个 checkpoint、失败是否可重试、等待哪个审批、预算用了多少、最近一次错误是什么”这些信息外置成可读取的控制面。这样当 agent 上下文被压缩、服务重启、worker 中断时，系统不是靠口头回忆恢复，而是读取 state file、progress log 和 DB runtime state 重新判断。

13.2 Progress file 为什么是 append-only

面试回答

progress file 必须 append-only，因为它承担的是审计和恢复线索，而不是当前状态视图。当前状态可以覆盖写，但历史动作不能覆盖写；否则一旦恢复失败，你无法知道是哪个 worker、哪个 checkpoint、哪个错误分类导致了当前状态。

深入分析

append-only 的价值不是“日志看起来完整”，而是能建立事件序列。长任务系统里很多 bug 不是静态状态能解释的，比如任务从 running 变成 retryable failed，再变成 running，最后又 awaiting approval。只看最终 JSON，你只能知道它现在暂停了；看 progress file，你才能知道中间经历了几次重试、是否发生 rollback、是否重复进入同一个失败分支。

这也是为什么日志格式要 grep-friendly。错误类型、session id、task id、checkpoint 和 category 要在同一行，而不是散在多行自然语言里。面试官如果追问“为什么标准化日志格式”，答案不是方便人看，而是方便 agent 和脚本恢复时快速定位。

13.3 Tasks JSON 为什么是 structured state

面试回答

progress file 负责事件历史，tasks JSON 负责当前可执行状态。恢复时真正驱动决策的是 tasks JSON：哪些任务 pending，哪些 in progress，哪些 failed 但可重试，哪些 blocked，哪个 worker 持有 lease，checkpoint 在哪里。

字段拆解

最小可用的 tasks JSON 至少要有六类字段。第一类是身份字段，例如 task id、title、priority。第二类是生命周期字段，例如 status、runtime status、attempts、max attempts、completed at。第三类是依赖字段，例如 depends on，用来判断 blocked propagation。第四类是恢复字段，例如 started at commit、checkpoint、state snapshot。第五类是 worker 字段，例如 claimed by、lease expires at、last

heartbeat。第六类是验证字段，例如 validation command、timeout 和 on failure cleanup。

DeepIntel 当前的 Harness 更偏研究 session supervisor，所以它把 checkpoint、budget、pending approval 和 state snapshot 作为主要状态；这和通用 harness 技能里的 validation command 语义不同。面试时要讲清楚：项目里的 Harness 是研究任务控制面，不是通用 CI runner。

13.4 Active marker 的意义

面试回答

active marker 是开关，不是锁。它的作用是告诉 hook 或外部 supervisor：“当前目录有 harness 任务需要接管”。没有 marker 时相关 hook 应该 no-op，避免普通项目操作被 harness 误拦截。

容易混淆的点

active marker 不能替代 lease，也不能替代 lock。marker 回答“有没有 harness 任务处于活跃状态”，lease 回答“这个任务现在由谁执行”，lock 回答“谁能修改共享状态文件”。这三个问题不同，不能用一个文件同时解决。面试时把这三个概念拆开，会比简单说“有个.harness-active 文件”可信得多。

13.5 Lease 为什么是长任务 Agent 的核心字段

面试回答

lease 解决的是所有权和超时恢复。没有 lease，新 session 看到一个 in progress 任务时无法判断它是仍在运行，还是上一个 worker 已经死掉。lease 过期后才有资格进入恢复逻辑，否则并发 worker 会互相踩状态。

租约设计的三个细节

第一，lease 必须有稳定 worker id，不能只靠进程号。进程号在容器、重启和不同机器上都不稳定，worker id 至少要能解释“哪个执行者认领了任务”。第二，lease 必须续约，而不是只在开始时写一次。长任务执行过程中每个 checkpoint 都应更新 heartbeat，否则一个正常运行但耗时很长的任务会被误判为 stale。第三，lease 过期只代表可以恢复，不代表可以无脑覆盖。恢复前仍要检查 checkpoint、DB runtime state、pending approval 和最近错误。

13.6 为什么恢复边界不能只靠 LangGraph checkpointer

面试回答

LangGraph checkpointer 保存的是图内状态快照；Harness 负责的是任务级控制决策。checkpointer 能告诉你图走到哪里，不能单独告诉你是否还有预算、审批是否通过、worker 租约是否过期、是否应该重试或终止。

职责边界

把恢复问题全部交给 checkpointer，会漏掉任务级语义。比如一个节点已经写入 checkpoint，但它等待审批；如果恢复时只从 checkpoint 继续跑，而不查本地审批聚合，就可能绕过人工确认。再比如浏览器节点失败三次，checkpoint 里可能记录了失败结果，但是否还有 retry budget、是否需要 rollback、是否应该进入 terminal failed，是 supervisor 的判断。

所以更稳的架构是：checkpointer 负责“状态在哪里”，Harness 负责“能不能继续、从哪里继续、由谁继续”，DB 负责“审批、预算、审计的事实源”。三者叠加才是完整恢复，不是某一个组件包打天下。

13.7 Rollback 为什么不能随便做

高风险点

通用 harness 技能里会提到失败后 `git reset --hard` 和 `git clean -fd`。这是对隔离工作树的任务 runner 才安全。DeepIntel 当前是在共享工作区里协作，不能把这种 rollback 当作默认行为，否则可能删除用户改动。

面试表达

正确表达是：rollback 需要满足两个条件。第一，任务必须运行在独立工作树或独立 clone 里，避免清理其他 worker 或用户改动。第二，任务开始时必须记录 base commit，失败后验证这个 commit 仍存在。没有这两个条件，自动 rollback 就不是安全恢复，而是破坏性操作。

这也是为什么当前项目里更适合把 Harness 定位成 session supervisor，而不是完全自动化代码任务 runner。它能记录 checkpoint、lease、预算和审批状态，但不会在共享工作区里随意 reset。

13.8 Dependency 与 blocked propagation

面试回答

依赖传播解决的是“下游任务是否还有意义”。如果一个 P0 任务永久失败，依赖它的任务应该被标记 blocked 或 failed with dependency reason，而不是一直 pending 让 worker 反复挑选。

工程意义

没有 blocked propagation 的任务队列会出现假进度：pending 数量看起来还有很多，但其中大部分永远不能执行。成熟的 supervisor 要能区分“还没轮到的 pending”和“因为上游失败永远不能跑的 blocked”。这个区别不仅影响调度，也影响面试里对系统可靠性的解释。

13.9 Stats 为什么是恢复协议的一部分

面试回答

stats 不只是 UI 汇总，它是每个 session 结束时对任务状态的一次结构化结算。它让下一次恢复能快速知道 completed、failed、pending、blocked、attempts、checkpoint 的整体分布。

为什么要写进 progress

stats 写进 progress file 的好处是可审计。即使 tasks JSON 后续被更新，你仍能看到每个 session 结束时的全局状态。这对于长任务尤其重要，因为问题常常不是某一个 task 错了，而是“系统什么时候开始进入大量 retryable failed、什么时候出现 blocked、什么时候 session count 打满”。

13.10 Harness 面试追问清单

追问	稳妥回答
为什么不把 Harness 放进 LangGraph 图里？	因为图表达业务执行，Harness 表达任务级控制。把 lease、审批、预算、恢复都塞进图，会让图变成混杂控制器。
progress 和 tasks 为什么要分两个文件？	一个是事件历史，一个是当前状态。历史要 append-only，当前状态要可覆盖、可恢复、可查询。
lease 过期后能直接接管吗？	不能直接覆盖。过期只代表有资格恢复，还要检查 checkpoint、DB runtime state、pending approval 和最近错误。

追问	稳妥回答
为什么 active marker 不是锁？	marker 只表达 harness 是否活跃，不表达谁持有状态写权限，也不表达任务所有权。
多 worker 最难在哪里？	难在共享状态锁、任务租约、独立工作树、rollback 隔离和幂等提交，不是多开几个进程。
checkpoint 和 snapshot 有什么区别？	checkpoint 是恢复点，snapshot 是当时的完整状态材料。checkpoint 更偏指针，snapshot 更偏事实内容。
为什么 premature completion 是大风险？	Agent 容易把“完成了一部分步骤”误报成“目标已完成”。结构化 task criteria 和 validation 是防线。
验证命令缺失怎么办？	不能宣布 completed。应该记录 CONFIG 错误并停止，避免没有客观校验的完成。
JSON 损坏怎么办？	先恢复.bak；没有有效备份就停止。不能从日志凭空重建 validation、depends on 和 cleanup。
为什么日志要 grep-friendly？	因为恢复和排障时需要脚本快速筛 ERROR、SESSION、CHECKPOINT、RECOVERY，而不是靠人读散文。

14 第十二轮拷打：MCP 协议与治理怎么讲深

一句话定位 MCP

MCP 是外部能力接入协议，不是安全治理系统。它让 client 和 server 用统一方式描述资源、提示和工具，但“该不该信任这个 server、该不该允许这个 tool、参数是否合法、返回值是否脱敏、写操作是否需要审批”仍然是应用自己的责任。

14.1 MCP 的三层结构

协议层、传输层、治理层

第一层是消息层，核心是 JSON-RPC 2.0。无论本地 stdio 还是远程 Streamable HTTP，本质都是 client 发 method、params、id，server 回 result 或 error。第二层是传输层，本地常见 stdio，远程推荐 Streamable HTTP。第三层是治理层，这一层不是 MCP 自动提供的，而是 DeepIntel 通过 McpPolicyProxy 补上的。面试时不要把这三层混在一起。说“我们用了 MCP，所以安全”是不严谨的；更准确的说法是“我们按 MCP 接入外部能力，但所有调用先经过 policy proxy 做 server trust、tool allowlist、schema、审批和 redaction”。

14.2 为什么 JSON-RPC 适合 MCP

面试回答

JSON-RPC 适合 MCP，是因为它轻量、跨语言、可读、调试成本低。MCP server 可能用 Python、TypeScript、Go 或其他语言写，JSON-RPC 给它们一个共同的调用结构。

不是因为先进，而是因为合适

协议设计的价值不总是越复杂越好。MCP 的核心诉求是让模型应用发现和调用工具，而不是构建一个重型微服务网关。JSON-RPC 的 method、params、result、error 已经足够表达 tools/list、tools/call、resources/read 这类交互。它不像 REST 那样把语义拆到 URL 和 HTTP verb，也不像 gRPC 那样需要 IDL 和二进制协议，对 agent 工具生态来说门槛更低。

14.3 stdio 与 Streamable HTTP 怎么比较

维度	stdio	Streamable HTTP	面试落点
部署位置	本地子进程	远程服务	本地工具优先 stdio，企业共享能力更适合 HTTP。
暴露面	无网络端口	有网络入口	HTTP 必须认真做 auth、trust 和 rate limit。
调试	看 stdin/stdout	看 HTTP 请求与 SSE 流	都是 JSON-RPC，消息层不变。
扩缩容	依赖本地进程	可放到服务侧扩展	远程 server 更适合团队共享。
安全风险	本地文件/进程权限	网络、身份、租户、跨域	不同传输有不同治理重点。

14.4 Tool inputSchema 为什么必须落到执行前

面试回答

inputSchema 是 Agent 工具调用从自然语言走向真实执行的第一道确定性边界。模型可以理解错参数，但 schema 不能跟着理解错；参数不符合 type、required、enum、additionalProperties，就不应该执行。

DeepIntel 的实现边界

当前项目实现的是轻量 JSON Schema 子集，覆盖 object、properties、required、additionalProperties、基础类型和 enum。这个选择是有意的：它足以挡住大多数错误参数和意外字段，又不会引入完整 schema 引擎的复杂度。面试时可以说“当前是工程上足够用的 schema 子集，不是完整 JSON Schema 解释器”。

14.5 为什么 tool annotations 不能当安全依据

关键风险

MCP server 可能给 tool 标注 read-only、destructive、open-world 等信息，但这些 annotation 只能作为提示，不能作为最终安全裁决。因为 server 本身就是外部信任边界的一部分，不能让它自己声明自己安全，然后应用就无条件相信。

正确做法

正确做法是应用侧维护 registry：server 是否 trusted、允许哪些 tools、哪些 tools 是 write tools、是否 read-only default、需要哪些 approval。server 提供的 schema 和 annotation 可以进入 registry 审核流程，但最终执行时要以本地 registry 为准。这和审批系统的 source of truth 原则是一致的：外部提供信号，本地做裁决。

14.6 MCP authorization 怎么回答

面试回答

远程 MCP 不能只靠“知道 endpoint”来调用，必须有 authorization。规范层面会涉及 OAuth 语义，但工程落地时还要做 server registry、token 管理、scope、租户隔离和审计。授权解决“你是谁”，policy proxy 解决“你能做什么、这次参数能不能执行”。

授权和治理的区别

Authorization 不是全部治理。一个用户有权访问某个 MCP server，不代表他可以调用这个 server 的所有工具；有权调用某个工具，也不代表任意参数都合法；参数合法，也不代表返回值可以原样进入模型上下文。面试里要把这条链路拆开：身份认证、工具授权、参数校验、审批、执行、返回脱敏、审计。

14.7 MCP 面试追问清单

追问	稳妥回答
MCP 是不是等于工具治理?	不是。MCP 是协议，治理要在应用侧做 policy proxy。
为什么不用直连 MCP tool?	直连会丢失 server trust、tool allowlist、schema、审批和 redaction。
MCP 底层消息是什么?	JSON-RPC 2.0, method、params、id、result/error。
本地 MCP 为什么常用 studio?	配置简单、延迟低、没有网络暴露面，适合本地工具。
远程为什么用 Streamable HTTP?	单端点承接请求和响应，需要流式时返回 SSE，比双端点 SSE 更简单。
inputSchema 怎么用?	执行前校验参数结构，不通过就返回 schema invalid。
写工具怎么处理?	默认需要 approval request，本地审批聚合通过后才执行。
返回值为什么要 redaction?	MCP server 返回可能含 token、内部 ID、secret，进入上下文前必须脱敏。
server fingerprint 有什么用?	审计时证明调用的是哪个 server 版本或实例，避免 tool list 漂移无法追责。
Planner 为什么还没大量生成 MCP 节点?	当前 MCP 是安全接入通路，主流量仍以 search/browser/rag 为主，MCP 主流量化是后续演进。

15 第十三轮拷打：Agent 开发岗位底层能力

岗位视角

Agent 开发岗位不是只会写 prompt。真正会被追问的是运行时、状态、工具、权限、检索、评测、成本、观测和失败恢复。下面按平台基础设施、应用/RAG 研究、安全治理三条线拆。

15.1 平台基础设施线

底层问题	应该掌握什么	DeepIntel 怎么讲
运行时状态	state machine、checkpoint、thread id、resume	ResearchState 是业务状态，Harness 是任务级状态，DB 是事实状态。

底层问题	应该掌握什么	DeepIntel 怎么讲
并发调度	DAG batch、fan-out、join、依赖	Planner 产 DAG, Lang-Graph 按可执行批次分发 Search/Browser/RAG。
长任务恢复	checkpoint、lease、heart-beat、retry budget	Harness 记录 lease、checkpoint 和 progress，避免上下文丢失后无法恢复。
工具执行	tool schema、timeout、sandbox、error category	工具调用先过确定性规则，再进入真实资源。
HITL	interrupt、approval、resume、idempotency	动作审批以 DB 为权威源，外部系统只是信号。
观测性	trace id、span、tool history、event stream	SSE 推进 UI, tool history 和 audit 解释每次调用。
成本控制	token budget、tool calls、wall clock、hard stop	session budget state 和 breaker 控制长任务膨胀。
错误语义	retryable vs terminal vs awaiting approval	runtime status 和 public status 分离，避免 UI 状态污染执行语义。
存储设计	session、checkpoint、audit、budget、skill meta/content	PostgreSQL 保持事实源，Redis/缓存只做加速。
扩展边界	plugin、skill、MCP、custom nodes	Skill 是领域能力层，MCP 是外部工具协议，二者不是同一层。

15.2 应用/RAG 研究线

底层问题	应该掌握什么	DeepIntel 怎么讲
Query decomposition	子问题、依赖、证据源选择	Planner 不是写答案，而是生成可执行研究 DAG。
Hybrid retrieval	BM25、vector、rerank、metadata filter	RAG 负责内部证据，不替代 Web 取证。
Citation grounding	evidence id、claim attribution、coverage	Report 必须用 citation 回溯证据，Reflection 检查覆盖率。
Browser extraction	accessibility、DOM、readability、fallback	Browser 节点不是简单抓 HTML，而是多策略提取。
Research quality	faithfulness、answer relevance、coverage	质量评估要看证据充分性，不只看文风。
Replanning	failure hints、known bad paths、budget	Reflection 不通过时进入 replan，但受预算和失败记忆约束。
Internal-first	内部资料优先、web 补充	DeepIntel 默认 internal-first，避免一上来就联网扩大成本和风险。

底层问题	应该掌握什么	DeepIntel 怎么讲
Report synthesis	structure、引用、冲突证据、边界	Analyst 综合, Report 输出, 边界和不确定性要写明。
Eval dataset	query set、expected evidence、metrics	metrics 目录已有实验基础, 但统一 dashboard 未完成。
成本/质量权衡	top-k、depth、browser budget、model choice	研究深度不是无限扩张, 要有 budget profile。

15.3 安全治理线

底层问题	应该掌握什么	DeepIntel 怎么讲
Prompt injection	instruction hierarchy、untrusted content、tool boundary	浏览网页和 MCP 返回都属于不可信内容, 不能让其改写系统策略。
Excessive agency	least privilege、approval、read-only default	高风险工具默认审批, MCP 非只读不直接执行。
Data leakage	redaction、tenant scope、row limit	MCP 和 tool result 进入上下文前要脱敏, Skill scope 不能越权。
Tool poisoning	server trust、tool allowlist、schema drift	MCP server 必须 registry 审核, fingerprint 进入审计。
Supply chain	dependency、MCP server provenance、skill content	Skill 和 MCP 都是扩展面, 需要版本和来源管理。
Auditability	who/what/why/result hash	工具调用必须记录调用者、参数范围、结果摘要、是否被拦。
Sandbox	filesystem、network、CPU/memory、timeout	真正有副作用的工具要在受限环境里跑。
Approval SOT	DB aggregate、external signal、idempotency	外部审批不能直接驱动执行, 本地 DB 是最终裁决。
Policy testing	unit tests、red-team cases、regression	安全规则要用确定性测试证明, 不靠 prompt 承诺。
Incident response	pause、revoke、rollback、forensics	当前有暂停/审批/审计基础, 但全量法证平台未完成。

15.4 Agent 岗高压题库：平台基础设施

1. 如果 LangGraph 已经有 checkpoint, 为什么还要 Harness? 回答时强调图内状态与任务级控制面不同。
2. 任务恢复时先看 DB、Harness 还是 checkpointer? 回答时讲三者职责: DB 是事实,

Harness 是控制，checkpointer 是图状态。

3. 多 worker 怎么避免抢同一个任务？回答 lease、state transaction lock、独立工作树。
4. 为什么不能在共享仓库里自动 `git reset --hard`？回答 rollback 隔离和用户改动保护。
5. retryable failed 和 terminal failed 怎么区分？回答错误类别、attempt budget、依赖传播。
6. 为什么 public status 和 runtime status 要分开？回答 UI 友好状态不能污染执行语义。
7. budget breaker 是软提示还是硬控制？回答必须是硬熔断加优雅降级。
8. 工具超时算模型失败还是工具失败？回答是工具执行失败，进入 tool error category。
9. Trace 和 audit 有什么区别？Trace 解释流程，audit 证明动作事实。
10. Session snapshot 为什么要版本化？避免恢复时状态结构不兼容。
11. 如果 checkpoint 存在但 approval 已过期怎么办？不能继续执行，必须重新进入审批或终止。
12. 如何防止 Agent 伪造 tool result？Observation 必须来自工具层真实返回。
13. 长任务怎么判断完成？不能靠模型说完成，要靠任务 criteria 和验证。
14. SSE 断了任务是否停止？不应该，SSE 是交互通道，不是任务事实源。
15. 如果 DB 写入成功但 Harness 写入失败怎么办？要记录错误并在恢复时以 DB 事实为准补偿。
16. 为什么任务状态不能只放 Redis？Redis 适合缓存，不适合唯一事实源。
17. 什么是 idempotency key？用于避免审批回调、工具执行或状态更新重复生效。
18. 如何设计 stop/resume？stop 改 runtime state，resume 读取 checkpoint 和 harness snapshot。
19. 如何做 model fallback？fallback 不能破坏结构化输出 schema 和工具调用契约。
20. Agent 失败后如何生成部分报告？基于已有证据输出，并标注覆盖不足和未执行节点。
21. 为什么 orchestration 不是 prompt engineering？因为它处理状态、依赖、恢复和副作用。
22. 如何把任务拆成 DAG？节点是可执行取证动作，边表达依赖，不是自然语言段落。
23. 什么情况下 replan？Reflection 发现证据不足、失败路径重复、原 DAG 不可达。

24. 如何限制 replan 膨胀? max revision、budget、known failures 和 node cap。
25. 如果 worker 死在工具执行中怎么办? lease 过期后恢复, 检查副作用审计和 check-point。

15.5 Agent 岗高压题库: 应用与 RAG

1. Deep Research 和普通 RAG 的核心差异是什么? 回答任务状态机 vs 上下文增强回答。
2. 为什么 internal-first? 回答成本、安全和内部证据优先级。
3. BM25 和向量检索怎么互补? BM25 抓关键词, 向量抓语义。
4. 为什么还要 rerank? 召回集合和最终上下文选择不是同一个问题。
5. citation 怎么保证可信? 证据 id、引用覆盖、claim 对齐和 reflection。
6. 网页内容怎么防 prompt injection? 网页是证据, 不是指令。
7. Browser 提取失败如何降级? accessibility、readability、raw text、snippet fallback。
8. 证据冲突怎么办? 不强行合并, 报告冲突来源和可信度。
9. RAG 命中后还要不要 Web? 看任务类型、证据时效性和用户允许策略。
10. 如何评价研究报告? faithfulness、coverage、answer relevance、citation precision。
11. 为什么不能只看 BLEU/ROUGE? 研究任务更关注事实和证据, 不是字面重合。
12. 如何构建 eval dataset? 按任务类型、证据源、难度、时间敏感性分层。
13. 如何处理时间敏感问题? 必须重新取证, 不能靠模型记忆。
14. 如何减少 hallucination? 证据优先、拒答、引用、reflection、工具真实 observation。
15. Planner 输出 JSON 失败怎么办? schema 校验、repair、fallback DAG。
16. fallback DAG 会不会太死? 它是最低可执行骨架, 不是替代高级规划。
17. 什么是 evidence coverage? 关键 claim 是否都有证据支持。
18. 如何避免检索上下文过长? top-k、rerank、摘要、chunk budget。
19. 报告应该输出多长? 由 budget profile 和任务复杂度控制。
20. 为什么 Analyst 和 Report 分开? 分析是证据综合, 报告是表达和引用组织。
21. Reflection 是否会增加成本? 会, 所以要受 revision budget 控制。
22. 搜索结果不可靠怎么办? 多源交叉、来源评级、Browser 深读。

23. 内部资料过期怎么办？标注时间，必要时 Web 补证。
24. 如何处理用户给的错误前提？先校正前提，再回答条件化结论。
25. 什么时候不应该联网？用户禁止、内部证据足够、风险或预算不允许。

15.6 Agent 岗高压题库：安全治理

1. Prompt injection 的本质是什么？不可信内容试图越权改变上层指令。
2. 为什么 tool schema 比 prompt 安全？schema 是确定性约束，prompt 是概率性引导。
3. MCP tool 为什么默认不可信？server 是外部边界，tool list 可能漂移。
4. read-only default 有什么意义？把副作用作为例外，而不是默认。
5. 写操作审批如何防重复执行？approval id、action id、DB version、幂等键。
6. 为什么外部审批只能是信号？最终执行发生在本地系统，本地 DB 必须裁决。
7. 数据脱敏在哪做？进入模型上下文和审计前都要做。
8. 如果 redaction 漏了怎么办？补充 policy key、回归测试、审计影响范围。
9. 如何防止 skill 越权？scope 过滤、manual enable 不绕过、session snapshot。
10. Skill 内容是不是也可能被投毒？是，所以要版本、来源、启用状态和审核。
11. Agent 能不能直接写 SQL？不应该，应该走参数化模板和受控 DB 接口。
12. 错误信息为什么要净化？原始错误可能泄露 schema、路径、secret 或内部拓扑。
13. 审计日志要记录什么？调用者、tool、参数摘要、结果 hash、策略裁决、时间。
14. 如何处理高风险工具？默认隐藏或只读，必要时审批和沙箱。
15. 什么是 excessive agency？Agent 被授予超出任务需要的权限和自主性。
16. 如何限制 excessive agency？least privilege、budget、approval、tool allowlist。
17. MCP authorization 和 allowlist 区别？auth 证明身份，allowlist 限制能力。
18. 如何做红队测试？注入、越权、敏感字段、schema 绕过、审批重放。
19. 安全策略失败时返回什么？稳定 safe error，不暴露内部细节。
20. 为什么不能相信模型自我审查？模型审查是辅助，执行面必须确定性控制。
21. Tool result 能不能带指令？可以出现文本，但应作为数据处理，不能提升为指令。

22. 多租户下最大风险是什么？数据越界和 skill/tool scope 漂移。
23. 如何处理删除或更新的 skill？新 session 不再匹配，旧 session 继续使用版本 snapshot。
24. 审批过期怎么办？执行前检查本地状态和 expiry，过期则拒绝或重新申请。
25. 如何解释“安全不是阻碍能力”？治理让能力在可控边界内扩大，否则工具越多风险越大。

16 第十四轮拷打：Skill 体系深挖

一句话定位 Skill

Skill 是领域能力的系统级扩展点，不是多拼一段 prompt。它要能被发现、匹配、启用、禁用、版本化、审计，并能影响 Planner、Agent hints 和工具 allowlist。

16.1 为什么 Markdown + YAML frontmatter

面试回答

frontmatter 承载机器可校验字段，Markdown body 承载人类可编辑说明。这比纯 prompt 可治理，比纯 JSON/YAML 更适合人维护。

字段拆解

frontmatter 至少要包含 name、slug、description、version、enabled、priority、trigger patterns、allowed tools、agent hints、tags、keywords、scope。body 里可以有 overview、prompt、constraints。这样 registry 能用 metadata 做匹配和治理，Agent 能用 body 做领域行为调整。

16.2 Progressive disclosure 在 Skill 里的含义

面试回答

Progressive disclosure 不是 UI 折叠，而是计算和上下文逐步展开：启动只加载 meta，匹配时先粗筛候选，命中后再加载 content，最终只把 top-k skill 的摘要和约束进入 session snapshot。

为什么这很重要

如果启动时加载所有 skill 正文，冷启动慢；如果匹配时对所有正文做复杂规则，延迟高；如果最终把所有命中 skill 都塞进 prompt，模型上下文失控。所以成熟设计必须把 meta、content、matching、resolved snapshot 分层。这也是 DeepIntel 当前 registry 的核心价值。

16.3 Scope 为什么不能被 manual enable 绕过

风险

如果用户手动启用可以绕过 tenant/project scope，那么 skill 就会成为权限绕过入口。比如另一个租户的财务分析 skill 可能携带工具 allowlist 或内部提示，不能因为手动启用就进入当前 session。

当前状态

当前测试已经覆盖 manual enable 不绕过 scope。这一点面试时可以讲成已落地能力：手动操作是 preference，不是 authorization。

16.4 Snapshot 为什么是 Skill 运行时的关键

深入分析

Skill 更新、禁用、删除是常见操作。如果运行中的 session 每一步都回查全局 registry，那么同一个任务可能前半段用 v1 prompt，后半段突然用 v2 prompt，行为漂移且难以解释。resolved skill snapshot 的价值是把 skill id、version、matched reason、prompt sections、agent hints、allowed tools 固化到 session。这样新 session 可以使用新版本，旧 session 继续使用旧版本。面试时要强调这不是缓存小优化，而是运行时一致性设计。

16.5 Skill 和 MCP 的关系

回答

Skill 和 MCP 不在同一层。Skill 描述“这个任务应该采用什么领域策略、提示、工具边界”；MCP 描述“外部 server 暴露了哪些资源和工具”。Skill 可以影响 MCP tool 是否进入 allowlist，但不能替代 MCP 的 server trust 和 schema 校验。

16.6 Skill 面试追问清单

追问	稳妥回答
Skill 和 prompt 有什么区别？	Prompt 是文本，Skill 是可注册、可匹配、可审计、可约束工具的系统对象。
为什么需要 version？	防止更新导致运行中 session 行为漂移。
为什么要 top-k？	防止 prompt 膨胀和工具边界过宽。
manual enable 是不是最高优先级？	不是。manual enable 不能绕过 scope 和权限。
allowed tools 怎么合并？	对有效 skill 做有序去重，形成 session tool allowlist。
trigger pattern 失败怎么办？	还有 keywords、tags、coarse terms 和 manual enable；但不应全量正文扫描。
Skill 删除后旧 session 怎么办？	新 session 不再匹配，旧 session 使用 snapshot，等待版本回收。
Skill 能不能定义 DAG 节点？	当前未完成；这是后续从提示层升级到执行模板层的方向。
Skill 内容会不会被投毒？	会，所以要来源、版本、启用状态、审核和 scope。
为什么 meta/-content 分离？	meta 快速匹配，content 延迟加载，降低冷启动和上下文成本。

17 第十五轮拷打：优化路线图怎么讲不虚

路线图原则

路线图不是许愿池。每一项都要说明为什么现在没做、做了要解决什么问题、会引入什么成本、怎么验收。

路线项	解决的问题	代价	验收标准
多 worker Harness	长任务并行吞吐和中断恢复	lock、lease、独立工作树、冲突恢复	两个 worker 不会抢同一任务，失败 rollback 不影响彼此。
MCP Planner 主流量化	让外部能力进入业务 DAG	server registry、tool schema、planner policy	Planner 能生成受 allowlist 控制的 MCP 节点。
Skill DAG 模板	让领域能力影响执行结构	模板校验、节点权限、版本兼容	Skill 可声明节点模板且通过 schema 验证。

路线项	解决的问题	代价	验收标准
统一 dashboard	eval 比较质量、成本、恢复能力	指标标准化、数据采样、可视化	每次实验输出 quality/cost/recovery 三类指标。
法证级审计	事故复盘和合规证明	存储成本、PII 脱敏、retention policy	能按 session/action 重建工具调用链。
外部审批集成	贴合企业流程	回调幂等、状态漂移、权限同步	Slack/Jira 只做信号，本地 DB 裁决不变。
Prompt injection 红队	验证治理强度	样本构造和持续回归	注入样本不能越权调用工具或改写策略。
模型路由优化	降低成本并保持结构稳定	模型能力差异和 fallback 策略	结构化输出失败率、成本、延迟可量化比较。

最终收口话术

我不会把这个项目包装成已经具备完整生产级 Agent 平台。当前已经落地的是研究 DAG、工具审计、预算状态、审批入口、Harness 最小控制面、MCP policy proxy 和 Skill registry。下一步真正要补的是多 worker 调度、MCP 主流量化、Skill 执行模板和统一评测。这个边界说清楚，反而能证明我知道 Agent 系统的复杂度在哪里。

18 第十五轮补充：行业调研来源与选型记录

为什么要单独记录来源

这一节专门回答“这些优化从哪里来、为什么不是拍脑袋”。面试时如果只说“我做了 Harness、MCP、Skill”，会显得像功能堆砌；如果能把外部最佳实践、项目约束、备选方案和最终落地串起来，就能证明你做的是系统设计。

18.1 外部资料对当前设计的校准

资料	关键结论	对 DeepIntel 的影响
Anthropic long-running harness	Effective harnesses for long-running agents 强调跨 context window 的核心问题不是模型不够聪明，而是新 session 没有可靠上下文；它用 progress file、feature list、git history、incremental progress 和测试来降低 premature completion。	DeepIntel 没有照搬 coding harness 的 git rollback，而是吸收“状态外置、进度可恢复、每次都要验证”的原则，落地为 <code>harness-tasks.json v2</code> 、progress log、backup、lease、summary 和测试证据。
Anthropic agent patterns	Building effective agents 把 workflow 和 agent 区分开，建议先用简单可组合模式，复杂时再引入 routing、parallelization、orchestrator-workers 和 evaluator-optimizer。	DeepIntel 选择显式 DAG + Lang-Graph，而不是纯多 agent 自由对话。这样能讲清楚依赖、预算、审批和失败传播，也符合研究任务需要可审计流程的约束。
MCP 2025-11-25 transport	MCP Transports 明确 MCP 消息由 JSON-RPC 编码，标准传输是 stdio 和 Streamable HTTP；Streamable HTTP 涉及 Origin 校验、认证、session id、resumability 和协议版本头。	DeepIntel 的 MCP 设计不能只写“连接一个 server”。面试回答必须拆成协议层、传输层和治理层；当前代码重点落在治理层，即 server trust、tool allowlist、schema、approval 和 redaction。
MCP authorization	MCP Authorization 说明 HTTP transport 的授权基于 OAuth 语义，但授权是 optional，stdio 场景更多从环境取凭据。	这说明“用了 MCP”不等于“天然安全”。DeepIntel 要在 registry 和 policy proxy 里补应用侧授权、租户 scope、审批和审计，而不是把外部 server 的声明当最终裁决。
MCP tools	MCP Tools 说明 tool 是 model-controlled，包含 inputSchema 和可选 outputSchema；同时提示 tool annotations 不能在不可信 server 上盲信，客户端应按 outputSchema 校验 structuredContent。	这直接对应本轮实现：MCP proxy 加入轻量 input/output schema 校验、write tool approval、result hash、server fingerprint 和返回值 redaction。

资料	关键结论	对 DeepIntel 的影响
OpenAI Agents SDK	Agent orchestration 区分 LLM orchestration 与 code orchestration, 并给出 agents-as-tools 与 hand-offs 的取舍; Guardrails 把 input/output/tool guardrails 分层; Human-in-the-loop 支持敏感工具调用暂停、审批、恢复; Results 暴露 pending approvals 和 resumable RunState; Tracing 覆盖 LLM generations、tool calls、handoffs 和 guardrails。	这些资料支撑 DeepIntel 的拆分: Agent 运行时要有 orchestration、guardrail、approval、state、trace 五条线, 而不是把全部逻辑塞进一个 prompt 或一个节点。
Skill capability 设计	OpenAI Agents SDK Skill capability 把 skill 建模为带 name、description、content、scripts、references、assets 和 deferred 标记的能力对象; 本地 skill-creator/SKILL.md 进一步强调 metadata 先触发、正文按需加载、资源渐进披露、正文保持精简。	这支持 DeepIntel 的 meta/content 分离、top-k、version snapshot、scope 检查和工具 allowlist。Skill 不是提示附件, 而是会影响运行时边界的能力对象。

18.2 Harness 选型记录

方案	优点	风险/成本	决策
只写 progress log	最简单, append-only, 容易 grep。	只能解释历史, 不能稳定驱动恢复; 无法表达 lease、pending、terminal、retryable、approval。	否。作为辅助日志保留, 但不能做控制面。

方案	优点	风险/成本	决策
只依赖 Lang-Graph checkpoint	和业务图贴合，能恢复图内状态。	checkpoint 解决“图走到哪里”，不解决“任务能否继续、谁持有 lease、预算是否熔断、审批是否等待”。	否。继续保留图内 checkpoint，但外面加 Harness supervisor。
文件型 Harness v2	低成本、易测试、适合当前研究 session；能做 backup、atomic replace、summary、lease 和恢复。	不是完整分布式调度器；跨进程 lock 和多 worker 隔离还没做满。	选。当前已落地，测试覆盖 JSON 恢复、marker、lease、summary。
DB/队列型调度平台	并发、事务、审计和租约都更强。	改造范围大，需 job queue、worker pool、worktree 隔离、幂等 action 和回滚策略。	暂不选。作为多 worker 路线图。

最终选择

当前选择“文件型 Harness v2 + 业务 DB 事实源 + LangGraph checkpoint”的组合。理由是它用较小改动解决跨上下文恢复和 premature completion 的核心风险，同时不把共享工作区暴露给自动 `git reset --hard`。未完成边界必须说清楚：它不是成熟分布式 scheduler。

18.3 MCP 选型记录

方案	优点	风险/成本	决策
Planner 直连 MCP tool	快速接入，开发量低。	丢失 server trust、tenant scope、tool allowlist、schema、审批、redaction 和审计。	否。高风险，面试不能讲成生产治理。
MCP policy proxy	把协议调用和治理分离；可对每次调用做 allowlist、schema、approval、redaction、fingerprint。	需要维护 registry、policy JSON、审计记录和工具 schema。	选。当前已落地最小治理闭环。

方案	优点	风险/成本	决策
全量 API gateway + OAuth broker	企业级能力最完整，可统一 token、scope、rate limit、audit。	建设成本高，当前 MCP 主流量还不够，不适合先做重网关。	暂不选。后续远程 MCP 主流量化后再推进。

最终选择

当前选择 MCP policy proxy，因为它能把“协议可调用”和“应用允许执行”分开。MCP 只规定 server 暴露工具、资源和传输，不替应用做所有信任裁决。DeepIntel 的核心优化点就是把这些裁决显式化。

18.4 Agent 架构选型记录

方案	优点	风险/成本	决策
单体 autonomous loop	灵活，prompt 少，迭代快。	状态、依赖、预算、审批和失败传播不清楚；复盘困难。	否。只适合探索，不适合当前研究型可审计任务。
显式 DAG + StateGraph	依赖、批次、replan、approval、budget 和 citation 都可检查。	需要维护状态 schema、节点类型、聚合器和测试。	选。符合当前 DeepIntel 主链路。
多 agent 自由对话	表达力强，可模拟专家协作。	容易重复、漂移、难以判定事实所有权；成本和延迟高。	否。可作为 brainstorming，不作为默认执行 runtime。
Manager + specialists as tools	Manager 保持最终控制，专家作为有边界能力。	需要明确 specialist contract 和 trace correlation。	设计已定。未来用于 RAG/browser/MCP 专家化，但不替代 DAG。

最终选择

当前选择“显式 DAG + StateGraph + Harness + policy guardrail”。这不是保守，而是因为研究型 Agent 最重要的是可解释调度和可恢复执行。多 agent 可以作为局部优化，但必须先有合同、预算、trace 和失败语义。

18.5 Skill 选型记录

方案	优点	风险/成本	决策
把 Skill 当 prompt snippet	实现最快。	无法版本化、审计、scope、top-k、snapshot，也无法解释工具边界变化。	否。面试时不要这样描述当前系统。
版本化 Skill registry	可做 meta/content 分离、匹配、manual enable、scope、tool allowlist 和 snapshot。	需要解析、缓存、排序、session 固化和测试。	选。当前已落地 top-k 与 resolved snapshot 测试。
Skill 自定义 DAG 节点模板	能让领域能力直接影响执行结构。	需要模板 schema、节点权限、版本兼容和安全审查。	未完成。作为下一阶段路线图,不宣称已具备。

最终选择

当前选择版本化 Skill registry，而不是简单 prompt 拼接。理由是 Skill 会影响 planning、tool usage 和 command execution，本质上已经进入安全边界。已落地部分可以讲：scope 不被 manual enable 绕过、top-k 控制 prompt budget、session snapshot 防止运行漂移。

18.6 本轮实现 TODO 与验收证据

TODO	状态	验收证据	面试边界
Harness v2 state/migration	已落地	tests/governance/test_harness.py	不是完整 job queue。
Harness atomic write + backup recovery	已落地	JSON corruption recovery 测试	backup 同坏时会停止。
Harness active marker + lease + summary	已落地	marker、expired lease、summary tests	没有多 worker lock。
MCP secret policy parsing	已落地	test_mcp_policy.py	policy schema 还轻量。
MCP tool schema validation	已落地	schema invalid tests	不是完整 JSON Schema 引擎。
MCP write tool approval	已落地	approval request tests	审批最终还要接 DB 聚合根。
MCP recursive redaction + hash	已落地	configured redaction keys tests	需要业务持续补 redact keys。

TODO	状态	验收证据	面试边界
Skill top-k 与 tool allowlist 合并	已落地	tests/skills/test_skill_top_k.py	top-k 仍是规则排序。
Skill resolved snapshot	已落地	registry update 后旧 snapshot 保持 v1 测试	版本回收 UI 未完成。
Agent 岗题卡扩展	已落地	agent-position-cardbook-test, 106 张题卡	题卡是面试准备, 不是功能代码。
面试笔记扩展到 200 页左右	已落地	PDF 当前约 200 页	页数不是质量指标, 内容边界才是。

不能夸大的地方

本轮没有把 DeepIntel 改成完整企业级 Agent 平台。准确说法是：Harness、MCP 和 Skill 的治理面已经明显增强，并且有测试覆盖；多 worker 调度、MCP 主流量化、Skill DAG 模板、统一 eval dashboard、法证级审计仍是路线图。

19 第十七轮拷打：继续优化还能补哪些细节

本轮定位

这一轮不是继续堆功能，而是补“面试官会追问但实现容易被忽略”的治理细节：状态文件并发写、依赖失败传播、MCP 参数级审批幂等、嵌套 schema、Skill scope 元数据一致性。它们都不大，但都直接决定 Agent 系统是不是能在真实运行中解释清楚。

19.1 本轮发现的可优化点

可优化点	风险	本轮处理	状态
Harness 状态并发写	两个执行器同时写 harness-tasks.json, 可能覆盖 checkpoint 或 backup。	增加 .harness-state.lock 轻量 transaction lock, 并支持 stale lock 清理。	已落地
Harness 依赖校验	循环依赖或依赖失败后, 任务可能一直 pending, 表面看没有错误。	增加 validate_dependencies(), 标记 circular dependency 与 blocked by failed。	已落地

可优化点		风险	本轮处理	状态
MCP schema	嵌套	顶层 schema 通过但嵌套 object、array item 出错，真实工具仍可能收到坏参数。	增加递归 object、array items、enum 校验。	已落地
MCP schema	输出	只校验输入参数，不校验工具返回结构，坏的 structuredContent 可能进入后续上下文和审计。	支持 outputSchema/tool_output_schemas，对 structuredContent 做轻量递归校验。	已落地
MCP 审批幂等		如果 approval id 只绑定 tool，不同参数可能复用同一审批；如果只绑定脱敏参数，不同 secret 可能合并。	approval id 绑定 raw args hash；审批 payload 只展示 redacted args 和 args hash。	已落地
MCP 审计 hash 稳定性		字典字段顺序不同导致 result hash 不同，审计比对噪音大。	result hash 改为 canonical JSON hash。	已落地
Skill scope 元数据		scope=global 却带 tenant/project，或 tenant/project scope 字段缺失，可能形成灰色越权入口。	在 SkillMetadata 加 model validator 强制字段绑定。	已落地
统一 eval dashboard		现有 metrics 分散，难以一眼比较质量、成本、恢复能力。	本轮只记录路线图，未扩展 dashboard。	未完成
Skill DAG 模板		Skill 还不能直接声明执行节点模板。	本轮不做，避免把提示能力和执行能力混在一起。	未完成

19.2 选型对比与决策

决策点	方案 A	方案 B	本轮选择
Harness lock	文件目录锁，低成本、可测试、适合单机状态文件。	引入 DB/Redis 分布式锁，能力更强但改造范围大。	选文件目录锁；它解决当前 state transaction 风险，但不宣称多 worker 平台已完成。

决策点	方案 A	方案 B	本轮选择
依赖失败处理	只在 summary 里统计 blocked。	主动把循环依赖和被失败依赖阻塞的任务标成 terminal failed。	选主动标记;否则任务会伪装成 pending, 恢复时没有明确失败语义。
MCP approval hash	绑定 redacted args, 避免敏感值进入 hash 输入。	绑定 raw args, 只在 payload 展示 redacted args。	选 raw args hash; 审批幂等必须区分真实写入意图, 否则不同 secret 值会复用审批。
MCP schema	只做输入顶层校验, 简单但漏嵌套和坏输出。	轻量递归 JSON Schema 子集, 同时覆盖 inputSchema 和 outputSchema。	选轻量递归; 覆盖 object、array、enum、required 和 structured-Content 校验, 仍不冒充完整 JSON Schema 引擎。
Skill scope	只在 registry resolve 时过滤。	metadata 创建时就校验 scope 字段绑定。	选双层防护; 创建时约束减少脏数据, resolve 时过滤防止越权。

19.3 本轮落地证据

模块	代码位置	验收测试
Harness 状态锁	app/governance/harness.py: _state_lock(), _clear_stale_lock()	test_harness_state_lock_releases_after_state_lock_releases(), test_harness_state_lock_removes_stale_lock_after_clear_stale_lock()
Harness 依赖校验	validate_dependencies()	test_harness_validate_dependencies_marks_dependency_cycle_path_as_failed(), test_harness_validate_dependencies_is_idempotent(), test_harness_validate_dependencies_marks_dependency_failed()
MCP 递归 schema	app/governance/mcp.py: _validate_object(), _validate_value()	test_mcp_policy_validates_nested_schema_against_json_schema(), test_mcp_policy_validates_output_schema_against_tool_output_schema()
MCP 审批幂等	_stable_hash(), _approval_id(), raw args hash	test_mcp_policy_approval_id_includes_args_hash(), test_mcp_policy_approval_id_uses_raw_args_hash()
MCP 审计 hash	canonical JSON result hash	test_mcp_result_hash_is_stable_for_equivalent_results()

模块	代码位置	验收测试
Skill scope 元数据	app/skills/models.py: validate_scope_binding()	test_skill_scope_metadata_requires_consistent_data

本轮仍不能夸大的边界

第一，`.harness-state.lock` 是状态事务锁，不是完整分布式 worker coordination。第二，MCP schema 是工程上够用的轻量子集，不支持 `oneOf`、`pattern`、复杂引用解析；输出校验也只覆盖结构化结果，不等于完整语义校验。第三，Skill scope validator 防的是元数据不一致，不等于完整 RBAC。第四，approval id 绑定参数指纹能减少重复审批风险，但真正执行写操作仍需要 DB 聚合根、action id 和幂等键共同保证。

19.4 下一阶段可扩展功能清单

- 1. **统一 eval dashboard:** 把 RAG、Browser、MCP、Harness 恢复、预算命中、approval wait time 合成一个 session-level scoreboard。
- 2. **MCP registry UI:** 展示 server trust、fingerprint、allowed tools、schema、write tools、redaction keys 和最近调用失败。
- 3. **Skill provenance:** 给 skill 增加 author、source、review status、content hash、signed version，防止 skill 内容投毒。
- 4. **Approval action id:** 把审批从 tool-level 推进到 action-level，审批记录绑定 action id、args hash、node id、DB version。
- 5. **Harness recovery planner:** lease 过期后不是直接接管，而是输出恢复建议：resume、retry、replan、await approval、terminal failed。
- 6. **Skill DAG 模板:** 只有在模板 schema、权限和版本兼容做完后，才让 skill 影响 DAG 节点结构。

面试收口

这轮优化最重要的不是“又加了几个函数”，而是把 Agent 系统里容易被忽视的确定性边界补上：谁能写状态、任务为什么阻塞、工具输入和输出是否真合法、审批是否绑定真实动作、skill scope 是否一致。Agent 工程不是让模型更自由，而是在模型自由行动前后都留下可验证的控制面。

20 第十八轮拷打：高级工程师故障注入题

这一轮定位

这一节专门替换掉低价值概念题。高级 Agent 岗面试不会满足于“为什么要系统设计”，而会直接问：这个系统在哪些事故场景下会错、如何检测、如何恢复、如何证明没有复发。回答时必须落到状态不变量、代码位置、测试证据和未完成边界。

20.1 事故 1：审批回调重复到达，写工具会不会执行两次

工程回答

不能只靠 approval id 防重复。正确不变量是：同一个 `action_id + args_hash + db_version` 只能从 pending 转成 approved/executed 一次；工具执行要有幂等键，审批回调只是状态迁移信号，不是直接执行命令。

DeepIntel 落点

本轮已经把 MCP approval id 绑定 raw args hash，避免不同写入意图复用同一审批；但完整闭环还需要 DB 聚合根记录 action id、审批版本和执行结果。面试时要诚实说：当前做到的是“审批请求幂等识别”，不是“写操作全链路 exactly-once”。下一步应在审批表上加唯一约束，在执行表上加幂等键，并把 MCP executor 结果写成可重放审计事件。

20.2 事故 2：MCP server 返回的 structuredContent 结构错了怎么办

工程回答

输入 schema 只能证明请求参数形状合法，不能证明工具返回可被下游消费。生产系统必须校验 outputSchema，尤其是 MCP 的 structuredContent；校验失败时应进入 terminal error 或 safe error，不能把坏结构送给 Analyst、Report 或审计 hash。

DeepIntel 落点

本轮已在 McpPolicyProxy 中支持 outputSchema/tool_output_schemas，并优先校验 structuredContent。验收测试是 test_mcp_policy_validates_output_schema_structured_content。边界是它仍是轻量 JSON Schema 子集，只覆盖 object、array、enum、required、

`additionalProperties` 等基础规则，不做语义级事实校验。

20.3 事故 3：两个 worker 同时写 Harness 状态，checkpoint 丢了怎么办

工程回答

长任务状态文件必须有状态事务锁、写前备份、临时文件和原子替换。否则两个 worker 同时写 `harness-tasks.json` 时，后写者可能覆盖前写者的 checkpoint、lease 或 last error，恢复时系统会读到一个看似合法但语义过期的状态。

DeepIntel 落点

本轮加入 `.harness-state.lock`、stale lock 清理和状态写入保护，测试覆盖 lock release 与 stale cleanup。需要继续强调边界：这是单机状态事务锁，不是完整分布式 scheduler；多 worker 真正上线还要独立 worktree、lease renewal、claim compare-and-set、rollback 隔离和 DB/队列级调度。

20.4 事故 4：Skill 手动启用绕过租户 scope 怎么办

工程回答

manual enable 是偏好信号，不是授权信号。Skill 会影响 prompt、tool allowlist、agent hints 和执行边界，所以 scope 必须在 metadata 创建时校验，在 session resolve 时再次过滤。只在 UI 层隐藏不可用 skill 不够，因为 API 和缓存仍可能被绕过。

DeepIntel 落点

当前已有 `SkillMetadata.validate_scope_binding()`，强制 global/tenant/project 字段绑定；测试还覆盖 manual enable 不能绕过 scope filters。下一步可以补 provenance、review status、content hash 和 signed version，防止 skill 内容被投毒或跨环境漂移。

20.5 事故 5：RAG 答案错了，如何定位不是一句“模型幻觉”

工程回答

先看正确证据有没有进入候选集，再看是否排到足够靠前，最后看生成是否按证据回答。也就是按 retrieval、ranking、generation 三层定位，而不是直接换模型。生产事故复盘要同时给出失败 query、gold doc、Hit@K、MRR、Context Recall、Faithfulness 和工具轨迹。

DeepIntel 落点

项目里已有 retrieval/generation 指标记录、tool history 和 node outcome，本轮文档已把 Agent Eval 改成六层定位：任务完成、证据召回、工具执行、答案可信、成本受控、恢复能力。后续应补统一 eval dashboard，把 RAG、Browser、MCP、Harness 和 budget breaker 聚合成 session-level scoreboard。

20.6 事故 6：预算硬熔断发生在证据不足时，如何避免粗暴失败

工程回答

预算 breaker 不是直接 kill，而是触发策略收缩：停止可选 Web 扩展、禁止新 replan、保留当前证据、标记 coverage gap，并输出部分报告或请求用户确认是否追加预算。关键不变量是成本不能继续无限增长，同时不能伪装成完整结论。

DeepIntel 落点

当前图里已有 budget breaker 和 should-continue 路由测试；面试时要把它讲成“控制规则参与路由”，不是“打个 warning”。未完成边界是还没有统一的预算策略配置 UI，也没有按任务类型动态调预算曲线。

20.7 事故 7：Prompt injection 藏在网页正文里，为什么 schema 仍不够

工程回答

schema 只能约束工具参数形状，不能判断网页内容是不是试图覆盖系统指令。对 Browser/RAG 返回内容，要把不可信文本作为 evidence 处理，而不是作为 instruction 处理；还要做引用边界、命令隔离、敏感工具审批和输出审查。

DeepIntel 落点

DeepIntel 当前通过 tool audit、approval、MCP policy proxy 和 Reflection 降低风险，但还需要更系统的 prompt-injection eval set、source trust score 和 per-source quarantine。高级回答的重点是承认 schema 不是万能护栏，治理要覆盖输入、工具、输出和状态恢复四层。

21 第十六轮拷打：Agent 开发岗单页题卡

题卡使用方式

这一章按单页题卡组织。每张题卡都对应 Agent 开发岗位可能出现的一类底层追问：先给面试官可听懂的回答，再讲底层机制、DeepIntel 里的落点、后续优化和不能夸大的边界。分页不是为了凑版面，而是为了复习时能一题一页地训练。

21.1 题卡 1: Agent 和普通 LLM 应用的分水岭是什么

面试回答

分水岭不是有没有调用模型，而是有没有显式状态、工具执行、环境反馈、失败语义和恢复机制。普通 LLM 应用更像一次生成，Agent 系统更像一个能在约束下持续推进任务的运行时。

底层拆解

Agent 的核心对象不是 prompt，而是 task state。它要知道当前目标、可用工具、已观察到的证据、下一步动作、失败记录、预算和停止条件。只有这些状态被结构化保存，系统才可能从“会说”变成“会执行”。

DeepIntel 落点

DeepIntel 用 Planner 生成研究 DAG，用 LangGraph 执行业务状态机，用 Harness 保存任务级 checkpoint、lease 和 progress，用 DB 保存审批、预算和审计事实。

优化与边界

不能说“用了模型就是 Agent”。当前项目已有研究型 Agent 主链路，但完整生产级 Agent 平台还需要多 worker 调度、统一 eval dashboard 和更强审计。

21.2 题卡 2：为什么 Agent 系统的核心是状态机

面试回答

因为 Agent 任务不是一次线性调用，而是会在规划、取证、分析、反思、重规划、审批、恢复之间切换。状态机能显式表达当前阶段、可走路径和停止条件。

底层拆解

如果没有状态机，步骤会散落在 if-else、prompt 和局部变量里。排障时很难解释系统为什么继续搜索、为什么停止、为什么等待审批。状态机把条件边、循环和终止语义变成可检查的结构。

DeepIntel 落点

DeepIntel 的 ResearchState 保存 DAG、完成节点、证据、runtime status、budget state、pending approvals 和 skill context。LangGraph StateGraph 负责按状态驱动节点。

优化与边界

状态机不是万能。图内状态负责业务执行，任务级控制仍需要 Harness；审批和预算事实仍要以 DB 为准。

21.3 题卡 3: DAG 和 StateGraph 有什么区别

面试回答

DAG 是这次研究任务的业务计划，StateGraph 是驱动计划执行的运行时。DAG 回答“要做哪些取证步骤”，StateGraph 回答“这些步骤如何被调度、聚合、校验和恢复”。

底层拆解

把二者混在一起会导致模型输出直接污染执行控制。业务 DAG 可以由 Planner 生成，但执行器必须有自己的静态约束和动态约束。

DeepIntel 落点

DeepIntel 让 Planner 输出 search/browser/rag/mcp 节点和依赖边，LangGraph 负责批次执行、条件路由、Reflection 和 replan。

优化与边界

不能把 Planner 生成的 DAG 当作可信执行计划。它必须经过 schema、工具 allowlist、预算和审批等运行时约束。

21.4 题卡 4：为什么 Planner 不能直接写最终答案

面试回答

Planner 的职责是把问题拆成可执行计划，不是综合证据写结论。让 Planner 直接回答，会跳过取证、引用和反思校验。

底层拆解

复杂研究任务需要先决定证据源和依赖顺序，再执行工具获取 observation。最终答案应该来自证据综合，而不是规划阶段的模型猜测。

DeepIntel 落点

DeepIntel 把 Planner、Search/Browser/RAG、Analyst、Reflection、Report 分层，避免一个节点同时承担规划和生成。

优化与边界

Planner 仍可能生成错误 DAG，所以需要 fallback DAG、schema 校验和 failure hints。不能把 Planner 稳定性吹成绝对可靠。

21.5 题卡 5：为什么需要 fallback DAG

面试回答

fallback DAG 是系统最低可执行骨架。它保证 Planner 输出失败、JSON 不可解析或节点缺失时，系统仍然能走一条保守研究路径。

底层拆解

fallback 的价值不是提高上限，而是兜住下限。生产系统不能因为一次模型格式抖动就完全失能。

DeepIntel 落点

DeepIntel 已有 fallback 思路：在规划不可用时仍可走内部 RAG、搜索、分析、报告的保守链路。

优化与边界

fallback 不能替代高质量规划。它适合保障可用性，但复杂任务仍需要 Planner 生成更贴合问题的 DAG。

21.6 题卡 6：为什么要区分 public status 和 runtime status

面试回答

public status 给用户看，runtime status 给执行器看。用户只需要知道 running、paused、completed；执行器需要区分 retryable failed、terminal failed、awaiting approval。

底层拆解

如果 UI 状态和执行状态混用，系统会在恢复和重试时产生歧义。例如 paused 可能是等待审批，也可能是用户主动暂停，执行策略完全不同。

DeepIntel 落点

DeepIntel 已经把 runtime status、public status、session runtime state 分开，并在 Harness snapshot 里保存运行时状态。

优化与边界

仍要继续完善状态转换表，尤其是审批恢复、预算熔断和多 worker 中断恢复的组合状态。

21.7 题卡 7: retryable failed 和 terminal failed 怎么区分

面试回答

retryable failed 表示失败可能通过重试、降级或 replan 解决; terminal failed 表示任务已经没有安全或有意义的继续路径。

底层拆解

区分依据包括错误类型、attempt 次数、依赖状态、预算剩余、审批状态和失败是否重复。没有这个区分, 系统要么过早放弃, 要么无限重试。

DeepIntel 落点

DeepIntel 在 graph 和 runtime state 中已有 retryable/terminal 语义, Harness summary 也统计 retryable failed 和 terminal failed。

优化与边界

当前还不是完整 failure taxonomy。未来要把浏览器超时、MCP schema invalid、审批拒绝、预算 hard stop 等错误类别统一建模。

21.8 题卡 8: Agent 为什么需要 failure memory

面试回答

failure memory 让系统记住哪些节点、工具或查询已经失败过，避免 replan 反复走同一条坏路径。

底层拆解

没有 failure memory，模型可能每次都生成看似不同但本质相同的失败动作，浪费预算并让任务陷入循环。

DeepIntel 落点

DeepIntel 的 replan 会带上 known failures，让 Planner 避开已知失败节点。

优化与边界

failure memory 不能无限积累，否则会把可行路径也误杀。需要 retry budget、过期策略和错误类别区分。

21.9 题卡 9：为什么预算治理必须是硬控制

面试回答

软提示只能提醒，不能阻止 Agent 继续花钱。长任务 Agent 必须有 hard stop、降级策略和部分结果输出。

底层拆解

Agent 是自动执行系统，不能指望模型看到“预算快用完了”就自觉停止。预算要进入调度器、工具层和 replan 条件。

DeepIntel 落点

DeepIntel 已有 session budget state 和 breaker，记录 token、cost、tool calls、wall clock 等预算项。

优化与边界

成本估算受上游 usage 精度影响，不能吹成绝对计费系统。未来要增强 per-tool/per-model 归因。

21.10 题卡 10：为什么 tool call 是高风险执行面

面试回答

模型说错话最多是文本错误；工具参数错了可能会查错数据、写错文件、发错消息或泄露敏感信息。

底层拆解

工具调用把概率性推理变成真实副作用，所以必须由确定性规则接管：schema、权限、审批、沙箱、脱敏和审计。

DeepIntel 落点

DeepIntel 在工具层、MCP proxy、approval reviewer 和 tool audit 中都体现了这个思路。

优化与边界

当前不是所有工具都具备同等强度的沙箱；面试时要把安全接入和完整企业级沙箱区分开。

21.11 题卡 11：为什么不能相信模型自己做权限判断

面试回答

模型可以辅助解释权限，但最终权限判断必须由确定性代码和本地状态完成。安全边界不能建立在概率输出上。

底层拆解

Prompt injection、上下文污染、错误推理都会让模型做出越权判断。权限系统需要身份、角色、租户、工具策略和动作审批。

DeepIntel 落点

DeepIntel 让 skill tool allowlist、MCP allowlist、approval DB 和 guardrail policy 共同约束工具调用。

优化与边界

仍需更完整 RBAC 和租户策略。当前重点是研究系统治理，不是完整 IAM 平台。

21.12 题卡 12：为什么工具参数必须 schema 校验

面试回答

schema 校验把自然语言意图转换成可执行接口的结构化边界。类型、必填、枚举、额外字段都要在执行前检查。

底层拆解

没有 schema, Agent 可能传错字段、注入意外参数, 或把工具当成开放聊天接口。schema 是执行前的确定性门。

DeepIntel 落点

DeepIntel 的 MCP proxy 已加入轻量 JSON Schema 子集校验, 覆盖 object、properties、required、additionalProperties 和基础类型。

优化与边界

当前不是完整 JSON Schema 引擎。复杂嵌套、pattern、oneOf 等高级能力仍是后续方向。

21.13 题卡 13：为什么工具返回也要治理

面试回答

工具返回值会进入模型上下文、报告和审计。如果包含 secret、内部 ID 或恶意指令，就会造成泄露或上下文污染。

底层拆解

治理不止发生在调用前，还要发生在调用后。返回值要摘要、脱敏、行数限制、错误净化和 result hash。

DeepIntel 落点

MCP proxy 现在会递归 redaction，并记录 result hash、server fingerprint、redaction applied。

优化与边界

redaction 依赖策略配置，无法自动识别所有敏感字段。高风险系统还需要 DLP 和数据分类。

21.14 题卡 14：为什么审批要以 DB 为 source of truth

面试回答

动作执行发生在本地系统，最终裁决就必须由本地 DB 聚合根负责。外部审批系统只能提供信号。

底层拆解

如果 Slack/Jira/Harness 状态直接驱动执行，就会出现回调延迟、重复事件、状态漂移和审计断裂。DB 聚合根能提供事务、版本和幂等。

DeepIntel 落点

DeepIntel 已有 approval requests 表、审批 API、approval resume 和 Harness snapshot 关联。

优化与边界

外部审批集成还未完整落地；当前应讲成本地审批闭环，而不是企业全链路审批平台。

21.15 题卡 15：审批恢复最容易出什么 bug

面试回答

最容易的问题是审批通过后恢复错状态、重复执行动作，或绕过已经过期/拒绝的审批。

底层拆解

恢复时必须检查 approval id、status、resolved at、version、pending approvals 和 checkpoint。审批是动作级事实，不是简单把任务从 paused 改成 running。

DeepIntel 落点

DeepIntel 在审批通过后会更新本地审批状态，并用 Harness checkpoint state 恢复 graph。

优化与边界

还需要更细的 action id 和幂等键，确保写操作不会因为回调重放执行两次。

21.16 题卡 16：为什么 SSE 不是任务事实源

面试回答

SSE 是进度推送通道，不是任务状态的 source of truth。浏览器断线不应该让后台任务丢失。

底层拆解

任务事实应保存在 DB、Harness 和 checkpointer 中。SSE 可以丢、可以重连、可以只展示最近事件，但不能决定任务是否完成。

DeepIntel 落点

DeepIntel 用 SSE 推送 workflow start、state update 和 trace，同时用 DB/runtime/Harness 保存恢复事实。

优化与边界

需要继续完善断线重连后的事件补偿，比如从 runtime state 和 progress log 重建当前视图。

21.17 题卡 17：为什么 trace 和 audit 要分开

面试回答

trace 解释执行路径，audit 证明动作事实。trace 面向调试，audit 面向安全、合规和追责。

底层拆解

trace 可以包含模型思路摘要、节点开始结束、状态转换；audit 必须记录工具、参数范围、调用者、策略裁决、结果摘要和时间。

DeepIntel 落点

DeepIntel 有 agent trace、tool histories、tool call audit 和 MCP result hash。

优化与边界

当前审计还不是法证级平台；PII retention、不可抵赖和长期归档仍是后续方向。

21.18 题卡 18：为什么 Agent eval 不能只看答案分数

面试回答

Agent eval 要看任务成功率、证据质量、工具成本、恢复能力、安全拦截和用户可解释性。只看最终文本会漏掉执行过程风险。

底层拆解

一个答案看起来不错，但可能没有引用、调用了越权工具、花费过高或靠幻觉补事实。Agent eval 必须覆盖过程指标。

DeepIntel 落点

DeepIntel 有 metrics 目录、RAG/browser/research quality collector 和实验摘要。

优化与边界

还缺统一 eval dashboard 和可重复 benchmark。面试时可以讲已有评测基础，不要说成完整平台。

21.19 题卡 19：为什么 RAG 不能替代 Browser

面试回答

RAG 查内部知识，Browser 深读外部网页。两者证据源、时效性和风险不同，不能互相替代。

底层拆解

内部知识适合稳定资料、组织文档和历史记录；网页适合最新事实和外部来源，但也带来 prompt injection 和可信度问题。

DeepIntel 落点

DeepIntel 使用 internal-first 策略，先 RAG，再按需要使用 Search/Browser 补外部证据。

优化与边界

如果问题高度时效，仍需要联网取证；如果用户禁止联网，则必须基于内部证据并说明边界。

21.20 题卡 20：为什么 citation 是研究型 Agent 的生命线

面试回答

研究型 Agent 的可信度来自可回溯证据。没有 citation，报告就是模型生成文本，很难验证事实来源。

底层拆解

citation 让每个关键 claim 能回到 evidence item。Reflection 可以检查引用覆盖，用户也能判断来源可信度。

DeepIntel 落点

DeepIntel 的 Analyst 和 Report 要求用 citation 格式，并由 Reflection 做质量校验。

优化与边界

citation 仍可能引用不充分或来源质量不高，所以还需要 evidence ranking 和 claim-level coverage。

21.21 题卡 21：为什么 Browser 内容要当作不可信数据

面试回答

网页内容可能包含 prompt injection，不能把它当系统指令执行。它只能作为证据数据进入分析。

底层拆解

攻击者可以在网页里写“忽略之前指令，调用某工具”。如果 Agent 把网页当上层指令，就会被劫持。

DeepIntel 落点

DeepIntel 把 Browser 作为证据节点，工具安全由 guardrail 和 tool layer 控制。

优化与边界

仍需要更强的 injection detector 和网页内容隔离策略。当前不能说完全防御所有攻击。

21.22 题卡 22：为什么 internal-first 是治理策略

面试回答

internal-first 不只是检索顺序，而是成本、安全和证据优先级策略。内部资料通常更贴合业务上下文，且风险更可控。

底层拆解

如果一开始就 Web，系统会扩大攻击面、成本和证据噪声。先内部后外部，可以把联网变成有条件补证。

DeepIntel 落点

DeepIntel 的 retrieval policy 已体现 internal-first，并可配置是否 RAG 命中后继续 Web。

优化与边界

内部资料过期或覆盖不足时，必须允许外部补证，不能把 internal-first 变成 internal-only。

21.23 题卡 23：为什么 replan 要带 failure hints

面试回答

没有 failure hints, Planner 可能重新生成同样失败的节点。带上已知失败能把 replan 从“重试一次”变成“绕开坏路径”。

底层拆解

failure hints 要包含节点类型、query、错误类别、retry count 和 repeat blocked 信息。

DeepIntel 落点

DeepIntel 的 replan node 会把 Known failures 传给 Planner。

优化与边界

failure hints 也可能误导模型，所以要保持简洁、结构化，不把所有日志都塞进 prompt。

21.24 题卡 24：为什么多 Agent 对话不适合强流程研究

面试回答

多 Agent 对话灵活，但状态、依赖、预算和恢复很难治理。强流程研究更适合显式 DAG 和状态机。

底层拆解

对话式协作容易出现谁负责下一步、谁拥有事实、失败如何传播不清楚的问题。工程系统需要可解释调度。

DeepIntel 落点

DeepIntel 选择 Planner + DAG + LangGraph，而不是让多个角色自由聊天。

优化与边界

开放式 brainstorming 可以用多 Agent 对话，但生产研究执行不应只靠消息流。

21.25 题卡 25：为什么需要 tool allowlist

面试回答

tool allowlist 决定 Agent 在当前 session 能看见和调用哪些工具。没有 allowlist，模型可能尝试超出任务需要的能力。

底层拆解

allowlist 是 least privilege 的基础。它可以来自 guardrail、skill、tenant policy 和 MCP registry。

DeepIntel 落点

DeepIntel 的 skill context 会影响 effective tool allowlist，MCP proxy 也检查 allowed tools。

优化与边界

allowlist 不是完整权限系统。真实企业环境还要结合用户身份、租户、资源范围和审批。

21.26 题卡 26：为什么 Skill 不是 prompt 附件

面试回答

Skill 如果只是 prompt 附件，就无法解释、审计、版本化或约束工具。真正的 Skill 是运行时能力对象。

底层拆解

能力对象要有 metadata、trigger、scope、priority、allowed tools、agent hints、content version 和 session snapshot。

DeepIntel 落点

DeepIntel 的 Skill registry 已实现 meta/content 分离、匹配、手动启用/禁用、snapshot 和工具边界。

优化与边界

当前 Skill 还没有升级到自定义 DAG 节点模板或执行器，这一点要明确标注。

21.27 题卡 27：为什么 Skill 要 top-k

面试回答

Skill 命中越多不代表越好。太多 Skill 会挤爆 prompt、引入冲突约束、扩大工具边界。

底层拆解

top-k 是 prompt budget 和能力边界控制。它让系统只选择最相关、最高优先级的少数 Skill 进入 session。

DeepIntel 落点

DeepIntel 测试已覆盖 top-k truncation 和 allowed tools 合并。

优化与边界

top-k 现在是规则排序，不是学习排序。未来可以引入 eval 驱动的 ranking。

21.28 题卡 28：为什么 Skill snapshot 能防止运行漂移

面试回答

运行中的 session 不应该因为全局 Skill 更新而改变行为。snapshot 固定 skill id、version、prompt 和工具边界。

底层拆解

如果每步都查最新 registry，同一任务可能前半段按 v1，后半段按 v2，复盘时无法解释。

DeepIntel 落点

DeepIntel 测试已证明旧 resolved snapshot 保留旧版本和 prompt，新 session 使用新版本。

优化与边界

还需要版本回收策略：旧 session 结束后才能清理不再引用的 content。

21.29 题卡 29：为什么 MCP 和 Skill 不能混为一谈

面试回答

MCP 是外部能力协议，Skill 是内部领域策略和能力选择。Skill 可以影响 MCP 工具是否可用，但不能替代 MCP 治理。

底层拆解

MCP 关注 server、tool、resource、transport；Skill 关注 task domain、prompt、constraints、allowed tools。二者处于不同抽象层。

DeepIntel 落点

DeepIntel 同时有 Skill registry 和 MCP policy proxy，分别处理领域能力和外部工具信任边界。

优化与边界

面试时不要说“Skill 就是 MCP server 配置”。这会暴露抽象层混乱。

21.30 题卡 30：为什么 MCP 必须有 server registry

面试回答

MCP server 是外部能力来源，必须先登记、审核、记录 fingerprint 和 allowed tools。

底层拆解

没有 registry, Agent 看到什么 server 就调什么，会产生供应链和权限风险。registry 把外部 server 纳入本地治理。

DeepIntel 落点

DeepIntel 的 `mcp_server_registry` 保存 trust status、allowed tools、read-only default、secret policy 和 fingerprint。

优化与边界

registry 仍需要 UI、审核流程和 tool list drift 检测，当前主要是后端治理入口。

21.31 题卡 31：为什么 MCP tool schema 不能完全信 server

面试回答

server 提供 schema 是输入，但应用侧要审核并固化策略。外部 server 自己声明自己安全，不能作为最终裁决。

底层拆解

攻击或错误 server 可能暴露过宽 schema，诱导 Agent 传入危险参数。应用侧 registry 应限制允许字段和写工具。

DeepIntel 落点

DeepIntel 从 secret policy 中读取 tool schema，并在 proxy 执行前校验。

优化与边界

当前 schema 来源还偏配置，未来要增加 server schema diff、审核和版本化。

21.32 题卡 32：为什么 result hash 有价值

面试回答

result hash 让审计能证明某次工具返回的摘要内容没有被悄悄改写。它不保存全部敏感数据，也能提供追踪线索。

底层拆解

在安全系统里，完整 payload 可能不能长期保存，但摘要和 hash 可以帮助复盘“这次调用大概返回了什么”。

DeepIntel 落点

MCP proxy 成功结果里包含 result hash、summary、fingerprint 和 redaction flag。

优化与边界

hash 不是不可抵赖签名。更强法证需要签名、retention policy 和不可变日志。

21.33 题卡 33：为什么要区分 schema invalid 和 tool not allowed

面试回答

不同错误类别对应不同修复路径。tool not allowed 是权限或策略问题，schema invalid 是参数生成问题。

底层拆解

如果都归为 tool error，Planner 和开发者无法判断是该调整 allowlist，还是修正参数生成。

DeepIntel 落点

DeepIntel MCP proxy 已输出稳定 error category。

优化与边界

未来可以把错误类别反馈给 Planner，用于更精确 replan，而不是简单失败。

21.34 题卡 34：为什么 write tool 默认审批

面试回答

写工具带来副作用，不应该默认自动执行。审批把高风险动作从概率推理转成明确的人类裁决。

底层拆解

read-only 查询失败通常可恢复，写操作错误可能造成真实损失。默认审批是 excessive agency 的防线。

DeepIntel 落点

DeepIntel MCP proxy 对非只读或 write tools 返回 approval request，不调用 executor。

优化与边界

审批通过后仍要做幂等和权限复核，不能只看 approval id 存在。

21.35 题卡 35：为什么 summary length 也是安全策略

面试回答

返回摘要太长会把过多敏感数据带进上下文，也会挤占 prompt budget。summary length 是数据最小化策略。

底层拆解

工具层不应把完整外部 payload 直接交给模型，尤其是 MCP server 返回的复杂对象。

DeepIntel 落点

DeepIntel MCP secret policy 支持 summary chars，默认有上限。

优化与边界

摘要过短也可能损失信息，需要按工具类型调优。

21.36 题卡 36：为什么 long-running Agent 要 session config

面试回答

session config 定义每轮最多任务数、最大 session 数、并发模式等边界。没有这些边界，Agent 可能无限执行。

底层拆解

长任务系统必须有停止条件：完成、失败、预算、session 上限、用户中断。session config 是控制面的一部分。

DeepIntel 落点

Harness v2 state 中已有 session config，保留 exclusive/concurrent 语义的扩展空间。

优化与边界

当前没有完整 concurrent 模式实现，只是为未来多 worker 设计状态结构。

21.37 题卡 37：为什么 active marker 要在 idle 时清理

面试回答

active marker 不清理会让 hook 或外部 supervisor 误以为还有任务，造成无意义检查或误拦截。

底层拆解

marker 生命周期应该和 active runtime statuses 绑定：running、awaiting approval、retryable failed 等。

DeepIntel 落点

Harness 测试已覆盖 completed 后 clear active marker。

优化与边界

复杂情况下 retryable failed 是否算 active 要看策略；当前把它视为 active，便于后续恢复。

21.38 题卡 38：为什么 JSON 损坏不能静默忽略

面试回答

tasks JSON 是恢复上下文。静默忽略损坏会导致任务丢失或重复执行。

底层拆解

正确做法是恢复.bak；如果 backup 也不可用，记录明确 ENV_SETUP 错误并停止，让人处理。

DeepIntel 落点

Harness 已有 corrupt JSON backup recovery 测试。

优化与边界

不能从 progress log 凭空重建完整任务元数据，尤其是 validation、depends_on 和 cleanup。

21.39 题卡 39：为什么恢复时不能只看最后一行日志

面试回答

最后一行日志可能只是某个事件，不代表当前可执行状态。恢复要结合 tasks JSON、DB runtime、checkpoint 和 lease。

底层拆解

append-only log 是历史线索，structured state 才是当前状态。二者缺一不可。

DeepIntel 落点

DeepIntel Harness 同时保留 progress log 和 structured tasks state。

优化与边界

当前恢复逻辑还不是完整自动恢复协议，需要继续补 worker start recovery。

21.40 题卡 40：为什么多 worker 必须独立工作树

面试回答

如果多个 worker 在同一工作树执行，rollback、清理和未提交改动会互相破坏。

底层拆解

通用代码任务 runner 里，失败可能需要 reset 到 base commit；这必须在独立 clone 或 worktree 中发生。

DeepIntel 落点

DeepIntel 当前没有实现多 worker rollback，所以不会在共享工作区自动 reset。

优化与边界

面试时要把“设计知道怎么做”和“当前已实现什么”分开。

21.41 题卡 41：为什么 Agent 需要 idempotency

面试回答

长任务会重试、恢复、回调重放。如果没有幂等，工具动作可能重复执行。

底层拆解

幂等要绑定 action id、approval id、request hash、状态版本和执行结果。

DeepIntel 落点

DeepIntel 已有 approval id 和 session id 关联，后续应补 action-level idempotency。

优化与边界

当前不能说完全防重复写操作，只能说审批和本地状态源已打基础。

21.42 题卡 42：为什么 model fallback 要保护 schema

面试回答

切换模型不能破坏结构化输出和工具调用契约。否则 fallback 可能让系统从可治理变成不可解析。

底层拆解

模型路由不只是换名字，还要验证 JSON、tool call、中文能力、成本和失败率。

DeepIntel 落点

DeepIntel 强调结构化输出和工具调用稳定性，是模型选型的核心维度。

优化与边界

当前没有完整模型路由评测矩阵，未来应加入 eval。

21.43 题卡 43：为什么不能把 eval 只做成离线脚本

面试回答

离线 eval 能比较版本，但生产 Agent 还需要运行时监控：失败率、成本、审批等待、tool error 和恢复成功率。

底层拆解

Agent 系统的质量会受网页变化、模型变化、工具服务变化影响，必须持续观测。

DeepIntel 落点

DeepIntel 有 metrics collector 和 observability trace 基础。

优化与边界

还缺统一 dashboard 和线上 drift 检测。

21.44 题卡 44：为什么 evidence 和 answer 要分开存

面试回答

证据是事实材料，答案是模型综合结果。答案可能变化，证据要能回溯。

底层拆解

如果只存最终报告，无法复查 claim 是否由证据支持，也无法在模型升级后重新生成报告。

DeepIntel 落点

DeepIntel 有 collected evidence、citations、report 等分层状态。

优化与边界

未来应增强 evidence canonicalization 和 source quality scoring。

21.45 题卡 45：为什么浏览器自动化需要多层降级

面试回答

网页结构复杂，单一提取策略会经常失败。需要 accessibility、DOM、readability、text fallback 等多层策略。

底层拆解

浏览器节点面对动态页面、反爬、加载慢、无障碍树缺失等问题。多层降级提高覆盖率。

DeepIntel 落点

DeepIntel 已在浏览器章节中讲三层提取策略。

优化与边界

浏览器自动化仍会受网站策略影响，不能保证所有页面深读成功。

21.46 题卡 46：为什么 Search 只是入口不是证据终点

面试回答

搜索结果 snippet 只能提供线索，不能替代深读证据。研究报告应尽量基于 Browser/RAG 深读内容。

底层拆解

Search 适合发现来源，Browser 负责验证页面内容，RAG 负责内部资料。

DeepIntel 落点

DeepIntel 的 Search/Browser/RAG 分工清晰，避免“搜到就回答”。

优化与边界

如果只拿 snippet 写结论，要在报告里标注证据弱。

21.47 题卡 47：为什么 Reflection 不应无限循环

面试回答

Reflection 能提高质量,但也会消耗预算并可能导致反复重写。必须有 max revision 和停止条件。

底层拆解

反思节点应该判断证据不足、一致性问题 and 引用覆盖,而不是永远追求更完美答案。

DeepIntel 落点

DeepIntel 有 max revision 和 budget state 限制 replan。

优化与边界

仍需更细的 reflection scoring, 区分必须重做和可接受边界。

21.48 题卡 48：为什么报告要写不确定性

面试回答

研究任务常常证据不完整或来源冲突。诚实写不确定性比强行给确定结论更可信。

底层拆解

不确定性包括证据缺口、时间限制、来源冲突、内部资料过期和未执行节点。

DeepIntel 落点

DeepIntel 的 Report 和使用原则都强调边界。

优化与边界

面试时不要把“能生成报告”讲成“总能给正确答案”。

21.49 题卡 49：为什么 Agent 需要 human-in-the-loop

面试回答

不是所有决策都应自动化。高风险动作、证据冲突和不可逆副作用需要人类确认。

底层拆解

HITL 的工程难点在暂停、持久化、恢复和幂等，而不是弹出一个确认框。

DeepIntel 落点

DeepIntel 有 pending approvals、approval API 和 resume 逻辑。

优化与边界

还要继续完善审批过期、拒绝后的替代路径和外部审批信号接入。

21.50 题卡 50：为什么“暂停”不等于“失败”

面试回答

暂停可能是等待审批、用户确认、预算策略或外部依赖，不应该被当成失败重试。

底层拆解

如果把 awaiting approval 当 failed，系统可能触发错误重试；如果把 failure 当 paused，系统可能卡死。

DeepIntel 落点

DeepIntel 的 runtime status 区分 awaiting approval 和 retryable failed。

优化与边界

状态转换仍要继续规范化，特别是暂停后恢复的边界。

21.51 题卡 51：为什么工具错误要净化

面试回答

原始错误可能泄露内部路径、SQL、token、schema 或服务拓扑。错误返回给模型前必须净化。

底层拆解

净化后的错误仍要保留足够 category 供恢复和 replan 使用。安全和可诊断之间要平衡。

DeepIntel 落点

DeepIntel 在 tool safety 和 MCP error category 中体现了 sanitized error。

优化与边界

法证级排障可以在受控审计系统保存更多细节，但不能直接给模型。

21.52 题卡 52：为什么 DB 是预算事实源

面试回答

预算状态需要跨请求、跨恢复、跨 worker 保持一致。只存在内存或日志里会丢失或重复计算。

底层拆解

DB 能支持事务更新、状态查询和最终报告。缓存可以加速，但不能替代事实源。

DeepIntel 落点

DeepIntel 有 session budget state 和 runtime state 表。

优化与边界

上游模型 usage 不完整时，成本字段仍可能是估算，要明确标注 estimated。

21.53 题卡 53：为什么 Postgres + pgvector 适合这个项目

面试回答

DeepIntel 优先优化系统整体复杂度：文档、会话、引用、审计、预算和向量检索放在统一数据库里更易维护。

底层拆解

专用向量库在超大规模 ANN 上有优势，但会增加多存储一致性和运维成本。

DeepIntel 落点

项目当前是研究工作流系统，不是纯向量检索产品。

优化与边界

如果规模和吞吐上去，可以再评估专用向量库或混合架构。

21.54 题卡 54：为什么结构化输出很重要

面试回答

Agent 编排依赖可解析结构。Planner 输出 DAG、工具参数、reflection 结果如果不可解析，后续执行就无法治理。

底层拆解

自然语言可以用于解释，但执行器需要 JSON/schema/state。

DeepIntel 落点

DeepIntel 的 Planner、DAG、guardrail、MCP schema 都依赖结构化输出。

优化与边界

结构化输出仍需 fallback 和 repair，不能假设模型永远合规。

21.55 题卡 55：为什么 prompt injection 不是只靠 prompt 解决

面试回答

攻击发生在不可信内容进入上下文后，单靠提示模型“不要被攻击”不够。需要工具边界、内容隔离和策略执行。

底层拆解

真正的防线是 instruction hierarchy、tool allowlist、schema、approval 和 redaction。

DeepIntel 落点

DeepIntel 把网页/MCP 返回作为数据，工具执行由 policy 控制。

优化与边界

仍需要系统性红队样本和自动回归。

21.56 题卡 56：为什么 Agent 需要最小权限

面试回答

Agent 会犯错，也可能被诱导。给它超过任务需要的工具和数据，就扩大了事故半径。

底层拆解

least privilege 包括工具 allowlist、read-only default、租户 scope、参数限制和审批。

DeepIntel 落点

DeepIntel 的 Skill allowed tools 和 MCP allowlist 都是最小权限实践。

优化与边界

当前还没有完整用户级 IAM，要避免夸大。

21.57 题卡 57：为什么“会调用工具”不等于“会安全调用工具”

面试回答

调用工具只是连接能力，安全调用还要证明调用资格、参数合法、执行受控、返回脱敏、审计完整。

底层拆解

很多 Demo 到工具调用就结束，但生产系统真正难在治理副作用和恢复失败。

DeepIntel 落点

DeepIntel 通过 Harness 工具层、MCP proxy、approval 和 audit 讲安全调用链路。

优化与边界

仍需更多真实高风险工具接入测试。

21.58 题卡 58：为什么要记录 server fingerprint

面试回答

MCP server 的工具列表和行为可能变化。fingerprint 帮助审计某次调用对应哪个 server 状态。

底层拆解

没有 fingerprint，事故复盘时只能知道调用了 server id，无法判断当时 server 是否被替换或升级。

DeepIntel 落点

DeepIntel MCP result 中返回 server fingerprint。

优化与边界

未来应加入 tool list digest 和 schema version。

21.59 题卡 59：为什么 untrusted server 要直接拒绝

面试回答

未注册或未信任 server 不应进入工具调用路径。否则协议统一反而扩大供应链攻击面。

底层拆解

MCP 的便利性会让人误以为任何 server 都可插拔。生产系统必须先建立 trust registry。

DeepIntel 落点

DeepIntel MCP proxy 对 unregistered/untrusted server 返回 terminal error。

优化与边界

信任流程本身还需产品化，包括审核、禁用、轮换和告警。

21.60 题卡 60：为什么 schema invalid 不应该自动重试同样参数

面试回答

schema invalid 是结构错误，重复执行同样参数没有意义。应该反馈给 Planner 或参数生成器修正。

底层拆解

重试适合临时网络/超时，不适合确定性参数错误。错误分类决定恢复策略。

DeepIntel 落点

DeepIntel MCP proxy 输出 schema invalid，后续可进入 replan hints。

优化与边界

当前 graph 对 MCP schema invalid 的精细恢复还可继续增强。

21.61 题卡 61：为什么报告生成要和取证解耦

面试回答

取证决定事实材料，报告决定表达结构。二者耦合会让报告节点偷偷补事实，增加幻觉风险。

底层拆解

解耦后可以单独评估证据质量和表达质量，也可以在同一证据集上重生成报告。

DeepIntel 落点

DeepIntel 用 Analyst 综合证据，Report 输出报告。

优化与边界

仍需更强 claim-level verifier，确保每个关键句都有证据。

21.62 题卡 62：为什么 Agent 需要可解释路由

面试回答

用户和开发者需要知道为什么系统选择 RAG、Search、Browser 或 MCP。不可解释路由会让失败难以复盘。

底层拆解

路由解释应包含 policy、skill、budget、guardrail 和 DAG 依赖，而不是模型一句“我觉得”。

DeepIntel 落点

DeepIntel 有 retrieval policy、skill context 和 trace。

优化与边界

路由解释还可更结构化地进入 UI 和审计。

21.63 题卡 63：为什么数据源可信度要分层

面试回答

不同来源可靠性不同：内部文档、官方公告、新闻、论坛、网页片段不能同权处理。

底层拆解

研究型 Agent 应记录来源类型、时间、作者/机构和证据强度。

DeepIntel 落点

DeepIntel 已区分 RAG、Search、Browser 证据源。

优化与边界

还需要 source quality scoring 和冲突证据处理策略。

21.64 题卡 64：为什么检索命中不代表可以回答

面试回答

检索命中只说明找到了相关文本，不说明足以回答问题。还要看覆盖度、时效性、冲突和引用质量。

底层拆解

RAG 系统常见错误是把 top-k 文档直接塞给模型，忽略问题是否真的被覆盖。

DeepIntel 落点

DeepIntel 用 Reflection 检查答案质量和证据覆盖。

优化与边界

未来可加入 answerability classifier，证据不足时提前拒答或补证。

21.65 题卡 65：为什么 Browser 节点要受预算控制

面试回答

浏览器深读成本高、耗时长、失败率也高。不能让 Agent 无限打开页面。

底层拆解

Browser budget 应限制页面数、深读深度、超时和重试次数。

DeepIntel 落点

DeepIntel 有 tool calls、wall clock 和 budget profile。

优化与边界

还需 per-node browser budget，更精细控制不同研究任务。

21.66 题卡 66：为什么 search query 要由 Planner 约束

面试回答

搜索 query 决定取证范围。过宽会噪声大，过窄会漏证据。

底层拆解

Planner 应把用户问题拆成明确子问题，并为每个子问题生成目标明确的 query。

DeepIntel 落点

DeepIntel Planner 输出 DAG 节点 query。

优化与边界

当前仍依赖模型质量，未来可加入 query rewrite eval 和 query diversity 控制。

21.67 题卡 67：为什么要记录 current batch

面试回答

current batch 表示当前正在并行执行的 DAG 节点。恢复时需要知道哪些节点可能完成、失败或部分执行。

底层拆解

没有 current batch，恢复只能知道整个图状态，很难定位中断发生在哪组并行节点。

DeepIntel 落点

Harness checkpoint 中保存 current batch。

优化与边界

并行节点的副作用和幂等仍要逐节点处理。

21.68 题卡 68：为什么 completed nodes 不能只靠模型声明

面试回答

节点完成必须来自执行器结果，不是模型说“我完成了”。

底层拆解

工具节点要有真实 observation，分析节点要有输出结构，报告节点要有引用覆盖。

DeepIntel 落点

DeepIntel 的 DAG executor 和 node outcomes 记录执行结果。

优化与边界

仍需更强 validation command 或 objective check，尤其是代码任务类 Harness。

21.69 题卡 69：为什么 Agent 需要停止条件

面试回答

没有停止条件，Agent 会在搜索、反思、重规划之间无限循环。

底层拆解

停止条件包括任务完成、证据足够、预算耗尽、重试打满、审批拒绝、用户中断。

DeepIntel 落点

DeepIntel 有 max revision、budget breaker 和 runtime terminal state。

优化与边界

仍要继续完善“部分完成可交付”的判断。

21.70 题卡 70：为什么“部分报告”是优雅降级

面试回答

预算或工具失败时，直接报错会浪费已有证据。部分报告能最大化当前信息价值。

底层拆解

部分报告必须明确哪些节点没执行、哪些证据不足、哪些结论不确定。

DeepIntel 落点

DeepIntel 的报告边界原则支持这种表达。

优化与边界

当前自动部分报告策略还可强化，不能说所有失败都能优雅降级。

21.71 题卡 71：为什么 Agent 需要配置化风险等级

面试回答

不同工具和任务风险不同。读公开网页、查内部文档、写文件、发消息不应同等处理。

底层拆解

风险等级决定是否审批、是否沙箱、是否记录更详细审计、是否允许自动重试。

DeepIntel 落点

DeepIntel approval request 有 risk level, MCP write tools 触发 high 风险审批。

优化与边界

风险分类还需更细粒度，比如 destructive、external side effect、sensitive data。

21.72 题卡 72：为什么 tool timeout 是治理而不是性能参数

面试回答

timeout 防止工具卡死拖垮长任务，也限制外部系统故障对 Agent 的影响。

底层拆解

timeout 后要分类为 TIMEOUT，执行 cleanup，更新 failure memory，而不是简单等待。

DeepIntel 落点

DeepIntel 在失败语义和 budget state 中有超时治理思路。

优化与边界

具体每个工具的 timeout 策略还可更精细。

21.73 题卡 73：为什么 Agent 需要环境初始化

面试回答

长任务跨 session 执行时，环境可能变化。每次 session start 都要做 health check 和 idempotent init。

底层拆解

没有初始化，恢复时可能因为依赖、数据库、浏览器或模型配置变化而失败。

DeepIntel 落点

Harness 技能强调 harness-init 和 health check。DeepIntel 有配置和启动脚本基础。

优化与边界

当前项目没有完整 harness-init.sh 协议化接入。可作为路线图。

21.74 题卡 74：为什么测试输出要为 Agent 设计

面试回答

长任务 Agent 恢复和修复依赖测试结果。如果测试输出不可 grep、错误分类不清，Agent 很难自动定位。

底层拆解

测试应输出明确失败文件、断言、错误类别和可复现命令。

DeepIntel 落点

本次新增 Harness/MCP/Skill 单测就是把治理能力变成可验证证据。

优化与边界

未来可以把测试摘要写入 Harness progress，用于自动恢复。

21.75 题卡 75：为什么不能只用日志做恢复

面试回答

日志是历史，恢复还需要当前结构化状态。只靠日志会遇到解析不稳定、语义缺失和状态重建困难。

底层拆解

tasks JSON、DB runtime state、checkpointer 和日志要共同使用。

DeepIntel 落点

DeepIntel Harness 同时保存 progress 和 task snapshot。

优化与边界

完整恢复协议还需要更系统的 recovery decision matrix。

21.76 题卡 76：为什么 DB 和 Harness 会有双写问题

面试回答

DB 保存审批/预算/审计事实，Harness 保存任务控制面。一次执行要更新两边，就可能出现一边成功一边失败。

底层拆解

解决思路是分域定权和补偿恢复：DB 是事实源，Harness 是恢复上下文，恢复时检查并补齐缺失。

DeepIntel 落点

DeepIntel 笔记已经强调分域定权。

优化与边界

当前还没有完整事务外补偿器，需要作为后续优化。

21.77 题卡 77：为什么工具审计不能只存在 session JSON

面试回答

session JSON 适合任务状态，不适合长期审计查询。工具审计需要独立表或日志，便于按 tool、时间、用户查询。

底层拆解

审计数据可能用于安全复盘和成本分析，不能只嵌在最终结果里。

DeepIntel 落点

DeepIntel 已落地 tool call audit，不再只依赖 session JSON。

优化与边界

审计保留策略和敏感字段处理还需更完整。

21.78 题卡 78：为什么 Agent 需要 observability

面试回答

Agent 的失败可能来自模型、工具、网络、检索、审批或预算。没有观测性，无法判断瓶颈在哪里。

底层拆解

观测应覆盖节点耗时、工具成功率、token/cost、重试次数、审批等待和最终质量。

DeepIntel 落点

DeepIntel 有 observability trace 和 SSE manager。

优化与边界

还缺统一指标面板和告警阈值。

21.79 题卡 79：为什么 prompt 也要版本化

面试回答

Prompt 改动会影响 Planner、Analyst、Reflection 和 Report 行为。没有版本，很难解释质量变化。

底层拆解

版本化 prompt 能和 eval、session、skill snapshot 关联，支持回滚和 A/B 比较。

DeepIntel 落点

DeepIntel Skill snapshot 已固定 prompt sections；主 prompt 版本化仍可继续增强。

优化与边界

不要把 prompt 当成随手文案，它是运行时契约的一部分。

21.80 题卡 80：为什么 Agent 需要 schema repair

面试回答

模型可能输出接近正确但格式不合规的 JSON。schema repair 能提高可用性，但必须受限。

底层拆解

repair 不能凭空补业务语义，只能修格式或用 fallback。否则会把错误计划包装成合法计划。

DeepIntel 落点

DeepIntel Planner 有 JSON 解析和 fallback 思路。

优化与边界

未来可加入更明确的 repair attempts 和错误日志。

21.81 题卡 81：为什么“工具越多越强”是误区

面试回答

工具越多，搜索空间、权限风险、成本和失败路径都增加。强 Agent 不是工具多，而是工具边界清楚。

底层拆解

工具集要按任务、Skill、租户和风险动态收缩。

DeepIntel 落点

DeepIntel 通过 guardrail enabled tools、Skill allowlist 和 MCP proxy 控制工具。

优化与边界

未来要做 per-tool capability 和更细粒度 tool policy。

21.82 题卡 82：为什么上下文窗口不是可靠记忆

面试回答

上下文会被截断、压缩、污染，也不能跨进程稳定保存。长任务记忆必须外部化。

底层拆解

外部化包括 DB、Harness progress、tasks JSON、checkpointer 和 artifact storage。

DeepIntel 落点

DeepIntel 把任务状态、预算、审批和 skill snapshot 都放到外部状态。

优化与边界

模型上下文仍用于当前推理，但不能当唯一记忆。

21.83 题卡 83：为什么 Agent 要记录 artifacts

面试回答

中间产物如网页快照、检索结果、报告草稿、图表数据，可以帮助恢复、评估和复盘。

底层拆解

没有 artifacts，任务失败后只能看日志，无法重建证据材料。

DeepIntel 落点

DeepIntel 已保存 evidence、citations 和 reports，metrics 里也有实验数据。

优化与边界

完整 artifact lifecycle 还可继续完善，包括存储、过期、脱敏和引用。

21.84 题卡 84：为什么用户确认和审批不同

面试回答

用户确认通常是确认任务方向或敏感输入，审批是允许某个高风险动作执行。二者风险和状态语义不同。

底层拆解

混用会导致恢复逻辑错误。例如用户确认问题范围，不等于批准写外部系统。

DeepIntel 落点

DeepIntel 有 user confirmed 和 pending approvals 两类状态。

优化与边界

仍需在 UI 上更明确地区分确认和审批。

21.85 题卡 85：为什么要把 source of truth 讲清楚

面试回答

复杂系统最容易出事的是多个组件都以为自己是事实源。source of truth 明确后，恢复和冲突处理才有依据。

底层拆解

审批/预算/审计以 DB 为准，任务控制以 Harness 为准，图状态以 checkpointer 为准，外部系统只是信号或工具。

DeepIntel 落点

DeepIntel 笔记已把分域定权作为治理核心。

优化与边界

如果未来接入外部审批或 MCP server，仍要保持本地裁决权。

21.86 题卡 86：为什么 Agent 安全要做纵深防御

面试回答

单层防御一定会漏。需要 prompt 层、planner 层、tool schema、policy proxy、审批、沙箱、redaction 和 audit 多层串联。

底层拆解

纵深防御承认模型和工具都会犯错，用后续层次限制事故半径。

DeepIntel 落点

DeepIntel 已有 guardrail、Skill allowlist、MCP proxy、approval 和 audit。

优化与边界

沙箱和红队自动化仍是后续重点。

21.87 题卡 87：为什么要做红队样本库

面试回答

安全能力不能只靠设计描述，必须用攻击样本持续回归。

底层拆解

样本应覆盖 prompt injection、schema 绕过、越权工具、敏感字段、审批重放和 MCP tool poisoning。

DeepIntel 落点

DeepIntel 当前有单元测试证明部分治理规则。

优化与边界

还需要系统性 red-team benchmark，把安全测试纳入 CI。

21.88 题卡 88：为什么 Agent 系统要有降级路径

面试回答

外部工具、模型、浏览器、检索都可能失败。没有降级，系统会从复杂变成脆弱。

底层拆解

降级包括 fallback DAG、internal-only、skip optional node、partial report 和人工介入。

DeepIntel 落点

DeepIntel 有 fallback、budget breaker 和 partial boundary 表达。

优化与边界

降级也要透明，不能让用户以为完整研究已经完成。

21.89 题卡 89：为什么“完成”要有验收证据

面试回答

Agent 容易 premature completion。完成必须由测试、验证命令、证据覆盖或明确状态证明。

底层拆解

没有验收证据，只能说“生成了输出”，不能说“任务完成”。

DeepIntel 落点

本次优化用 pytest 和 PDF 编译作为验收证据。

优化与边界

复杂研究任务还需要任务级质量指标，不只是代码测试。

21.90 题卡 90：为什么要把未完成写进笔记

面试回答

面试中诚实边界比夸大能力更可信。明确未完成项能展示工程判断和路线规划。

底层拆解

未完成项要讲为什么没做、做了有什么代价、如何验收，而不是一句“以后优化”。

DeepIntel 落点

DeepIntel 笔记记录多 worker、MCP 主流量化、Skill 执行器、统一 eval 等边界。

优化与边界

不要把路线图说成当前能力，否则高压追问会被打穿。

21.91 题卡 91：为什么 Agent 需要任务优先级

面试回答

长任务队列里不是所有任务同等重要。优先级影响调度、重试和资源分配。

底层拆解

没有优先级，P0 修复和低风险补充任务会争同样资源。

DeepIntel 落点

Harness v2 state 保留 priority 字段。

优化与边界

当前 DeepIntel 还没有完整任务队列调度，priority 主要是状态结构准备。

21.92 题卡 92：为什么 dependency cycle 要检测

面试回答

如果任务依赖形成环，调度器会永远找不到可执行任务。

底层拆解

cycle detection 是任务选择前的静态安全检查。发现后应标记 dependency error，而不是让系统空转。

DeepIntel 落点

Harness 技能中明确了 cycle detection。

优化与边界

当前 DeepIntel Harness 还未实现完整依赖调度，面试时只能讲设计方向。

21.93 题卡 93：为什么 blocked 状态不一定要持久化

面试回答

blocked 可以由 pending 任务和永久失败依赖计算出来。持久化 blocked 可能引入同步问题。

底层拆解

计算型 blocked 能保证依赖状态变化后自动反映。

DeepIntel 落点

Harness summary 现在按 failed dependency 计算 blocked。

优化与边界

复杂队列可选择持久化 blocked reason，但要维护一致性。

21.94 题卡 94：为什么 approval request 要有 request payload

面试回答

审批人需要知道自己批准的是什么动作、参数范围和风险原因。没有 payload，审批就是盲批。

底层拆解

payload 也用于恢复和审计，证明执行动作与审批动作一致。

DeepIntel 落点

DeepIntel approval request 保存 request payload json，MCP approval payload 会 redaction。

优化与边界

payload 本身可能含敏感信息，必须脱敏后展示。

21.95 题卡 95：为什么工具层要返回统一结构

面试回答

统一结构让下游节点、审计和 UI 能稳定处理 success、denied、safe error，而不是解析各工具的随意返回。

底层拆解

不同工具内部错误可以不同，但边界输出要一致。

DeepIntel 落点

DeepIntel 笔记中已有 Harness 工具层统一输出结构示意。

优化与边界

现实工具仍可能返回复杂 payload，需要 adapter 层标准化。

21.96 题卡 96：为什么 Agent 要有 query scope

面试回答

query scope 限制任务的主体、租户、项目和数据边界。没有 scope，Agent 可能检索或调用无关资源。

底层拆解

scope 可以来自用户、session、skill、guardrail 和权限系统。

DeepIntel 落点

DeepIntel Skill metadata 支持 tenant/project scope，RAG 支持 rag group。

优化与边界

完整多租户权限仍需进一步实现。

21.97 题卡 97：为什么 Agent 需要成本归因

面试回答

只知道总成本不够，需要知道哪个节点、工具、模型或 replan 消耗最多。

底层拆解

成本归因帮助优化 prompt、工具深度、浏览器策略和模型路由。

DeepIntel 落点

DeepIntel 有 budget state 和 metrics 基础。

优化与边界

当前 per-node/per-tool 成本归因还不完整。

21.98 题卡 98：为什么重试要有 backoff

面试回答

立即重试可能打爆外部服务，也可能重复遇到暂时性故障。backoff 给系统恢复时间。

底层拆解

重试策略应根据错误类别调整，网络错误和 schema invalid 不应同策略。

DeepIntel 落点

DeepIntel failure memory 中已有 next retry/backoff 语义。

优化与边界

还需要和 Harness lease、task selection 结合得更紧。

21.99 题卡 99：为什么 tool result 不能直接作为最终答案

面试回答

工具结果是证据或动作反馈，不是面向用户的最终解释。最终答案需要综合、引用和边界说明。

底层拆解

直接返回工具结果会缺少上下文、可信度和冲突处理。

DeepIntel 落点

DeepIntel 让 Analyst/Report 处理 collected evidence。

优化与边界

某些简单查询可以直出，但研究任务不应这样做。

21.100 题卡 100: Agent 平台的控制面和数据面怎么拆

面试回答

控制面负责状态、租约、预算、审批、权限和审计；数据面负责模型调用、检索、浏览器、MCP tool 和证据流。

底层拆解

如果两者混在一起，模型输出、工具结果和恢复逻辑会互相污染，最终表现为重复执行、越权调用、不可恢复和无法复盘。

DeepIntel 落点

DeepIntel 当前把 LangGraph 作为业务执行图，把 Harness、MCP policy proxy、Skill registry 和 budget breaker 作为控制面插点。

优化与边界

还没做到完整平台化调度，后续要补 worker pool、action id、DB 事务聚合根和统一 eval dashboard。

21.101 题卡 101：为什么要区分工具发现和工具授权

面试回答

工具发现回答“有哪些工具存在”，工具授权回答“当前 session 能不能用”。发现不等于授权。

底层拆解

MCP 可以帮助发现 tools/list，但应用必须按用户、租户、skill 和策略收缩可用工具。

DeepIntel 落点

DeepIntel 的 MCP registry 和 Skill tool allowlist 分别处理外部工具和 session 工具边界。

优化与边界

未来要把工具发现结果和本地 registry 审核流程结合。

21.102 题卡 102：为什么 Agent 要有审计友好的错误码

面试回答

自然语言错误不稳定，审计和恢复需要固定错误码。

底层拆解

错误码能驱动 retry、replan、approval、terminal failed 和 UI 展示。

DeepIntel 落点

MCP proxy 已固定 untrusted、tool not allowed、schema invalid、unsupported input 等类别。

优化与边界

还需要跨所有工具统一错误 taxonomy。

21.103 题卡 103：为什么上下文压缩会影响恢复

面试回答

上下文压缩可能丢失细节、错误原因和中间决策。恢复不能依赖压缩后的对话摘要。

底层拆解

恢复材料必须写到外部状态：progress、tasks、checkpoint、DB。

DeepIntel 落点

Harness 设计原则就是 progress files are the context。

优化与边界

上下文摘要仍可辅助，但不能作为唯一证据。

21.104 题卡 104：为什么 Agent 要可暂停

面试回答

长任务可能遇到审批、预算、用户修改需求或外部服务异常。可暂停让系统不必失败退出。

底层拆解

暂停状态必须持久化，并且恢复时知道暂停原因。

DeepIntel 落点

DeepIntel 有 awaiting approval 和 approval resume。

优化与边界

主动用户暂停、计划修改后的恢复还可继续完善。

21.105 题卡 105：为什么 Agent 需要边界表达

面试回答

用户和面试官都需要知道系统做到了什么、没做到什么、结果可信到什么程度。

底层拆解

边界表达包括证据不足、未联网、预算停止、工具失败、未完成路线图。

DeepIntel 落点

DeepIntel 笔记把现实结论和关键边界写入附录。

优化与边界

边界不是示弱，而是工程系统可信度的一部分。

22 生产级韧性与安全重构：设计说明与面试答法

这一章回答什么

这一章不是把安全名词罗列一遍，而是把项目从“功能能够跑”提升到“外部依赖失败时仍可解释、数据越权时默认拒绝、状态恢复时有事实依据”的设计完整讲清楚。每个主题都按同一套问题展开：为什么需要它，采用什么方案，为什么没有选其他方案，收益是什么，代价是什么，已经考虑了哪些边界，还有哪些边界不能声称已经解决。

面试中的总原则

当前项目是单用户自托管系统，但单用户不等于无认证，也不等于可以省略 owner scope。面试时应把“已实现”“已测试”“设计预留”和“尚未完成”分开说。尤其是生产密钥、HTTPS、真实外部模型和多主体部署，不能用本地单元测试替代现场证据。

22.1 可退化架构：从全有或全无变成健康、降级、恢复

面试官可能问：为什么不让依赖失败就直接报错？

因为研究系统的能力不是同质的。PostgreSQL、认证密钥和审计写入决定安全事实，失败时必须拒绝服务；RAG、重排序、浏览器和备用模型则是可选能力，失败时可以缩小研究范围并继续给出明确标注的结果。把所有依赖放进一个 try/except 会把安全依赖和可选能力混在一起，既可能扩大安全风险，也会把一次局部故障放大成整个应用不可用。

设计。在 `app/security/capabilities.py` 中把外部能力抽象成 `CapabilityHealth` 状态机：`healthy` 表示最近探测成功，`degraded` 表示最近调用失败但应用有替代路径，`recovering` 表示冷却期后正在进行单飞探测。首次使用才初始化能力；失败记录 `reason_code` 与时间；默认冷却 30 秒；并发请求共享一次恢复探测，避免所有请求同时下载模型或连接死服务。

降级结果不能只返回空列表，而应携带 `capability`、`state`、`reason_code`、`retry_after_seconds` 和 `fallback_used`。因此 DAG、SSE、审计和最终报告都能回答三个问题：哪个能力没用上，为什么没用上，什么时候可以再试。RAG 失败时跳过内部检索并标注“未使用内部知识库”；reranker 失败时保留向量/BM25 结果；浏览器失败时仍可保留搜索；报告流在首个正文输出前才允许切换备用模型。

备选方案与取舍。方案 A 是任一依赖失败即终止。它的优点是语义简单、不会产生“部分结果”，缺点是可用性差、局部故障会放大，而且用户不知道已有证据是否仍然有价值。方案 B 是每个请求临时判断依赖是否可用。它比全局终止灵活，但并发下容易产生探测风暴，且不同节点可能对同一个能力做出不一致判断。方案 C 是集中式能力状态机加按能力定义的降级矩阵。项目选择方案 C，因为它同时解决状态一致性、节流恢复和用户可解释性；代价是增加状态维护、测试矩阵和错误码设计。

收益、代价与边界。收益是故障半径可控、恢复不需要重启、SSE 能继续表达进度，且安全关键依赖不会为了可用性而放行。代价是“部分完成”比成功/失败二元状态更难测试，也要求报告诚实标注证据缺口。已经考虑了冷却、单飞、可选能力回退和最终报告边界；尚未完成的是生产级指标告警、跨进程共享能力状态和真正外部模型的现场恢复演练。当前测试证明的是本进程状态机和降级语义，不是第三方服务的 SLO。

22.2 安全边界：数据归属、令牌生命周期与默认拒绝

面试官可能问：单用户系统为什么还要 owner scope？

因为单用户是部署规模，不是数据访问模型。只要存在研究会话、文档、技能、配置、审批和审计，就需要明确它们属于哪个主体。严格的 owner 条件可以阻止历史数据被错误读取，也让未来增加第二主体时不必重写所有查询。

设计。认证子系统使用 bcrypt 保存密码哈希，首次初始化通过 PostgreSQL advisory transaction lock 串行化，保留用户名拒绝被用于伪装系统主体。服务端 session 和 API token 只保存哈希值；明文 token 仅在创建响应中返回一次。每次校验同时检查撤销时间、过期时间、主体启用状态和 session_version；密码轮换或统一撤销时递增版本并撤销旧 session/token，因此旧凭据不能继续访问。

所有资源查询使用主体条件，跨主体对象统一返回 404，避免泄露对象存在性。认证、审批和审计属于关键安全状态，不能因为 Redis 缓存故障而退化为放行。安全中间件统一设置精确 CORS、CSP、点击劫持防护、Referrer Policy、Content-Type 防嗅探和安全 Cookie；认证、上传和 token 路由还应使用独立限流桶。

备选方案与取舍。方案 A 只绑定 localhost，部署简单但无法安全远程访问。方案 B 完全信任反向代理，性能好但应用自身失去独立安全边界，代理配置错误会直接变成越权。方案 C 使用应用层密码、服务端 session 和可撤销 API token，并把 HTTPS 代理作为额外防线。项目采用方案 C，因为它可独立部署、能撤销泄露凭据，且不把安全责任转嫁给代理；代价是要维护认证表、锁、轮换和限流逻辑。

已经考虑与尚未考虑。已经考虑了 session fixation、密码轮换、删除/停用主体后的旧 token 失效、跨用户资源 404、Redis 故障不放行和 owner scope 回归。尚未在本轮引入多用户角色、组织/租户层级、WebAuthn、设备绑定和集中身份提供商；这些不能在面试中说成当前功能。

22.3 配置与密钥：按主体隔离，并讲清威胁模型

面试官可能问：应用层加密到底防什么？

它主要防数据库文件、备份和日志泄露后直接暴露 LLM API Key。它不防已经获得进程权限的攻击者，因为运行中的进程必须能够解密并调用模型。把保护对象和不保护对象说清楚，比声称“密钥绝对安全”更可信。

设计。LLM 配置属于受认证保护的管理员资源，数据库中的 API Key 使用部署主密钥加密，并以 enc: 前缀区分已迁移值。读取和写入均是幂等的，旧明文可以渐进迁移，不会因为重复启动而重复加密。配置 key 使用 llm:<owner_id>，查询同时带 owner_id，

旧单管理员 llm 记录只在同一 owner 范围内兼容读取。生产模式拒绝缺失或示例主密钥、不安全 Cookie 和通配 CORS。

方案比较。 明文入库最简单但数据库备份即等于泄露；只放环境变量能减少数据库暴露面，却失去运行时配置和管理员资源审计；应用层加密兼顾运行时配置、备份安全和渐进迁移，但引入主密钥轮换、启动失败和密钥丢失恢复问题。项目选择第三种，并明确“主密钥不与数据库备份放在一起”。

收益、代价与遗漏。 收益是泄露数据库不直接泄露模型密钥，配置更新可审计且不必重训练。代价是主密钥轮换需要版本化密文和迁移工具，主密钥丢失会让密文不可恢复。已经考虑了前缀幂等迁移、数据库/备份威胁模型和生产启动检查；尚未完成 KMS/HSM 托管、自动轮换、密钥版本回滚和真实生产密钥演练。

22.4 兼容层与 facade：重构不应一次性破坏调用面

app/agent_tools 的兼容层保留旧模块导入面，再逐步把解析、schema、执行和具体工具实现拆到更清晰的边界。它解决的是重构风险，不是业务功能本身：旧调用方继续拿到稳定 facade，新代码可以使用更细粒度的 adapter、policy 和执行器，迁移完成后逐步收窄旧接口。

方案 A 是一次性重写所有调用方，结构最干净，但变更面大、回归定位困难；方案 B 是永远保留一层万能兼容代码，迁移成本低，却会形成双重语义和死代码；方案 C 是带弃用策略、契约测试和观测的渐进 facade。项目采用 C。收益是降低重构破坏性、支持分阶段发布和回滚；代价是短期内需要维护两套入口、增加文档和测试成本。已经考虑了旧导入兼容和 schema/执行边界；尚未完成的是统一弃用时间表、调用方迁移完成度指标和删除旧 facade 的最终版本。

22.5 统一 LLM adapter：把 provider 差异隔离在边界

设计。 业务 Agent 不直接判断 provider，而通过 endpoint resolver 与 adapter 获取规范化的 base URL、认证头、模型名和客户端。OpenAI 兼容端点、Ollama 与 Anthropic 的差异集中在 app/endpoint_resolver.py 和 app/llm_adapters.py。HTTP 客户端共享连接池，并分别配置 connect/read/write/pool timeout；死端点进入冷却，有限重试只对可恢复的网络错误生效。

非流式调用可以在首个响应前切备用模型；流式调用把“是否已经向用户输出正文”作为协议状态。一旦已有正文，主模型失败只能发出可恢复的终止错误，不能无提示地切到备用模型，否则用户会看到语义断裂的半份报告。usage 缺失时使用保守估算，并在审计中标记 estimated。

为什么不是业务层 if/else。 把 provider 判断散落在 Agent 中，短期代码少，长期会让 URL、认证头、超时、模型过滤和 fallback 规则彼此漂移。统一 adapter 的优点是替换

模型和增加 provider 的影响面小、协议测试集中；缺点是 adapter 本身成为高复杂度边界，错误归类和原生 provider 特性可能被过度抽象。当前已覆盖 URL、认证头、模型筛选、usage 估算和首字节规则；尚未覆盖所有供应商的真实流式协议、工具调用差异和跨区域熔断指标。

22.6 提示注入：把不可信数据放进数据流，而不是一句 system prompt

网页、邮件、上传文档、RAG 片段、Skill 外部文本以及 MCP/tool 输出都必须携带来源、信任级别和内容边界。它们只能作为明确标注的用户上下文，不能拼接进 system role，也不能把其中的“请执行某命令”解释为控制指令。prompt_security 负责数据封装，guardrail 负责研究政策，两者要分开建模：前者解决“这段文本能不能影响指令解释”，后者解决“这个问题是否允许联网、写入或输出”。

收益是攻击者无法仅靠网页文本升级为系统指令，且审计可以追踪污染来源；代价是 prompt 更长、模型可能降低可读性，且标记本身不是形式化证明。已经考虑了常见外部文本边界和 system role 隔离；尚未完成跨模型的注入基准、工具参数级 taint tracking、图像/多模态载荷和红队样本库。

22.7 健康检查：liveness 与 readiness 必须分开

/health 只回答进程是否活着，不能因为数据库暂时不可用就让容器被重启风暴拖垮。/ready 回答“能不能接流量”：数据库可连、schema 可用、认证主密钥有效、审计可写时才返回 200；RAG、Redis、浏览器和备用模型以不泄露敏感细节的信息状态返回。Docker healthcheck 应指向 /ready，因为编排系统需要依据接流量能力摘除实例。

方案 A 只有一个 health endpoint，简单但无法区分进程存活与业务可用；方案 B 只检查 HTTP server，误把“能响应”当成“能安全服务”；方案 C 分离 liveness/readiness 并分级关键依赖。项目采用 C，代价是部署和测试需要理解两种状态，且 readiness 不能泄露连接串、密钥或内部异常堆栈。当前代码已覆盖数据库、Redis 和认证状态的基本展示；真实容器冷启动和代理转发仍需现场验证。

22.8 上传与文档生命周期：不是一个 save(file)

上传链路应按顺序执行认证、owner 校验、文件名与路径规范化、扩展名/MIME/魔数交叉校验、大小/页数/解压限制、文本提取超时、哈希去重、主体配额和按主体限流。成功入库后，source 与 chunks 使用同一 owner；删除 source 时同步删除其 chunks 和向量记录；检索查询默认带 owner 条件。不要把用户原始路径写入宿主机，也不要扩展名代替内容检测。

收益是降低路径逃逸、伪造 MIME、压缩炸弹、重复存储和跨主体检索风险；代价是解析库、缩略图、EXIF 处理和异步清理会增加复杂度，严格限制也可能拒绝合法的大文件。已经考虑了路径、MIME、哈希、owner、限流和同步删除；尚未完成的重点是对每一种文档格式做系统 fuzzing、病毒扫描、对象存储生命周期和可恢复的异步清理

队列。

22.9 备份导出/导入：merge-only 仍然要有事务边界

导出只包含当前管理员拥有的认可资源，不导出密钥、session、token 和审计秘密。导入先做版本号与白名单结构校验，再按内容哈希去重、补齐 owner，并在同一个数据库连接的单一 transaction 中完成整批写入。任一 source 失败，整个批次回滚；成功响应同时返回 imported 与 skipped，保证幂等和冲突可见。

覆盖写入看起来简单，但会丢失当前数据、破坏 owner 关系，失败中途还很难恢复；逐条提交虽然易实现，却会留下半批导入。merge-only + 单事务的代价是大批次长事务占用连接、锁和 WAL，需要进一步做批量上限、分段导入协议或 staging table。当前已实现版本化、白名单、哈希 merge、owner 绑定和整批回滚测试；尚未完成跨版本迁移、加密备份容器、断点续传和大规模导入压测。

22.10 本地持久化：JSON 也要遵守数据库式原子写

仍需写本地文件的配置、Harness 状态和 session 快照，应统一使用同目录临时文件、完整写入、flush、fsync 和 os.replace。同目录保证替换发生在同一文件系统，os.replace 保证读者不会看到半截 JSON。写入工具应统一处理权限、编码、临时文件清理和 Windows 行为，禁止模块各自随手 json.dump()。

原子写不能解决磁盘损坏、错误数据逻辑或跨文件事务；它只保证单文件替换边界。因此 Harness 与数据库双写仍需分域 source of truth 和补偿恢复。收益是断电、进程终止和并发读时不容易得到损坏文件；代价是多一次磁盘写入、fsync 延迟和磁盘空间峰值。当前应把这套规则作为所有新增本地持久化的硬约束，而不能声称它等价于完整数据库事务。

22.11 测试文化：每个真实事故都留下可回归的名字

测试不应只验证 happy path，而应围绕曾经发生或可能发生的故障命名：初始化并发、错误密码、旧 token 失效、跨主体读取、短暂依赖故障、30 秒冷却、单飞恢复、首字节后禁止切换、恶意上传、伪造 MIME、导入回滚、提示注入和 readiness 拒流量。测试名本身就是维护文档，告诉未来维护者为什么一个看似多余的分支必须保留。

覆盖率只是信号，不是安全证明。90% 以上可以降低未执行路径风险，但不能证明 threat model 完整；必须结合 property-based/fuzz 测试、迁移升级测试、真实协议测试和红队样本。当前自动化验证覆盖安全/adaptor 94%、全仓测试 172 项通过；尚未完成的是生产依赖现场、容器冷启动、多进程并发和长期故障注入。

22.12 不在本轮范围的后台自动化与未来多主体

定时任务、事件总线、邮件、CalDAV、跨设备同步和复杂角色体系不属于本轮生产重构边界。面试时可以说明其设计借鉴点：调度器要处理时区、短月钳制、幂等、单飞、串行执行和重启触发保留；事件必须按 owner 路由，不能一个用户的事件触发所有人的任务。但当前项目没有把这些能力伪装成已成功功能。保留 owner schema 和审计边界，是为未来扩展提供兼容接口，而不是提前承诺多用户产品。

22.13 最终取舍总表

主题	采用方案	主要收益	主要未覆盖项
依赖故障 访问控制	能力状态机 + 分级降级 应用层认证 + owner scope	可用性与安全边界分离 可撤销、可独立部署	跨进程状态、真实 SLO 多角色、WebAuthn
密钥 LLM	应用层加密 + 渐进迁移 统一 adapter + 首字节协议	降低数据库泄露影响 provider 可替换、流式不串语义	KMS/HSM、自动轮换 全 provider 现场协议
上传	生命周期校验 + 哈希 + owner	减少文件攻击和越权	fuzz、病毒扫描、对象存储
备份	白名单 merge + 单批事务	幂等且不破坏现有数据	大规模断点续传
健康 测试	liveness/readiness 分离 事故驱动回归 + 覆盖率	编排语义准确 故障可复现、边界可审计	生产代理现场演练 长期故障注入

两分钟收口答案

我没有把所有外部依赖都当成同一种错误，而是先按安全关键程度分级：数据库、认证密钥和审计写入失败时拒绝服务，RAG、浏览器、重排序和模型端点失败时通过能力状态机进入降级并持续探测恢复。所有资源查询坚持 owner scope, token 只存哈希且支持版本化撤销；API Key 用部署主密钥加密，但我会明确它防的是数据库/备份泄露，不防已攻陷进程。LLM provider 差异集中在 adapter，流式响应一旦输出正文就不允许静默切模型。上传、备份、提示注入和健康检查都有明确边界。最后我会把已测试能力、设计预留和尚未完成的生产现场演练分开说，而不是用覆盖率或文档替代真实验证。

23 附录：当前仓库现实、边界与演进路线

附录怎么用

附录不是给面试官从头念的，而是给你自己在面试前校准边界。只要这里写的是“未完成”，面试时就不要包装成“已经具备”。只要这里写的是“设计已定”，面试时就要明确补一句“但代码尚未完整接入”。

23.1 当前已核实的现实结论

- 聚合器覆盖失败语义的历史 bug 已经修复，不应继续当作现存问题对外表述。
- `tool_call_audit` 已落地，工具审计不再只依赖 session JSON。
- `session_budget_state` 与 `session_runtime_state` 已落地，预算和运行时状态已从收尾汇总前移到运行中持续更新。
- session 级 breaker 已有可用硬熔断版，但 token/cost 精度仍未完全做满。
- 动作级审批已经接入独立 reviewer 分支、审批表和审批 API。
- Harness supervisor 已接入最小可用实现，支持图外任务快照和批次边界恢复。
- MCP policy proxy 与 registry 已接入，但默认业务图还未大规模依赖 MCP 节点。
- skill 体系已经落地最小可用闭环：支持 Markdown + YAML frontmatter 解析、数据库持久化、CRUD、启动加载、热更新 registry、session 自动匹配与手动启用/禁用。
- skill 已不只是 prompt 附件，而是会真实影响 Planner、Analyst / Reflection / Report 提示，以及 session 级工具 allowlist。

23.2 当前仍存在的关键边界

需要诚实面对的点

第一，审批虽然已经进入动作级执行流，但恢复粒度主要还是批次边界。第二，tool audit 和 budget state 已经 crash-safe 强化，但还不是全量法证平台。第三，Harness supervisor 已落地最小版本，但还没形成成熟的多 worker 调度和租约竞争体系。第四，MCP proxy 已存在，但默认 Planner 仍不以 MCP 为主要节点来源。第五，计费系统仍受上游 usage/billing 可观测性限制，不能吹成绝对精确。第六，skill 当前主要影响提示和工具边界，还没有演进到“自定义执行器 / 自定义 DAG 节点类型”这一层。

23.3 建议的三阶段演进

1. Phase 1: 失败语义显式化、tool audit 持久化、会话级 budget breaker、session budget state。
2. Phase 2: 把最小可用 Harness 演进成多 worker supervisor，并把批次边界恢复做得更稳；同时把 skill 从“提示和工具约束层”继续推进到更精细的 session 策略解释和 API 治理。
3. Phase 3: 扩 Planner 的 MCP DAG 生成策略、细化 per-tool capability、接入外部审批系统信号，并评估 skill 是否需要升级成可挂载的自定义节点模板体系。

23.4 最终的面试总结句

收口

如果只看主链路，这个项目已经是一个能工作的研究型 Agent。真正让我认为它有工程价值的地方，在于它没有停留在“会规划、会搜索、会写报告”这一步，而是开始把失败语义、预算、审计、审批和外部工具边界都纳入系统设计。也正因为如此，它既能讲 Agent 能力，也能讲 Agent 治理。